

Smarter Regular Expression Matching

From Theory to Tools

Ilian Kohl

September 23, 2026

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Problem Statement	5
1.3	Contributions	5
1.4	Thesis Outline	6
2	Regular Expressions	7
2.1	Syntax	7
2.2	Semantics	8
3	Derivatives and Partial Derivatives	11
3.1	Naive Matching	11
3.2	Brzozowski Derivatives	14
3.3	Antimirov Partial Derivatives	21
4	Matching versus Parsing	28
4.1	Matching Ambiguity	28
4.2	Parse Trees as Matching Evidence	29
4.3	Disambiguation Problem	31
5	Derivative-Based Parser	33
5.1	From Answer to Evidence	33
5.1.1	Which Tree	34
5.2	Tree Construction	34
5.2.1	The Empty-Word Tree	34
5.2.2	Reinserting a Character	35
5.2.3	The Parser	36
5.2.4	No Simplification	36
5.3	Implementation: <code>deriv_std</code>	36
5.3.1	What the Parser Needs	36
5.3.2	<code>mk_eps</code>	37
5.3.3	<code>inject</code> : The Direct Rules	38
5.3.4	<code>inject</code> : Concatenation	40
5.3.5	Two Drivers	41
5.3.6	Trees and Flattening	43
5.4	Worked Example: <code>deriv_std</code>	43
5.4.1	Forward Pass	43
5.4.2	The Empty-Word Tree	44

5.4.3	The Two Injections	44
5.4.4	What It Cost	45
5.5	Bit-Coded Construction	45
5.5.1	Trees as Bit-Strings	46
5.5.2	Annotated Expressions	46
5.5.3	The Annotated Derivative	47
5.5.4	Extracting the Bits	47
5.5.5	Simplification	48
5.6	Implementation: <code>deriv_bc</code>	50
5.6.1	The Annotated Type	50
5.6.2	<code>internalize</code> and <code>fuse</code>	50
5.6.3	The Derivative	52
5.6.4	<code>mk_eps_bc</code> and <code>decode</code>	53
5.6.5	The Driver	55
5.7	Worked Example: <code>deriv_bc</code>	55
5.7.1	Before the Input	55
5.7.2	Two Steps	55
5.7.3	Bits to Tree	56
5.7.4	The Star Case	57
5.8	Comparison	57
6	Partial-Derivative-Based Parser	59
6.1	From One Expression to a Frontier	59
6.1.1	Where the Choice Is Actually Made	60
6.1.2	A Family of Witnesses	61
6.2	The Bit-Coded Construction	61
6.2.1	The Running Example, Strand by Strand	62
6.3	Rust Implementation: <code>pderiv_bc</code>	64
6.4	Rust Implementation: <code>pderiv_std</code>	64
6.4.1	What Each Strand Carries	65
6.4.2	One Table, Two Representations	65
6.4.3	Why the Smart Constructor Cannot Be Used Here	66
6.4.4	The Same Example, and What the Trace Cannot Show	66
6.5	Frontier Growth	67
6.6	The Two Policies Compared	69
7	External versus Internal Regex	71
7.1	Two ASTs, One Lowering Step	71
7.2	Parsing Surface Syntax	72
7.3	Translating <code>ExtPat</code> into <code>Regex</code>	74
7.4	The Cost of Translation	76
8	Evaluation	79
8.1	Compared Engines	79
8.1.1	Rust's <code>regex</code> Crate	79
8.1.2	RE2	80
8.2	Testing Strategy	80
8.3	Dataset Preparation	80
8.3.1	Substring Search: How the Corpus Is Run	82

8.4	Correctness Verification	83
8.4.1	Paper Examples Reproduced	83
8.4.2	Cross-Implementation Equivalence	84
8.5	Performance Analysis	85
8.5.1	Stack Depth: Recursive vs. Iterative Construction	85
8.5.2	Heap Allocation: <code>deriv_std</code> vs. <code>deriv_bc</code>	85
8.5.3	Derivative-Based vs. Partial-Derivative-Based	85
8.5.4	Pathological Inputs	85
8.5.5	Benchmark Results	85
8.6	Comparison with Existing Engines	86
8.6.1	Performance	86
8.6.2	Standardized Benchmarking via rebar	86
8.7	Optimization Challenges and Insights	86
8.8	Discussion of Results	87
9	Related Work and Conclusion	88
9.1	Related Work	88
9.2	Conclusion	88
A	Code	91
A.1	Core Types	91
A.2	Shared Algorithms	92
A.3	Matchers	94
A.4	The <code>deriv_std</code> Parser	94
A.5	The <code>deriv_bc</code> Parser	95
A.6	The <code>pderiv_bc</code> Parser	96
A.7	The <code>pderiv_std</code> Parser	98
A.8	Frontend	100
B	Tests	103
B.1	Shared Fixtures	103
B.2	Paper Examples Reproduced	104
B.3	Cross-Implementation Equivalence	105
B.4	Frontend	106
C	Diagnostics	107
C.1	<code>deriv_std_rec, match</code>	107
C.2	<code>deriv_bc, match</code>	109
C.3	<code>pderiv_std, match</code>	110
C.4	<code>pderiv_bc, match</code>	112
C.5	<code>deriv_std_rec, no match</code>	113
C.6	<code>deriv_bc, no match</code>	114
C.7	<code>pderiv_std, no match</code>	115
C.8	<code>pderiv_bc, no match</code>	117

1 Introduction

1.1 Motivation

Regular expressions are a fundamental formalism for describing regular languages and are among the most widely used abstractions in computer science. They underpin applications including lexical analysis, input validation, log processing, search tools, and network security. Across these applications, the problem is regular expression matching. Given a regular expression and an input string, determine whether the string belongs to the language denoted by the expression. Formally, this is the *membership problem*.

For example, the expression $(ab|a)^*$ matches "aba", as it decomposes into "ab" followed by "a", but does not match "abb", since no sequence of subexpressions can produce it.

For many applications, a Boolean answer is enough. An input validator may only need to accept or reject a password or URL. Others require more. For instance, a lexer must determine how input decomposes into tokens. Capturing groups, schema validation, and information extraction require identifying which input parts correspond to parts of the expression. Recognition alone is not enough; the matcher must also recover the *structure*.

Efficiently solving these problems remains an active research area. The choice of algorithm influences performance, memory consumption, and security. Once matching becomes parsing, the way in which match structure is recovered also influences correctness and raises the question of how to handle ambiguity when multiple matches are possible.

Existing approaches include backtracking and finite-automata-based techniques, such as nondeterministic (NFA) and deterministic (DFA) automata. These approaches have different trade-offs in implementation complexity, memory consumption, and worst-case performance. Backtracking engines are flexible and widely used, but may exhibit exponential running time on constructed inputs, leading to regular expression denial-of-service (ReDoS) vulnerabilities [Cox, 2007]. Automata-based approaches provide stronger worst-case guarantees but may require preprocessing or suffer from state-space explosion.

Derivative-based algorithms provide an alternative by transforming the regular expression as input symbols are consumed. Each derivative represents the language remaining after an input symbol has been consumed. By maintaining a correspondence between an expression and its successive residuals, derivative-based methods provide an elegant foundation for regular expression matching and, with suitable extensions, structured parsing.

However, derivative-based techniques are less common in practical regular expression tooling. Their use for practical matching, parsing, and diagnostic tools motivates this thesis: "From Theory to Tools".

1.2 Problem Statement

Derivative-based approaches provide a principled foundation for regular expression matching and parsing. Brzozowski introduced regular expression derivatives as a way of transforming a regular expression after consuming an input symbol [Brzozowski, 1964]. Antimirov later introduced partial derivatives, which represent sets of residual expressions and are closely related to nondeterministic automata [Antimirov, 1996].

Building on Brzozowski’s derivatives, Sulzmann and Lu developed a derivative-based algorithm for POSIX regular expression parsing [Sulzmann and Lu, 2014]. Their work also presents a bit-coded variant that represents parse information as a compact bit sequence, which can be decoded after a successful match rather than maintaining an explicit parse tree throughout the computation. A Haskell implementation of the POSIX parser is available in Sulzmann’s *Programming Paradigms* teaching material [Sulzmann, 2026].

Sulzmann and Lu also studied partial derivatives in the context of regular expression sub-matching [Sulzmann and Lu, 2012]. While sub-matching is not the focus of this thesis, their work provides relevant background for the partial-derivative approach. The *Programming Paradigms* teaching material also presents a bit-coded partial-derivative-based parser in Haskell.

These constructions have primarily been presented in a theoretical and functional programming setting, leaving open practical questions about execution time, memory consumption, and data representation.

This thesis investigates these questions through implementations in Rust. As a systems programming language, Rust provides fine-grained control over memory allocation and data representation, allowing the derivative-based and partial-derivative-based parsers, including their bit-coded variants, to be evaluated in practice.

The implementation also provides traced versions of the parsing algorithms that record intermediate derivative computations during matching. These traces show the transformations performed by the algorithms and help explain successful and unsuccessful matches.

1.3 Contributions

The following contributions are being made:

- **Rust implementation of derivative-based parsers.** The thesis implements derivative-based and partial-derivative-based parsing constructions in Rust, including direct and bit-coded variants. The implementations are based on the constructions described in the literature and the corresponding Haskell material, resulting in four parser variants in total.
- **Empirical evaluation of the parser variants.** The four implementations are evaluated with respect to execution time and memory consumption using synthetic benchmarks and a corpus of real-world regular expressions from Snort/Suricata, SpamAssassin, and RegexLib, following the source categories investigated by Mamouras et al. [Mamouras et al., 2024]. They are also compared with Rust’s `regex` crate and Google’s RE2.

1.4 Thesis Outline

The remainder of this thesis follows the parsing constructions from their formal concepts through their Rust implementation to their evaluation. Theory and implementation are presented together.

- **Chapter 2 - Regular Expressions.** This chapter introduces the syntax and semantics of regular expressions and the Rust representation used throughout the implementation.
- **Chapter 3 - Derivatives.** This chapter introduces Brzozowski’s derivatives and Antimirov’s partial derivatives, together with their basic matching algorithms and Rust implementations.
- **Chapter 4 - Matching vs. Parsing.** This chapter distinguishes regular expression matching from parsing and introduces the disambiguation problem.
- **Chapter 5 - Derivative Parser.** This chapter presents Sulzmann and Lu’s derivative-based POSIX parser and its Rust implementation in direct and bit-coded variants.
- **Chapter 6 - Partial-Derivative Parser.** This chapter presents the partial-derivative-based parser and its Rust implementation in direct and bit-coded variants. It also discusses the differences between the two derivative-based approaches.
- **Chapter 7 - External versus Internal Syntax.** This chapter describes the parsing of regular expression strings into an external AST and their translation into the internal representation used by the parsers.
- **Chapter 8 - Evaluation.** This chapter evaluates the four parser variants with respect to execution time, memory consumption, and correctness. The evaluation uses synthetic benchmarks and a corpus of real-world regular expressions, and compares the implementations with Rust’s `regex` crate and Google’s RE2. It also examines optimization challenges and the insights gained from the evaluation, including the derivative traces produced by the implementations.
- **Chapter 9 - Conclusion.** Finally, this chapter summarizes the findings of the thesis and discusses the practical implications of the results and directions for future work.

2 Regular Expressions

2.1 Syntax

Let Σ be an alphabet. Regular expressions over Σ are defined inductively by the grammar

$$r, s ::= x \mid \varepsilon \mid \emptyset \mid r + s \mid r \cdot s \mid r^* \quad (x \in \Sigma). \quad (2.1)$$

Here, x denotes a literal symbol, ε the expression matching only the empty word, and $\emptyset$ the expression matching no word. The operators $+$, $\cdot$, and $*$ denote alternation, concatenation, and Kleene star, respectively. Concatenation is often written implicitly, so rs denotes $r \cdot s$.

Unless parentheses are shown explicitly, Kleene star binds most tightly, followed by concatenation, followed by alternation. Thus, for example, ab^*c is parsed as $(a \cdot (b^*)) \cdot c$, while $a + bc^*$ is parsed as $a + (b \cdot (c^*))$.

At the language level, $+$ and $\cdot$ are associative: different groupings denote the same language. For example, $L((r + s) + t) = L(r + (s + t))$ and $L((r \cdot s) \cdot t) = L(r \cdot (s \cdot t))$. The grouping determines the *syntax tree* on which the algorithms operate. We therefore use left association for both operators: $r + s + t$ means $(r + s) + t$, while rst means $(rs)t$.

The grammar omits operators such as one-or-more, optional, and bounded repetition (r^+ , $r^?$, and $r^{\{m,n\}}$). These operators can be expressed using the six core constructs. For example, $r^+ = r \cdot r^*$ and $r^? = r + \varepsilon$. Bounded repetition can be expanded as

$$r^{\{m,n\}} = \underbrace{r \cdot \dots \cdot r}_{m \text{ times}} \cdot \underbrace{(r + \varepsilon) \cdot \dots \cdot (r + \varepsilon)}_{n-m \text{ times}}.$$

Thus, these convenience operators do not add expressive power to the core syntax. In the implementation, expressions using such operators are translated into the six core constructs before they are processed by the parsing algorithms; the translation from external regular expression syntax to this internal representation is described in Chapter 7.

As a running example, consider the regular expression for binary strings ending in "01". Over $\Sigma = \{0, 1\}$, let

$$r_{01} = (0 + 1)^* \cdot 0 \cdot 1. \quad (2.2)$$

The expression consists of a Kleene star applied to the alternation $0 + 1$, followed by the literals 0 and 1. By the precedence and left-association rules, its syntax tree is $((0 + 1)^* \cdot 0) \cdot 1$. The algorithms operate on this syntax tree, with each node corresponding to one of the constructs in the grammar.

2.2 Semantics

The syntax above describes the *shape* of a regular expression, while its semantics determines the language it denotes. A *word* over Σ is a finite sequence of symbols from Σ . The set of all words over Σ , including the empty word ε , is denoted by Σ^* .

For a regular expression r , write $L(r)$ for the language denoted by r , where $L(r) \subseteq \Sigma^*$. The semantics is defined by structural recursion on r :

$$\begin{aligned}
 L(x) &= \{x\}, \\
 L(\varepsilon) &= \{\varepsilon\}, \\
 L(\emptyset) &= \emptyset, \\
 L(r + s) &= L(r) \cup L(s), \\
 L(r \cdot s) &= \{vw \mid v \in L(r), w \in L(s)\}, \\
 L(r^*) &= \{\varepsilon\} \cup \{w_1 \cdots w_n \mid n \geq 1, w_1, \dots, w_n \in L(r)\}.
 \end{aligned} \tag{2.3}$$

Some simple examples are shown in Table 2.1.

Expression	Language
a	$\{a\}$
ε	$\{\varepsilon\}$
$\emptyset$	$\emptyset$
$a + b$	$\{a, b\}$
ab	$\{ab\}$
a^*	$\{\varepsilon, a, aa, aaa, \dots\}$

Table 2.1: Examples of regular expressions and their denoted languages.

A word w *matches* a regular expression r when $w \in L(r)$. Determining whether this holds is the *membership problem*.

For the running example, $L(0 + 1) = \{0, 1\}$, so $L((0 + 1)^*) = \{0, 1\}^*$. Concatenating this language with $L(0) = \{0\}$ and $L(1) = \{1\}$ gives

$$L(r_{01}) = \{w \cdot 0 \cdot 1 \mid w \in \{0, 1\}^*\}. \tag{2.4}$$

Thus, $L(r_{01})$ is exactly the set of binary strings ending in "01", including "01", "001", and "1101". The empty string is not in $L(r_{01})$, since every word in the language ends with the two symbols 0 and 1.

An expression therefore has both a syntactic structure and a meaning: the syntax determines how the expression is constructed, while its semantics determines the language it denotes. Different syntax trees may denote the same language, even when the expressions themselves are not syntactically identical. Thus, equality of regular expressions as syntax objects is distinct from equality of the languages they denote. This distinction is important for the algorithms developed later: the algorithms manipulate regular expressions and their syntax trees, while $L(r)$ gives the set of words represented by the expression.

In Rust. The grammar above is represented directly as a Rust enum [The Rust Project Developers, 2026b,f], shown in Figure 2.1, with one variant for each syntactic construct.

```
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub enum Regex {
    Phi,                // empty language
    Eps,                // empty word
    Lit(char),          // literal x
    Seq(Box<Regex>, Box<Regex>), // r . s
    Alt(Box<Regex>, Box<Regex>), // r + s
    Star(Box<Regex>),    // r*
```

Figure 2.1: The internal regular-expression type, with one variant for each construct of Equation (2.1). This is the representation every algorithm in this thesis operates on.

The derived traits also support the operations used by the later algorithms. `Clone` allows expressions to be duplicated when a derivative needs to retain an existing subexpression, while `PartialEq` and `Eq` provide structural equality for comparing expression trees. `Hash` makes the same structural representation usable as a key in a `HashSet`, which is required by the partial-derivative matcher.

The variants correspond one to one with the grammar: `Phi`, `Eps` and `Lit` represent $\emptyset$, ε and literals, while `Seq`, `Alt` and `Star` represent concatenation, alternation and Kleene star. The type carries no operator precedence and no associativity, because both were already resolved when the syntax tree was built: a `Seq` node holds two children and says nothing about how the original notation grouped them.

For example, the running example $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$ is represented by an expression tree whose root is the concatenation of $(0 + 1)^* \cdot 0$ and 1, following the left-associated grouping from Section 2.1. Its structure is:

$$\text{Seq}(\text{Seq}(\text{Star}(\text{Alt}(0, 1)), 0), 1). \quad (2.5)$$

The same structure is represented in Rust by nested enum values. The outermost `Seq` represents the final concatenation with 1, and its left child represents $(0 + 1)^* \cdot 0$. Thus, the Rust value stores the parsed structure of the expression directly, rather than only its textual notation such as $(0 + 1)^* \cdot 0 \cdot 1$. Since a regular expression can contain other regular expressions, these nested values allow functions on `Regex` to be defined recursively as well.

The recursive variants use `Box<Regex>` because Rust requires recursive types to have a finite size. A definition such as `Seq(Regex, Regex)` would have no finite size, since each `Regex` could contain another recursively. `Box<Regex>` provides the required indirection by storing the recursive value separately [The Rust Project Developers, 2026a,h].

This explicit indirection differs from the Haskell specification on which the implementation is based. Haskell allows recursive algebraic data types without an explicit `Box`-like constructor, whereas Rust requires the indirection because enum sizes must be known at compile time. The Rust representation therefore makes the heap-based structure of recursive expressions explicit, which is relevant to memory behaviour.

Constructors. Constructing recursive expressions directly requires explicit `Box::new` calls, so the implementation provides four constructors that encapsulate the boxing, shown in Figure 2.2. All subsequent implementations in Chapter 3 use them to construct regular expressions.

```
impl Regex {

    pub fn seq(r: Regex, s: Regex) -> Regex {
        Regex::Seq(Box::new(r), Box::new(s))
    }

    pub fn alt(r: Regex, s: Regex) -> Regex {
        Regex::Alt(Box::new(r), Box::new(s))
    }

    pub fn star(r: Regex) -> Regex {
        Regex::Star(Box::new(r))
    }

    pub fn lit(c: char) -> Regex {
        Regex::Lit(c)
    }

}
```

Figure 2.2: The four constructors. Each builds the corresponding `Regex` variant and performs the boxing required by the recursive representation.

The constructors are structural: they perform no simplification or semantic transformation. For example, `Regex::seq(Regex::Eps, r)` constructs a `Seq` node with an `Eps` left child rather than simplifying to `r`. This differs from the **smart** constructor introduced later, which can inspect its operands and avoid redundant expressions.

The constructors take their arguments by value, so building an expression bottom-up moves its subexpressions into the boxes rather than copying them. The running example is constructed as follows:

```
Regex::seq(
    Regex::seq(
        Regex::star(Regex::alt(Regex::lit('0'), Regex::lit('1'))),
        Regex::lit('0'),
    ),
    Regex::lit('1'),
)
```

This value has the structure of Equation (2.5) and is produced by the frontend of Chapter 7 for the surface pattern `"(0|1)*01"`.

3 Derivatives and Partial Derivatives

Chapter 2 introduced the syntax and semantics of regular expressions. This chapter introduces three algorithms for deciding whether $w \in L(r)$.

The first is a naive recursive matcher that follows the structure of the regular-expression semantics. The second uses Brzozowski derivatives to transform a regular expression as input symbols are consumed. The third uses Antimirov’s partial derivatives, representing the possible residual expressions as a set rather than combining them into a single expression. All three approaches are implemented in Rust.

All three matchers `match_naive`, `match_deriv`, and `match_pderiv` share the signature `fn(&str, &Regex) -> bool`, so the same expression and input can be processed by any of them.

3.1 Naive Matching

A direct way to decide whether $w \in L(r)$ is to interpret the regular-expression semantics as recursive matching rules. The resulting naive matcher follows the structure of the expression and searches for possible decompositions of the input.

Let w be given as an indexable sequence of characters, and let $w[i : j]$ denote the substring beginning at position i and ending immediately before position j . A recursive procedure `match(w, i, j, r)` determines whether $w[i : j] \in L(r)$ by following the structure of r :

$$\begin{aligned}
 \text{match}(w, i, j, \varepsilon) &\iff i = j, \\
 \text{match}(w, i, j, x) &\iff j = i + 1 \wedge w[i] = x, \\
 \text{match}(w, i, j, r + s) &\iff \text{match}(w, i, j, r) \vee \text{match}(w, i, j, s), \\
 \text{match}(w, i, j, r \cdot s) &\iff \exists k \in [i, j]. \text{match}(w, i, k, r) \wedge \text{match}(w, k, j, s), \\
 \text{match}(w, i, j, r^*) &\iff i = j \vee \exists k \in (i, j]. \text{match}(w, i, k, r) \wedge \text{match}(w, k, j, r^*).
 \end{aligned} \tag{3.1}$$

Membership of the complete word w is then given by `match(w, 0, |w|, r)`. Each rule directly corresponds to the semantics of the respective regular-expression construct.

The direct correspondence with the semantics comes at a cost. In particular, the concatenation rule does not prescribe where the input should be split. It asks whether *some* split point k exists such that r matches $w[i : k]$ and s matches $w[k : j]$. The naive matcher therefore searches the possible values of k .

For example, consider matching $r \cdot s = (a \cdot a) \cdot a$ against $w[0 : 3]$, where $w = \mathbf{aaa}$. Here, $r = a \cdot a$ and $s = a$, so the recursive rule requires the matcher to consider the possible split points $k \in \{0, 1, 2, 3\}$:

$$\begin{aligned}
& \underbrace{\text{match}(w, 0, 0, a \cdot a)}_{a \cdot a \text{ against } w[0:0]=\epsilon} \wedge \underbrace{\text{match}(w, 0, 3, a)}_{a \text{ against } w[0:3]=\mathbf{aaa}} \\
\vee & \underbrace{\text{match}(w, 0, 1, a \cdot a)}_{a \cdot a \text{ against } w[0:1]=\mathbf{a}} \wedge \underbrace{\text{match}(w, 1, 3, a)}_{a \text{ against } w[1:3]=\mathbf{aa}} \\
\vee & \underbrace{\text{match}(w, 0, 2, a \cdot a)}_{a \cdot a \text{ against } w[0:2]=\mathbf{aa}} \wedge \underbrace{\text{match}(w, 2, 3, a)}_{a \text{ against } w[2:3]=\mathbf{a}} \\
\vee & \underbrace{\text{match}(w, 0, 3, a \cdot a)}_{a \cdot a \text{ against } w[0:3]=\mathbf{aaa}} \wedge \underbrace{\text{match}(w, 3, 3, a)}_{a \text{ against } w[3:3]=\epsilon}.
\end{aligned}$$

The first alternative fails because $a \cdot a$ cannot match the empty substring $w[0 : 0]$. The matcher therefore proceeds to $k = 1$, where $a \cdot a$ would have to match the one-character substring $w[0 : 1] = \mathbf{a}$; this also fails. At $k = 2$, however, the first component receives $w[0 : 2] = \mathbf{aa}$ and the second receives $w[2 : 3] = \mathbf{a}$, so both recursive calls succeed. The final split $k = 3$ is therefore never reached. The successful match is obtained only after the preceding split points have been tried.

The successful alternative produces two recursive calls:

$$\underbrace{\text{match}(w, 0, 2, a \cdot a)}_{a \cdot a \text{ against } w[0:2]=\mathbf{aa}} \quad \text{and} \quad \underbrace{\text{match}(w, 2, 3, a)}_{a \text{ against } w[2:3]=\mathbf{a}}.$$

The second call succeeds immediately because the substring contains exactly one character. The first call is itself a concatenation match, so the same rule is applied again. This time the possible split points are $k \in \{0, 1, 2\}$:

$$\begin{aligned}
& \underbrace{\text{match}(w, 0, 0, a)}_{a \text{ against } w[0:0]=\epsilon} \wedge \underbrace{\text{match}(w, 0, 2, a)}_{a \text{ against } w[0:2]=\mathbf{aa}} \\
\vee & \underbrace{\text{match}(w, 0, 1, a)}_{a \text{ against } w[0:1]=\mathbf{a}} \wedge \underbrace{\text{match}(w, 1, 2, a)}_{a \text{ against } w[1:2]=\mathbf{a}} \\
\vee & \underbrace{\text{match}(w, 0, 2, a)}_{a \text{ against } w[0:2]=\mathbf{aa}} \wedge \underbrace{\text{match}(w, 2, 2, a)}_{a \text{ against } w[2:2]=\epsilon}.
\end{aligned}$$

Here, $k = 0$ fails because the first a cannot match the empty substring. The split $k = 1$ succeeds because the two a expressions match $w[0 : 1] = \mathbf{a}$ and $w[1 : 2] = \mathbf{a}$, respectively. The split $k = 2$ is therefore not reached. The original match consequently succeeds through the nested sequence of split points $k = 2$ and $k = 1$.

This shows that concatenation can cause the matcher to explore several split points recursively before finding a successful decomposition. Each recursive call repeats the same search on a smaller substring.

In Rust. The specification’s $\text{match}(w, i, j, r)$ becomes a recursive helper over an indexable word,

```
fn match_naive_range(word: &[char], i: usize, j: usize,
                    r: &Regex) -> bool
```

with the public entry point collecting the input once and calling it over the whole word:

```
pub fn match_naive(input: &str, r: &Regex) -> bool {
    let chars: Vec<char> = input.chars().collect();
    let len = chars.len();
    match_naive_range(&chars, 0, len, r)
}
```

The two rules that require a search become the two arms shown in Figure 3.1.

```
Regex::Seq(r, s) => {
    (i..=j).any(|k| {
        match_naive_range(word, i, k, r)
        && match_naive_range(word, k, j, s)
    })
}
Regex::Star(r) => {
    i == j
    || (i + 1..=j).any(|k| {
        match_naive_range(word, i, k, r)
        && match_naive_range(word, k, j, &Regex::star(*r.clone()))
    })
}
```

Figure 3.1: The searching arms of the naive matcher. The $\exists k$ of Equation (3.1) becomes a range and `any`, which stops at the first k that succeeds.

The existential quantifier of Equation (3.1) becomes a range and `any`, which short-circuits at the first successful k . Thus, the worked example above stops once $k = 2$ is found. The interval bounds correspond directly to the specification: concatenation searches $(i..=j)$, matching $\exists k \in [i, j]$, while star searches $(i + 1..=j)$, matching $\exists k \in (i, j]$. Excluding $k = i$ prevents the recursive call from considering the same interval again when r matches the empty substring.

The matcher indexes `&[char]` rather than `&str`, because Rust strings are UTF-8 and cannot be indexed directly by character position. The input is therefore collected into a `Vec<char>` once at the entry point above, and addressed by position from then on. The star arm also reconstructs the expression with `Regex::star(*r.clone())` on each recursive call, since the recursive call requires an owned `Regex`. Each recursive iteration therefore clones the star body.

Although inefficient, this matcher is useful as a reference implementation because it follows the regular-expression semantics without any intermediate transformation.

The derivative approach takes a different approach. Rather than searching for a decomposition of w while keeping r fixed, it transforms r as each character of w is consumed.

The resulting expression describes the language that can still match the remaining suffix.

3.2 Brzowski Derivatives

Brzowski's derivative of r with respect to a symbol x , written $r \setminus x$, is the expression describing exactly what remains of r 's language after consuming the leading symbol x of a word $w \in L(r)$ [Brzowski, 1964]:

$$L(r \setminus x) = \{ w \mid xw \in L(r) \}. \quad (3.2)$$

For $w = x_1 x_2 \cdots x_n$, consume one symbol at a time by repeatedly taking derivatives:

$$r, \quad r \setminus x_1, \quad (r \setminus x_1) \setminus x_2, \quad \dots$$

After all symbols have been consumed, the remaining input is ε . Thus, the resulting expression must accept ε for the original word to belong to $L(r)$. This is the question of whether the expression is *nullable*. Let $n(r)$ be true exactly when $\varepsilon \in L(r)$:

$$\begin{array}{llll} n(\emptyset) = \text{false} & (\mathbf{N}_\emptyset) & n(r + s) = n(r) \vee n(s) & (\mathbf{N}_+) \\ n(\varepsilon) = \text{true} & (\mathbf{N}_\varepsilon) & n(r \cdot s) = n(r) \wedge n(s) & (\mathbf{N}_{\text{seq}}) \\ n(x) = \text{false} & (\mathbf{N}_{\text{lit}}) & n(r^*) = \text{true} & (\mathbf{N}_*) \end{array} \quad (3.3)$$

The empty language $\emptyset$ is not nullable because it contains no words, including ε , while a literal cannot match the empty word. ε and r^* always match ε . An alternation is nullable when either branch is nullable, and a concatenation when both components are.

In Rust. The definition is transcribed one `Regex` variant at a time, using the enum of Figure 2.1. The case split of Equation (3.3) becomes the arms of a `match`, shown in Figure 3.2.

```
pub fn nullable(r: &Regex) -> bool {
    match r {
        Regex::Phi => false,
        Regex::Eps => true,
        Regex::Lit(_) => false,
        Regex::Alt(r, s) => nullable(r) || nullable(s),
        Regex::Seq(r, s) => nullable(r) && nullable(s),
        Regex::Star(_) => true,
    }
}
```

Figure 3.2: `nullable`, transcribing Equation (3.3). The six arms are $\mathbf{N}_\emptyset$, $\mathbf{N}_\varepsilon$, $\mathbf{N}_{\text{lit}}$, $\mathbf{N}_+$, $\mathbf{N}_{\text{seq}}$ and $\mathbf{N}_*$.

The correspondence is exact, and Rust's exhaustiveness check on `match` is what keeps it that way [The Rust Project Developers, 2026c,g]: a function defined by matching on

Regex must account for every construct of the grammar, so a missing nullability case is a compile error and not a silent gap.

Three details are worth naming. The expression is borrowed, since **nullable** only asks a question about r and every caller needs r afterwards. The two underscores say that $\mathbf{N}_{\text{lit}}$ and $\mathbf{N}_*$ ignore their operands: a literal is never nullable whatever character it holds, and a star is always nullable whatever its body. And $||$ and $\&\&$ short-circuit, so a nullable left disjunct means the right one is never visited. That does not change the answer, because the function is pure, but it does mean the cost of **nullable** depends on where the decisive subexpression sits.

The derivative rules specify the expression that remains after consuming x :

$$\begin{aligned}
\emptyset \backslash x &= \emptyset & (D_{\emptyset}) \\
\varepsilon \backslash x &= \emptyset & (D_{\varepsilon}) \\
y \backslash x &= \begin{cases} \varepsilon & x = y \\ \emptyset & x \neq y \end{cases} & (D_{\text{lit}}) \\
(r + s) \backslash x &= (r \backslash x) + (s \backslash x) & (D_{+}) \\
r^* \backslash x &= (r \backslash x) \cdot r^* & (D_{*}) \\
(r \cdot s) \backslash x &= \begin{cases} (r \backslash x) \cdot s + s \backslash x & n(r) = \text{true} \\ (r \backslash x) \cdot s & n(r) = \text{false} \end{cases} & \begin{matrix} (D_{\text{seq}}^n) \\ (D_{\text{seq}}) \end{matrix}
\end{aligned} \tag{3.4}$$

The empty language contains no words, so it cannot match a word beginning with x and $\emptyset \backslash x = \emptyset$. Likewise, ε contains no symbol to consume, so $\varepsilon \backslash x = \emptyset$. For a literal y , consuming x leaves ε when $x = y$ and no possible match otherwise. For $r + s$, the symbol may come from either alternative, so both derivatives are preserved. For r^* , the symbol is consumed by one occurrence of r , after which r^* may still match the remainder.

Concatenation is the only derivative rule that depends on nullability. If r is not nullable, r must consume the leading x , leaving $r \backslash x$ as the remainder of r while s remains untouched. If r is nullable, it may either consume x , as in the non-nullable case, giving $(r \backslash x) \cdot s$, or r matches ε and is finished, so s consumes x and leaves $s \backslash x$. Hence the nullable case produces the alternation $(r \backslash x) \cdot s + s \backslash x$.

In Rust. Figure 3.3 transcribes Equation (3.4) the same way, one variant at a time.

```

pub fn deriv(r: &Regex, x: char) -> Regex {
  match r {
    Regex::Phi => Regex::Phi,
    Regex::Eps => Regex::Phi,
    Regex::Lit(c) => {
      if *c == x { Regex::Eps } else { Regex::Phi }
    }
    Regex::Alt(r1, r2) => {
      Regex::alt(deriv(r1, x), deriv(r2, x))
    }
    Regex::Seq(r1, r2) => {
      if nullable(r1) {
        let dr1 = Regex::seq(deriv(r1, x), *r2.clone());
        Regex::alt(dr1, deriv(r2, x))
      } else {
        Regex::seq(deriv(r1, x), *r2.clone())
      }
    }
    Regex::Star(r1) => {
      Regex::seq(deriv(r1, x), Regex::star(*r1.clone()))
    }
  }
}

```

Figure 3.3: `deriv`, transcribing Equation (3.4). The six arms are $D_\emptyset$, D_ϵ , D_{lit} , D_+ , the $D_{\text{seq}}^n/D_{\text{seq}}$ pair, and D_* .

The signature is where `deriv` differs from `nullable`. It borrows its input and returns an owned `Regex`, because a derivative is a new expression and not a fact about an existing one. Every arm therefore builds, and it builds with the constructors of Figure 2.2: `Regex::alt`, `Regex::seq` and `Regex::star` do the boxing, so the arms read like the equation.

The `if nullable(r1)` is the case split of Equation (3.4), with the branches taken in the order D_{seq}^n then D_{seq} . This is the only place a derivative rule consults nullability, and it is also the only place a derivative can produce a larger expression than it consumed, since the nullable branch returns an alternation where the other returns a concatenation.

The expression `*r2.clone()` is the cost of that growth made explicit. In the arm, `r2` is a borrowed `Box<Regex>`, so `.clone()` duplicates the entire subtree it points at and `*` moves the copy out of its box. Both branches pay it once, because s appears untouched in both, and the nullable branch pays for `deriv(r2, x)` on top. The star arm does the same with `*r1.clone()`, since r^* 's body appears both derived and underived in D_* . Every derivative of a concatenation or a star therefore reproduces a subtree, which is the mechanism behind the growth of Equation (3.12) below and the reason the matcher simplifies after every step.

Why the case split. Brzowski's original definition wrote the concatenation case as

$$x^{-1}(r \cdot t) = (x^{-1}r) \cdot t + o(r) \cdot x^{-1}t, \quad (3.5)$$

in the notation Antimirov uses, where $x^{-1}r$ is the derivative of r by x and $o(r)$ is the constant part of r , equal to ε when r is nullable and $\emptyset$ otherwise. Antimirov observed that deriving $(x + y)^*$ by x , then xx , then xxx , and so on under this rule gives “an infinite sequence of regular terms non-equivalent modulo $\mathcal{E}(ACI)$ ” [Antimirov, 1996, Remark 1.3, p. 298], and to restore finiteness, replaces it with his following [Antimirov, 1996, Definition 1.1, Eq. (21), p. 297]:

$$x^{-1}(r \cdot t) = \text{if } o(r) = 0 \text{ then } (x^{-1}r) \cdot t \text{ else } (x^{-1}r) \cdot t + x^{-1}t \text{ fi} \quad (3.6)$$

This is the same rule as (3.4): $x^{-1}r$ is written $r \setminus x$ there and t is written s , while $o(r) = 0$ is the non-nullable case $n(r) = \text{false}$. Its two branches are therefore D_{seq} and D_{seq}^n .

Worked example: single-symbol matching. Consider x^* against the word $\mathbf{xx}$. Writing $r \xrightarrow{x} r \setminus x$ for one derivative step,

Step 1. The expression is a star, so D_* applies with body $r = x$:

$$\begin{aligned} x^* \setminus \mathbf{x} &= (x \setminus \mathbf{x}) \cdot x^* & D_*, r = x \\ &= \varepsilon \cdot x^* & D_{\text{lit}}, \text{ since } \mathbf{x} = x \end{aligned} \quad (3.7)$$

Step 2. The expression is now the concatenation $\varepsilon \cdot x^*$, and deriving it consumes the second $\mathbf{x}$. The result is an alternation because D_{seq}^n applies with $r = \varepsilon$ and $s = x^*$, its nullable case being selected since $n(r) = n(\varepsilon) = \text{true}$ by N_ε :

$$\begin{aligned} (\varepsilon \cdot x^*) \setminus \mathbf{x} &= (\varepsilon \setminus \mathbf{x}) \cdot x^* + x^* \setminus \mathbf{x} & D_{\text{seq}}^n, r = \varepsilon, s = x^* \\ &= \emptyset \cdot x^* + x^* \setminus \mathbf{x} & D_\varepsilon \\ &= \emptyset \cdot x^* + (x \setminus \mathbf{x}) \cdot x^* & D_*, r = x \\ &= \emptyset \cdot x^* + \varepsilon \cdot x^* & D_{\text{lit}} \end{aligned} \quad (3.8)$$

After both symbols are consumed, we check whether the result is nullable:

$$\begin{aligned} n(\emptyset \cdot x^* + \varepsilon \cdot x^*) &= n(\emptyset \cdot x^*) \vee n(\varepsilon \cdot x^*) & N_+, r = \emptyset \cdot x^*, s = \varepsilon \cdot x^* \\ &= (n(\emptyset) \wedge n(x^*)) \vee (n(\varepsilon) \wedge n(x^*)) & N_{\text{seq}}, \text{ twice} \\ &= (\text{false} \wedge \text{true}) \vee (\text{true} \wedge \text{true}) & N_\emptyset, N_*, N_\varepsilon \\ &= \text{true}. \end{aligned} \quad (3.9)$$

Thus x^* matches $\mathbf{xx}$.

Simplification. Every nullable concatenation makes D_{seq}^n introduce an alternation, and each alternation carries a copy of a subexpression that the next derivative step will carry further. Nothing in Equation (3.4) bounds this, so expressions are simplified after each derivative step, using identities that preserve the denoted language:

$$\begin{aligned}
\emptyset \cdot r &= \emptyset & (S_1) \\
\varepsilon \cdot r &= r & (S_2) \\
\emptyset + r &= r & (S_3) \\
r + r &= r & (S_4)
\end{aligned} \tag{3.10}$$

S_1 to S_3 remove units $\emptyset$ and ε wherever a derivative step has left one behind, and each holds equally with the unit on the right. S_4 collapses two syntactically identical summands into one, which is what keeps repeated alternations from doubling. The implementation applies all four recursively over the expression tree, matching on both sides, and additionally reduces ε^* and $\emptyset^*$ to ε . Applying them to Equation (3.8):

$$\begin{aligned}
\emptyset \cdot x^* + \varepsilon \cdot x^* &= \emptyset + \varepsilon \cdot x^* & S_1, \emptyset \cdot x^* &= \emptyset \\
&= \varepsilon \cdot x^* & S_3, \emptyset + \varepsilon \cdot x^* &= \varepsilon \cdot x^* \\
&= x^* & S_2, \varepsilon \cdot x^* &= x^*
\end{aligned} \tag{3.11}$$

The case split of Equation (3.6) repaired the finiteness result that Equation (3.5) had broken, but only up to the ACI-equivalence named in the quotation above, which Antimirov [1996, Section 1.1, p. 293] defines. It does not prevent expressions from growing syntactically. This can be seen by repeatedly deriving Antimirov’s example $(x + y)^*$ by the same symbol x , without simplifying between steps:

$$\begin{aligned}
&(x + y)^* && 4 \\
\overset{x}{\rightarrow} &(\varepsilon + \emptyset) \cdot (x + y)^* && 8 \\
\overset{x}{\rightarrow} &(\emptyset + \emptyset) \cdot (x + y)^* + (\varepsilon + \emptyset) \cdot (x + y)^* && 17 \\
\overset{x}{\rightarrow} &(\emptyset + \emptyset) \cdot (x + y)^* + ((\emptyset + \emptyset) \cdot (x + y)^* + (\varepsilon + \emptyset) \cdot (x + y)^*) && 26
\end{aligned} \tag{3.12}$$

The starting expression has four nodes: a star, an alternation and two literals. The first derivative places it behind a concatenation with $(\varepsilon + \emptyset)$, adding a concatenation, an alternation and two units, so $4 + 4 = 8$. The second is an alternation of two eight-node expressions of that shape, so $1 + 8 + 8 = 17$, and the third has the entire second as its right operand, so $1 + 8 + 17 = 26$. Each step from the second onwards adds exactly nine nodes, one alternation and one eight-node summand.

Those nine nodes are dead every time. D_{seq}^n fires at each step because the left factor is nullable, and the summand it adds, $(\emptyset + \emptyset) \cdot (x + y)^*$, can consume nothing, since $\emptyset$ matches no symbol. It derives to itself from then on while a fresh one joins at every step, so only the rightmost summand is ever live.

The identities remove exactly this. S_3 reduces $\varepsilon + \emptyset$ to ε and S_2 reduces $\varepsilon \cdot (x + y)^*$ to $(x + y)^*$, so the first derivative collapses back to the expression it started from. Under “derive by x , then simplify”, $(x + y)^*$ is a fixed point and the size stays at four at every step.

In Rust. The simplification identities are implemented recursively in the same structural manner as `nullable` and `deriv` above. Figure 3.4 shows the complete simplifier.

```

pub fn simplify(r: Regex) -> Regex {
  match r {
    Regex::Seq(r, s) => {
      let r = simplify(*r);
      let s = simplify(*s);
      match (&r, &s) {
        (Regex::Phi, _) | (_, Regex::Phi) => Regex::Phi,
        (Regex::Eps, _) => s,
        (_, Regex::Eps) => r,
        _ => Regex::seq(r, s),
      }
    }
    Regex::Alt(r, s) => {
      let r = simplify(*r);
      let s = simplify(*s);
      match (&r, &s) {
        (Regex::Phi, _) => s,
        (_, Regex::Phi) => r,
        _ if r == s => r,
        _ => Regex::alt(r, s),
      }
    }
    Regex::Star(r) => match simplify(*r) {
      Regex::Eps | Regex::Phi => Regex::Eps,
      r => Regex::star(r),
    },
    other => other,
  }
}

```

Figure 3.4: Implementation of the simplification rules in Equation (3.10).

Each inner arm implements an identity from Equation (3.10); the guard `_ if r == s` implements S_4 by structural equality via `PartialEq` (Section 2.2). Thus only syntactically identical summands are collapsed, while the recursive calls apply the identities bottom-up.

The ownership distinction between `deriv` and `simplify` is visible where the two are composed, inside the matcher below:

```
current = simplify(deriv(&current, c));
```

`deriv` borrows its input and constructs a new expression, whereas `simplify` takes ownership of that result. It can therefore return simplified subexpressions directly, as in `(Regex::Eps, _) => s`, without copying them.

Worked example: $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$. Recall r_{01} from Section 2.1. It is left-associated as $((0 + 1)^* \cdot 0) \cdot 1$, so the outer operands are $r = (0 + 1)^* \cdot 0$ and $s = 1$. We derive against $w = 101$ one symbol at a time.

1. **Derive by 1.** At the outer node, $r = (0 + 1)^* \cdot 0$ and $s = 1$. This r is not nullable:

$$n((0 + 1)^* \cdot 0) = \text{true} \wedge \text{false} = \text{false},$$

by N_{seq} , N_* and N_{lit} . Thus D_{seq} applies without a second summand:

$$r_{01} \setminus 1 = (((0 + 1)^* \cdot 0) \setminus 1) \cdot 1.$$

At the middle node, the left operand $(0 + 1)^*$ is nullable, so D_{seq}^n applies. First,

$$\begin{aligned} (0 + 1)^* \setminus 1 &= ((0 + 1) \setminus 1) \cdot (0 + 1)^* && D_* \\ &= ((0 \setminus 1) + (1 \setminus 1)) \cdot (0 + 1)^* && D_+ \\ &= (\emptyset + \varepsilon) \cdot (0 + 1)^* && D_{\text{lit}} \\ &= (0 + 1)^* && S_3, S_2. \end{aligned} \tag{3.13}$$

Hence

$$\begin{aligned} ((0 + 1)^* \cdot 0) \setminus 1 &= (0 + 1)^* \cdot 0 + (0 \setminus 1) && D_{\text{seq}}^n \\ &= (0 + 1)^* \cdot 0 + \emptyset && D_{\text{lit}} \\ &= (0 + 1)^* \cdot 0 && S_3. \end{aligned} \tag{3.14}$$

Substituting back into the outer node gives

$$r_{01} \setminus 1 = ((0 + 1)^* \cdot 0) \cdot 1 = r_{01}. \tag{3.15}$$

Thus a 1 not yet part of the final 01 is absorbed by the leading star.

2. **Derive by 0.** The expression is unchanged. Again, $(0 + 1)^* \setminus 0$ simplifies to $(0 + 1)^*$, but now the second summand at the middle node survives:

$$\begin{aligned} ((0 + 1)^* \cdot 0) \setminus 0 &= (0 + 1)^* \cdot 0 + (0 \setminus 0) && D_{\text{seq}}^n \\ &= (0 + 1)^* \cdot 0 + \varepsilon && D_{\text{lit}}. \end{aligned} \tag{3.16}$$

No simplification applies. The outer node is still non-nullable, so

$$r_{01} \setminus 1 \setminus 0 = ((0 + 1)^* \cdot 0 + \varepsilon) \cdot 1. \tag{3.17}$$

After 10, either the trailing 01 is still owed in full, represented by $(0 + 1)^* \cdot 0$, or the consumed 0 was already its first symbol, represented by ε .

3. **Derive by 1.** The outer binding has now changed. The left operand is $(0 + 1)^* \cdot 0 + \varepsilon$, which is nullable by N_+ and N_ε . So D_{seq}^n applies and contributes the second summand $1 \setminus 1 = \varepsilon$. For the first summand, D_+ applies, reusing the result from step 1:

$$\begin{aligned} ((0 + 1)^* \cdot 0 + \varepsilon) \setminus 1 &= ((0 + 1)^* \cdot 0) \setminus 1 + (\varepsilon \setminus 1) && D_+ \\ &= (0 + 1)^* \cdot 0 + \emptyset && \text{step 1; } D_\varepsilon \\ &= (0 + 1)^* \cdot 0 && S_3. \end{aligned} \tag{3.18}$$

Therefore

$$r_{01} \setminus 1 \setminus 0 \setminus 1 = ((0 + 1)^* \cdot 0) \cdot 1 + \varepsilon = r_{01} + \varepsilon. \tag{3.19}$$

The final expression is nullable by N_+ and N_ε , so $101 \in L(r_{01})$. The ε summand witnesses that the match is complete.

For comparison, $w = 10$ leaves $((0 + 1)^* \cdot 0 + \varepsilon) \cdot 1$. Neither branch is nullable: $(0 + 1)^* \cdot 0$ still requires a 0, while $\varepsilon \cdot 1$ still requires a 1. Hence $10 \notin L(r_{01})$.

The matcher. The implementation can be summarized by the sequence of residual expressions produced while consuming the input. For $w = x_1x_2 \cdots x_n$, the initial residual is $r_0 = r$, and each subsequent residual is obtained by deriving with respect to the next symbol and simplifying the result:

$$r_0 = r, \quad r_i = \text{simplify}(r_{i-1} \backslash x_i) \quad (1 \leq i \leq n). \quad (3.20)$$

After the final symbol has been consumed, the matcher accepts exactly when the remaining residual is nullable:

$$w \in L(r) \iff n(r_n) = \text{true}.$$

Thus $r_0 \xrightarrow{x_1} r_1 \xrightarrow{x_2} \cdots \xrightarrow{x_n} r_n$ can be viewed as a path through a lazily constructed DFA: each residual expression represents a state, and $\xrightarrow{x}$ denotes the transition obtained by taking the derivative with respect to x . Acceptance is determined by whether the final residual is nullable.

In Rust. Figure 3.5 is Equation (3.20) composed out of the three functions above.

```
pub fn match_deriv(input: &str, r: &Regex) -> bool {
    let mut current = r.clone();
    for c in input.chars() {
        current = simplify(deriv(&current, c));
    }
    nullable(&current)
}
```

Figure 3.5: `match_deriv`: derive and simplify once per input character, then ask whether what remains is nullable.

The loop is the residual sequence and nothing else. Only one expression is ever live, since `current` is reassigned in place and the previous residual is dropped, and that is what separates a matcher from the parsers of Chapter 5: a Boolean answer needs only the last expression, so nothing earlier has to be kept.

3.3 Antimirov Partial Derivatives

The derivative of Section 3.2 combines alternative residuals into a single expression using $+$. Antimirov’s partial derivatives instead keep them as a *set* of expressions, one per strand of nondeterminism [Antimirov, 1996]. Writing $pd(r, x)$ for this set,

$$\begin{aligned}
pd(\emptyset, x) &= \{\} & (P_{\emptyset}) \\
pd(\varepsilon, x) &= \{\} & (P_{\varepsilon}) \\
pd(y, x) &= \begin{cases} \{\varepsilon\} & x = y \\ \{\} & x \neq y \end{cases} & (P_{\text{lit}}) \\
pd(r + s, x) &= pd(r, x) \cup pd(s, x) & (P_{+}) \\
pd(r^*, x) &= \{ \text{smartC}(r', r^*) \mid r' \in pd(r, x) \} & (P_{*}) \\
pd(r \cdot s, x) &= \begin{cases} \{ \text{smartC}(r', s) \mid r' \in pd(r, x) \} \cup pd(s, x) & n(r) = \text{true} \\ \{ \text{smartC}(r', s) \mid r' \in pd(r, x) \} & n(r) = \text{false} \end{cases} & \begin{matrix} (P_{\text{seq}}^n) \\ (P_{\text{seq}}) \end{matrix}
\end{aligned} \tag{3.21}$$

Neither $\emptyset$ nor ε has a symbol to consume, so both give the empty set. This is already a difference worth noting: $D_{\emptyset}$ and D_{ε} returned the expression $\emptyset$, which denotes the empty language but is still a node in the result, while $P_{\emptyset}$ and P_{ε} return no residual at all. For a literal y , consuming x leaves $\{\varepsilon\}$ when $x = y$ and nothing otherwise. For $r + s$ the symbol may come from either alternative, so the two sets are unioned. For r^* the symbol is consumed by one iteration of r , and each residual of that iteration is placed in front of r^* so that the star may continue.

Concatenation depends on nullability for exactly the reason D_{seq} did. If r is not nullable it must consume the x , and each of its residuals is placed in front of an untouched s . If r is nullable it may instead contribute nothing, in which case s consumes the x and $pd(s, x)$ joins the set. The difference from Section 3.2 is that D_{seq}^n joined them with $+$ inside one expression, while P_{seq}^n keeps them as two elements of one set.

In Rust. Figure 3.6 shows the concatenation case, the one that carries the case split of Equation (3.21) over into code.

```

fn pderiv(r: &Regex, x: char) -> Vec<Regex> {
    match r {
        // ...
        Regex::Seq(r, s) => {
            let mut out: Vec<Regex> = pderiv(r, x)
                .into_iter()
                .map(|r_prime| smart_seq(r_prime, s))
                .collect();
            if nullable(r) {
                out.extend(pderiv(s, x));
            }
            out
        }
        // ...
    }
}

```

Figure 3.6: The concatenation case of $pd(r, x)$, corresponding to P_{seq}^n and P_{seq} of Equation (3.21).

The `map` implements the set comprehension of P_{seq} , applying *smartC* to each residual of the left operand, and the guarded `out.extend(pderiv(s, x))` implements the additional $\cup pd(s, x)$ of P_{seq}^n . The union of the two rules is therefore an `extend` on a vector rather than a set operation. `pderiv` itself returns a `Vec<Regex>` rather than a set, so duplicate residuals reached by different strands are only merged where the results are later inserted into a `HashSet`, below. The empty-set cases $P_\emptyset$ and P_ε return `Vec::new()` and therefore contribute no residual.

The smart constructor. The concatenation in the P_{seq} rules is handled by the constructor *smartC*, written as *SC*

$$\text{smartC}(r', s) = \begin{cases} s & r' = \varepsilon \\ r' \cdot s & \text{otherwise} \end{cases} \quad (3.22)$$

The constructor accounts for the case where the consumed symbol completes the left-hand expression. If $r' = \varepsilon$, there is nothing left to concatenate with s , so the residual is simply s . Otherwise, the residual r' remains in front of s . Thus *smartC* expresses the concatenation rule without introducing the redundant expression $\varepsilon \cdot s$.

A set element represents one possible residual rather than one branch of an explicit sum. When several alternatives can consume the same symbol, the alternatives therefore remain as separate elements of the set. Concatenation is then applied to each surviving residual independently. If one of those residuals is ε , that particular strand has finished the left operand and continues directly with s , while the other strands may still retain a non-empty residual in front of s .

In Rust. Figure 3.7 is *smartC*.

```
pub fn smart_seq(r: Regex, s: &Regex) -> Regex {
    match r {
        Regex::Eps => s.clone(),
        r => Regex::seq(r, s.clone()),
    }
}
```

Figure 3.7: `smart_seq`, the smart constructor *smartC* of Equation (3.22).

`smart_seq` takes `r` by value and `s` by reference, reflecting the construction in Equation (3.22). The residual r' comes from the recursive derivative and is consumed, while the right operand s must be retained in the resulting expression. The $r' = \varepsilon$ branch clones only `s` and omits the ε , avoiding the construction of $\varepsilon \cdot s$ altogether.

Worked example: single-symbol matching. Consider x^* against `xx` again, this time under *pd*. The partial derivative of the literal x with respect to `x` is $\{\varepsilon\}$, which the star rule then places in front of x^* using *smartC*:

$$\begin{aligned}
pd(x^*, \mathbf{x}) &= \{ smartC(r', x^*) \mid r' \in pd(x, \mathbf{x}) \} & P_*, \text{ star body } r = x \\
&= \{ smartC(r', x^*) \mid r' \in \{\varepsilon\} \} & P_{lit}, \text{ since } \mathbf{x} = x \\
&= \{ smartC(\varepsilon, x^*) \} \\
&= \{x^*\} & SC, r' = \varepsilon, s = x^*.
\end{aligned} \tag{3.23}$$

The result is again $\{x^*\}$, so deriving by the second $\mathbf{x}$ produces the same set:

$$pd(x^*, \mathbf{x}, \mathbf{x}) = \{x^*\}.$$

The remaining residual x^* is nullable, and therefore $\mathbf{xx} \in L(x^*)$. Thus the partial-derivative computation reaches the same result as the Brzozowski derivative computation, while keeping the residual as a set throughout.

Why no simplification is needed. The partial-derivative rules avoid the redundant expressions that required simplification in the Brzozowski computation. In particular, $P_\emptyset$ and P_ε produce no residual at all, while $smartC$ returns s directly when the left residual is ε . Thus the computation never constructs $\emptyset \cdot x^*$ or $\varepsilon \cdot x^*$ in the first place, so the simplification identities of Section 3.2 are not needed after a pd step. The partial-derivative representation therefore removes these redundant cases at the point where residuals are constructed, rather than constructing them first and simplifying them afterwards.

This also illustrates the different roles of simplification in the two constructions. A Brzozowski derivative produces one regular expression that may contain syntactically redundant structure, whereas a partial derivative produces a set whose elements are already the individual residual alternatives. The set itself supplies the separation that an explicit $+$ supplied in the ordinary derivative. The next example shows this distinction in a case where more than one residual survives.

Worked example: $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$. Derive r_{01} against $w = 101$, tracking the residual set. The concatenations bind as in Section 3.2, with $(r, s) = ((0 + 1)^* \cdot 0, 1)$ at the outer node and $(r, s) = ((0 + 1)^*, 0)$ at the inner node; the star body is $0 + 1$. The derivation proceeds from the inside outward.

1. **Derive by 1.** At the star body, P_+ distributes the derivative over two alternatives:

$$\begin{aligned}
pd(0+1, 1) &= pd(0, 1) \cup pd(1, 1) & P_+, r = 0, s = 1 \\
&= \{\} \cup \{\varepsilon\} & P_{lit}, \text{ both branches} \\
&= \{\varepsilon\}.
\end{aligned} \tag{3.24}$$

The star rule then applies $smartC$ to the resulting residual:

$$\begin{aligned}
pd((0 + 1)^*, 1) &= \{ smartC(r', (0 + 1)^*) \mid r' \in \{\varepsilon\} \} & P_*, r = 0 + 1 \\
&= \{(0 + 1)^*\} & SC, r' = \varepsilon.
\end{aligned} \tag{3.25}$$

At the middle concatenation, the left operand is nullable by N_* , so the nullable case contributes both the derivatives through the left and right operand:

$$\begin{aligned}
pd((0 + 1)^* \cdot 0, 1) &= \{ smartC(r', 0) \mid r' \in pd((0 + 1)^*, 1) \} \cup pd(0, 1) & P_{seq}^n \\
&= \{ smartC((0 + 1)^*, 0) \} \cup \{\} & \text{above; } P_{lit} \\
&= \{(0 + 1)^* \cdot 0\} & SC, r' \neq \varepsilon.
\end{aligned} \tag{3.26}$$

At the outer node, the left operand is not nullable, so only one branch contributes:

$$\begin{aligned} pd(r_{01}, 1) &= \{ smartC(r', 1) \mid r' \in \{(0+1)^* \cdot 0\} \} \quad P_{seq}, r = (0+1)^* \cdot 0, s = 1 \\ &= \{(0+1)^* \cdot 0 \cdot 1\} = \{r_{01}\} \quad SC. \end{aligned} \quad (3.27)$$

Thus the first symbol leaves a single residual, the r_{01} itself. The computation has returned to the same residual from which it started, representing the possibility that the consumed 1 belonged to the leading star.

2. **Derive by 0.** The current set is still $\{r_{01}\}$. As before,

$$pd((0+1)^*, 0) = \{(0+1)^*\}.$$

At the middle concatenation, however, the right operand is now the matching literal 0, so P_{lit} contributes $\{\varepsilon\}$:

$$\begin{aligned} pd((0+1)^* \cdot 0, 0) &= \{smartC((0+1)^*, 0)\} \cup pd(0, 0) \quad P_{seq}^n, r = (0+1)^*, s = 0 \\ &= \{(0+1)^* \cdot 0\} \cup \{\varepsilon\} \quad P_{lit}, \text{ since } 0 = 0 \\ &= \{(0+1)^* \cdot 0, \varepsilon\}. \end{aligned} \quad (3.28)$$

The outer concatenation applies $smartC$ separately to both residuals:

$$\begin{aligned} pd(r_{01}, 1, 0) &= \{ smartC(r', 1) \mid r' \in \{(0+1)^* \cdot 0, \varepsilon\} \} \quad P_{seq}, s = 1 \\ &= \{(0+1)^* \cdot 0 \cdot 1, 1\} \quad SC \\ &= \{r_{01}, 1\}. \end{aligned} \quad (3.29)$$

The two residuals represent the possible strands after 10: r_{01} keeps both symbols in the leading star, while 1 means the 0 began the final 01. Since $smartC(\varepsilon, 1) = 1$, the latter is represented directly as 1.

3. **Derive by 1.** The two residuals are derived independently and their results are unioned:

$$\begin{aligned} pd(r_{01}, 1, 0, 1) &= pd(r_{01}, 1) \cup pd(1, 1) \quad \text{union over the set} \\ &= \{r_{01}\} \cup \{\varepsilon\} \quad \text{step 1; } P_{lit} \\ &= \{r_{01}, \varepsilon\}. \end{aligned} \quad (3.30)$$

The final set contains ε , so one surviving residual is nullable and therefore $101 \in L(r_{01})$. The conclusion matches Equation (3.19).

For $w = 10$, the first two steps are the same, leaving $\{r_{01}, 1\}$. Neither residual is nullable: $n(r_{01}) = \text{false}$, while the literal 1 becomes nullable only after a further derivative. Hence $10 \notin L(r_{01})$.

Step 2 directly relates the two constructions. The set $\{r_{01}, 1\}$ represents the same alternatives as $((0+1)^* \cdot 0 + \varepsilon) \cdot 1$ in Equation (3.17): either the trailing 01 remains to be matched, or the consumed 0 was its first symbol. The Brzowski derivative combines these alternatives using $+$, whereas pd retains them as separate residuals.

Finiteness. The set of partial derivatives reachable from a fixed r is finite, bounded by $\|r\| + 1$, where $\|r\|$ counts literal occurrences [Antimirov, 1996, Theorem 3.4, p. 309]. For r_{01} , $\|r_{01}\| = 4$, so at most five residuals can occur; the example reaches only $\{r_{01}\}$, $\{r_{01}, 1\}$, and $\{r_{01}, \varepsilon\}$.

Ordinary derivatives are finite only after equivalent expressions have been identified [Brzozowski, 1964, Theorem 4.3]; the resulting bound is exponential, $d_R \leq 2^{d_P} - 1$. The simplifications of Equation (3.10) approximate this identification by removing units and collapsing identical summands, but do not identify all equivalent expressions. Antimirov’s bound is linear and applies directly to the partial-derivative set.

The matcher. The residual sequence of Equation (3.20) has a direct counterpart here: a *frontier* of residuals, updated one input symbol at a time by taking pd of every element currently in it and unioning the results, with membership decided by whether any residual in the final frontier is nullable.

In Rust. Figure 3.8 is that loop.

```
pub fn match_pderiv(input: &str, r: &Regex) -> bool {
    let mut states: HashSet<Regex> = HashSet::new();
    states.insert(r.clone());

    for c in input.chars() {
        let mut next: HashSet<Regex> =
            HashSet::with_capacity(states.len());
        for state in &states {
            next.extend(pderiv(state, c));
        }
        if next.is_empty() {
            return false;
        }
        states = next;
    }
    states.iter().any(nullable)
}
```

Figure 3.8: The partial-derivative matcher. The set of residuals is a `HashSet<Regex>`, one entry per surviving strand.

The residual set is a `HashSet<Regex>` [The Rust Project Developers, 2026e,d]. Here, `Regex`’s derived `Hash` and `Eq` (Section 2.2) allow residuals reached by different routes to be represented by a single set element. The contrast with `match_deriv` is visible in the declarations: the Brzozowski matcher maintains one `Regex`, while the partial-derivative matcher maintains a set of them.

The early `return false` follows from the construction: once the frontier is empty, no later derivative can produce a residual from it. The remaining input therefore cannot change the result. The final `states.iter().any(nullable)` is the acceptance condition: *some* surviving residual must be nullable, whereas `match_deriv` checks nullability of its single residual.

Thus, while the Brzozowski matcher follows one residual expression at a time, the partial-derivative matcher maintains all surviving residuals explicitly. The transition $\{r_{01}\} \rightarrow \{r_{01}, 1\} \rightarrow \{r_{01}, \varepsilon\}$ illustrates this difference. Each residual can be viewed as a possible NFA state, and the set records all states that can be active after reading the input.

4 Matching versus Parsing

Chapter 3 established matching with a Boolean answer: whether $w \in L(r)$. A matcher therefore answers only whether the input is accepted by the regular expression. Parsing goes beyond this membership question by identifying the derivation for the match.

This distinction is important because a regular expression can admit more than one way of matching the same word. A parser must therefore represent the choices made during a match. These choices can be represented by *parse trees*, which provide structured witnesses for membership in the language.

4.1 Matching Ambiguity

Consider the regular expression

$$r = (a + ab)(b + \varepsilon)$$

and the input word

$$w = \mathbf{ab}.$$

The matcher can establish that $w \in L(r)$ using the matching procedures of Chapter 3. There are, however, two ways in which the match can be obtained. In the first, the left subexpression matches **a** and the right subexpression matches **b**:

$$\underbrace{a}_{a+ab} \quad \underbrace{b}_{b+\varepsilon}.$$

In the second, the left subexpression matches the entire input and the right subexpression matches the empty word:

$$\underbrace{\mathbf{ab}}_{a+ab} \quad \underbrace{\varepsilon}_{b+\varepsilon},$$

Both choices therefore establish the same membership result,

$$\mathbf{ab} \in L((a + ab)(b + \varepsilon)),$$

but assign the input differently to the subexpressions of r . The Boolean result of matching does not distinguish these cases.

Parsing makes this additional structure explicit. A parser must represent which alternatives were chosen and which parts of the input were consumed by the corresponding subexpressions. A *parse tree* provides a structured witness for such a derivation, recording the choices made by the expression and the corresponding consumption of the input.

4.2 Parse Trees as Matching Evidence

Sulzmann and Lu view regular expressions as types and parse trees as their values [Sulzmann and Lu, 2014]. The judgment $\vdash v : r$ states that v is a valid parse tree for r and records the structural choices made when matching r :

$$\begin{array}{c}
 \vdash () : \varepsilon \quad (\mathsf{T}_\varepsilon) \\
 \\
 \frac{l \in \Sigma}{\vdash l : l} \quad (\mathsf{T}_{\text{lit}}) \\
 \\
 \frac{\vdash v_1 : r_1 \quad \vdash v_2 : r_2}{\vdash (v_1, v_2) : r_1 r_2} \quad (\mathsf{T}_{\text{seq}}) \\
 \\
 \frac{\vdash v_1 : r_1}{\vdash \text{Left } v_1 : r_1 + r_2} \quad (\mathsf{T}_+^L) \\
 \\
 \frac{\vdash v_2 : r_2}{\vdash \text{Right } v_2 : r_1 + r_2} \quad (\mathsf{T}_+^R) \\
 \\
 \frac{}{\vdash [] : r^*} \quad (\mathsf{T}_*^0) \\
 \\
 \frac{\vdash v : r \quad \vdash vs : r^*}{\vdash (v : vs) : r^*} \quad (\mathsf{T}_*)
 \end{array} \tag{4.1}$$

The rules correspond to the constructors of regular expressions. A concatenation uses a pair of parse trees, an alternation records the chosen branch with a tagged injection, and a Kleene star records its iterations as a list.

In Rust. The definitions above translate directly to Rust. Figure 4.1 shows the parse-tree type, with one variant for each constructor in Equation (4.1). The two star rules share a single **Star** variant, represented by a **Vec**.

```

#[derive(Debug, Clone, PartialEq, Eq)]
pub enum ParseTree {
    Empty, // ()
    Char(char), // l
    Pair(Box<ParseTree>, Box<ParseTree>), // (v1, v2)
    Left(Box<ParseTree>), // Left v
    Right(Box<ParseTree>), // Right v
    Star(Vec<ParseTree>), // [v1, ..., vn]
}

```

Figure 4.1: The parse-tree type (`types/tree.rs`). The variants correspond to the constructors of Equation (4.1); the two star rules share the **Star** variant.

The correspondence is one to one except for the star case: T_ε , T_{lit} , T_{seq} , T_+^L , and T_+^R correspond to **Empty**, **Char**, **Pair**, **Left**, and **Right**, respectively. The empty and non-empty star rules are both represented by **Star**, with $[]$ represented by **Star(vec![])**.

The recursive variants use **Box<ParseTree>** because Rust requires recursive types to have a statically known size. **Star** instead uses **Vec<ParseTree>**, which already provides indication through a fixed-size vector handle.

Derived traits. `PartialEq` and `Eq` allow direct comparison of parse trees, while `Hash` is omitted because parse trees are not used as keys for deduplication.

The typing judgment describes the structure of a match, but a parse tree must also be related to the word that it matches. For this purpose, they define a *flattening* function $|v|$ that maps a parse tree to the word represented by its structure:

$$\begin{aligned} |()| &= \varepsilon, & |(v_1, v_2)| &= |v_1||v_2|, \\ |l| &= l, & |Left\ v| &= |v|, \\ |[]| &= \varepsilon, & |Right\ v| &= |v|, \\ & & |v : vs| &= |v||vs|. \end{aligned} \tag{4.2}$$

The flattening function connects parse trees to the language of a regular expression:

$$L(r) = \{ |v| \mid \vdash v : r \}. \tag{4.3}$$

Thus, a parse tree v provides a structured *witness* for membership in $L(r)$, recording both the match and the structural choices by which r produces the word.

In Rust. Flattening is implemented directly from Equation (4.2):

```
pub fn flatten(v: &ParseTree) -> String {
  match v {
    ParseTree::Empty      => String::new(),
    ParseTree::Char(c)    => c.to_string(),
    ParseTree::Pair(v1, v2) =>
      format!("{}", flatten(v1), flatten(v2)),
    ParseTree::Left(v)    => flatten(v),
    ParseTree::Right(v)   => flatten(v),
    ParseTree::Star(vs)   => vs.iter().map(flatten).collect(),
  }
}
```

Figure 4.2: Flattening (`types/tree.rs`), corresponding to Equation (4.2).

The function takes `&ParseTree` rather than consuming the tree, so the same parse tree can be inspected, printed, and compared after flattening. The star case similarly borrows its elements through `vs.iter()`.

Keeping flattening separate from tree construction provides an independent validation of a parse: a tree for w must satisfy $|v| = w$, as required by Equation (4.5).

Unambiguous example. Consider $r = ab$ and $w = ab$. The sequence rule requires a parse tree for a followed by a parse tree for b . Since each subexpression is a literal, the literal rule gives the only possible derivation:

$$\frac{\frac{a \in \Sigma}{\vdash a : a} (\mathsf{T}_{\text{lit}}) \quad \frac{b \in \Sigma}{\vdash b : b} (\mathsf{T}_{\text{lit}})}{\vdash (a, b) : ab} (\mathsf{T}_{\text{seq}}) \tag{4.4}$$

The parse tree is $v = (a, b)$ and flattening recovers the input word: $|(a, b)| = |a| |b| = \mathbf{ab}$.

4.3 Disambiguation Problem

The typing judgment identifies all valid parse trees for a given regular expression, while the flattening function determines which word each tree represents. So for a regular expression r and word w , a parse result must satisfy:

$$\vdash v : r \quad \text{and} \quad |v| = w. \quad (4.5)$$

When several parse trees satisfy these conditions, the typing judgment does not determine which one should be returned. A *disambiguation policy* specifies how this choice is made. Two policies commonly considered in the literature are *Greedy* and *POSIX*:

- *Greedy* resolves choices by the order in which the expression offers them, preferring the earlier alternative and preferring a further iteration to stopping, whatever each consumes [Frisch and Cardelli, 2004].
- *POSIX* resolves choices by subpattern match length from left to right, with ties resolved by the expression order [IEEE and The Open Group, 2017].

The precise definitions of these policies are given in the corresponding literature; see Frisch and Cardelli [2004] and Sulzmann and Lu [2014] for formal treatments. Instead, the examples below show how the policies distinguish between valid parse trees.

Ambiguity in concatenation. For $r_1 = (a + ab)(b + \varepsilon)$ and $w = \mathbf{ab}$, there are two possible matches, corresponding to two distinct parse trees.

The first consumes $\mathbf{ab}$ with the second alternative of $a + ab$ and then chooses ε :

$$\frac{\frac{\vdash a : a \quad \vdash b : b}{\vdash (a, b) : ab}}{\vdash \text{Right}(a, b) : a + ab} \quad \frac{\vdash () : \varepsilon}{\vdash \text{Right}() : b + \varepsilon} \Rightarrow \vdash (\text{Right}(a, b), \text{Right}()) : (a + ab)(b + \varepsilon). \quad (4.6)$$

Thus,

$$v_1 = (\text{Right}(a, b), \text{Right}()), \quad |v_1| = \mathbf{ab}. \quad (4.7)$$

The second matches $\mathbf{a}$ and $\mathbf{b}$ using the first alternatives of $a + ab$ and $b + \varepsilon$:

$$\frac{\vdash a : a}{\vdash \text{Left } a : a + ab} \quad \frac{\vdash b : b}{\vdash \text{Left } b : b + \varepsilon} \Rightarrow \vdash (\text{Left } a, \text{Left } b) : (a + ab)(b + \varepsilon). \quad (4.8)$$

Thus,

$$v'_1 = (\text{Left } a, \text{Left } b), \quad |v'_1| = \mathbf{ab}. \quad (4.9)$$

Hence $v_1 \neq v'_1$ are valid parse trees for the same expression and word. POSIX selects v_1 because the leftmost subpattern matches the longer string **ab**, while Greedy selects v'_1 because its left alternative comes first.

Ambiguity in repetition. A similar ambiguity arises from repetition. Consider $r_2 = (a + b + ab)^*$ against $w = \mathbf{ab}$, with the alternation nested as $a + (b + ab)$. The word can either be consumed by one iteration using the ab alternative or by two iterations consuming **a** and **b** separately.

The first match is

$$\frac{\frac{\frac{\vdash a : a \quad \vdash b : b}{\vdash (a, b) : ab}}{\vdash \text{Right}(a, b) : b + ab}}{\vdash \text{Right Right}(a, b) : a + (b + ab)} \Rightarrow \vdash [\text{Right Right}(a, b)] : (a + (b + ab))^*. \quad (4.10)$$

Thus,

$$v_2 = [\text{Right Right}(a, b)], \quad |v_2| = \mathbf{ab}. \quad (4.11)$$

The second match uses two iterations:

$$\frac{\vdash a : a}{\vdash \text{Left } a : a + (b + ab)} \quad \frac{\frac{\vdash b : b}{\vdash \text{Left } b : b + ab}}{\vdash \text{Right Left } b : a + (b + ab)} \quad (4.12)$$

$$\Rightarrow \vdash [\text{Left } a, \text{Right Left } b] : (a + (b + ab))^*.$$

Thus,

$$v'_2 = [\text{Left } a, \text{Right Left } b], \quad |v'_2| = \mathbf{ab}. \quad (4.13)$$

Here POSIX selects the single-iteration tree v_2 , since the star's first iteration can match both characters through the ab alternative. Greedy selects the two-iteration tree v'_2 , since the alternative a comes first and consuming only **a** leaves **b** for a second iteration.

The examples show that the same regular expression and input can admit multiple valid parse trees, with the selected tree depending on the disambiguation policy. The next two chapters apply these ideas to derivative-based parsing and compare the resulting parser implementations.

5 Derivative-Based Parser

Chapter 3 built a matcher on the Brzozowski derivative. It returns a Boolean. Chapter 4 showed that a Boolean is not enough: one expression and one word can admit several parse trees, and a parser has to return exactly one of them.

This chapter closes that gap twice, following Sulzmann and Lu [Sulzmann and Lu, 2014]. The *direct* construction runs the derivative sequence forward and rebuilds a tree from it backward. The *bit-coded* construction builds the tree during the forward pass and needs no backward pass at all. Both return the POSIX tree, and both return the same tree.

Each construction gets three sections: its theory, its implementation, and a worked example checked against the running program. The direct construction needs little new theory, because Section 3.2 already defined $r \backslash x$, $n(r)$ and the derivative sequence. The bit-coded construction needs a full theory section, because it changes the objects the derivative operates on.

5.1 From Answer to Evidence

Take the smallest expression that has a tree worth rebuilding: $r = a \cdot b$ against $w = \mathbf{ab}$. This example is chosen because it has exactly *one* parse tree. Ambiguity is the subject of Section 5.4; here the question is only whether a tree can be recovered at all, so an unambiguous case isolates the mechanism from the choice.

Deriving $a \cdot b$ by $\mathbf{a}$ and then by $\mathbf{b}$ gives $\varepsilon \cdot b$ and then $\phi \cdot b + \varepsilon$. The last expression is nullable, so the word matches. That is where Chapter 3 stopped.

Now look at what the sequence destroyed. The first step turned the a into ε . The second step turned that ε into ϕ , and turned the b into ε . No consumed character survives anywhere in the final expression. The tree (a, b) cannot be read out of it, because the information needed to build it was discarded one character at a time.

It can be recovered by walking the same sequence backward. Figure 5.1 is the whole idea of the direct construction in one picture.

The figure names the two functions this chapter has to define. At the right-hand end, *mkEps* builds a tree for the empty word. That is all the final expression can justify, since nothing remains to be matched. Each *inj* step then moves one column to the left and reinserts one character. After the first step the tree is $((), b)$. The b is back in place, and $()$ holds open the position where the a belongs. After the second step that placeholder has become a and the tree is complete.

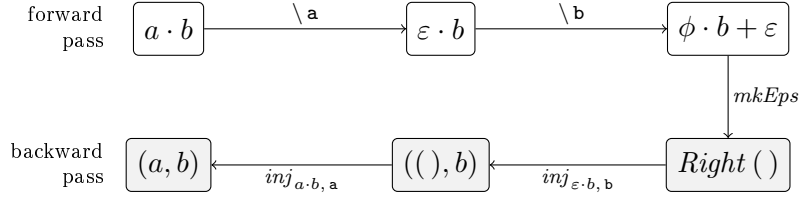


Figure 5.1: The direct construction on $a \cdot b$ against "ab". The top row is the derivative sequence of Section 3.2. *mkEps* turns at the end and builds a tree for the empty word. Each *inj* step undoes one derivative and puts back the character that step consumed. Every expression in the top row must stay in memory until the step below it has run.

The forward row is unchanged from Chapter 3. This chapter adds only the bottom row.

5.1.1 Which Tree

The $a \cdot b$ example has one tree, so it does not say which tree a parser should return when there are several. The chapter's running example does. It is r_1 from Section 4.3:

$$r_1 = (a + ab)(b + \varepsilon), \quad w = ab,$$

with the two candidates $v_1 = (Right(a, b), Right())$ and $v'_1 = (Left a, Left b)$.

Both parsers in this chapter return v_1 , because both implement POSIX. The standard states the rule in one sentence: consistent with the whole match being the longest of the leftmost matches, each subpattern, from left to right, shall match the longest possible string [IEEE and The Open Group, 2017, Section 9.1].

Read that against r_1 . The leftmost subpattern is $a + ab$. Of the two splits of **ab**, only one lets that subpattern match two characters. So $a + ab$ takes **ab** through its right alternative. That leaves nothing for $b + \varepsilon$, which must then take ε , also its right alternative. Two right choices give v_1 .

Greedy, the other policy named in Section 4.3, prefers the earlier alternative at each choice and so selects v'_1 . This chapter does not formalize either ordering. What matters is that the target is a definite tree, that it is v_1 , and that neither construction ever builds both candidates and compares them. Each makes one local decision, and the two theory sections below say exactly where that decision sits.

5.2 Tree Construction

5.2.1 The Empty-Word Tree

mkEps takes a nullable expression and returns a parse tree for the empty word. It recurses on the structure of the expression:

$$\begin{aligned}
mkEps_\varepsilon &= () & (E_\varepsilon) \\
mkEps_{r^*} &= [] & (E_*) \\
mkEps_{r_1 r_2} &= (mkEps_{r_1}, mkEps_{r_2}) & (E_{\text{seq}}) \\
mkEps_{r_1+r_2} &= \begin{cases} \text{Left } mkEps_{r_1} & \varepsilon \in L(r_1) & (E_+^L) \\ \text{Right } mkEps_{r_2} & \text{otherwise} & (E_+^R) \end{cases}
\end{aligned} \tag{5.1}$$

The rule names continue the scheme of Section 3.2. Every derivation below cites them and states the binding of r_1 and r_2 , since which subexpression each stands for is what decides the case.

Three of the four rules are forced. ε has one tree. A star matching the empty word took zero iterations, which is T_*^0 of Equation (4.1) read backward. A concatenation matches empty only when both factors do, by N_{seq} , so both halves of the pair exist. There is no rule for a literal or for $\emptyset$, because neither is ever nullable and the function is never called on one.

The fourth rule is the decision. When both disjuncts are nullable, both match the empty word and both consume nothing. "Longest possible string" cannot separate them, so POSIX falls back on position and prefers the left. That is the $\varepsilon \in L(r_1)$ guard.

The guard is a test, not a comparison. E_+^L and E_+^R are never both applicable, because the case split looks only at r_1 . If the left disjunct is nullable, E_+^L fires and the right disjunct is never examined. Sulzmann and Lu prove that deciding this way yields the POSIX tree [Sulzmann and Lu, 2014].

5.2.2 Reinserting a Character

inj inverts one derivative step. Given a parse tree of $r \setminus x$, it returns a parse tree of r with x put back:

$$\begin{aligned}
inj_{x \setminus x}() &= x & (I_{\text{lit}}) \\
inj_{(r_1+r_2) \setminus x}(\text{Left } v_1) &= \text{Left } (inj_{r_1 \setminus x} v_1) & (I_+^L) \\
inj_{(r_1+r_2) \setminus x}(\text{Right } v_2) &= \text{Right } (inj_{r_2 \setminus x} v_2) & (I_+^R) \\
inj_{r^* \setminus x}(v, vs) &= (inj_{r \setminus x} v) : vs & (I_*) \\
inj_{(r_1 r_2) \setminus x}(v_1, v_2) &= (inj_{r_1 \setminus x} v_1, v_2) & (I_{\text{seq}}), n(r_1) = \text{false} \\
inj_{(r_1 r_2) \setminus x}(\text{Left}(v_1, v_2)) &= (inj_{r_1 \setminus x} v_1, v_2) & (I_{\text{seq}}^L), n(r_1) = \text{true} \\
inj_{(r_1 r_2) \setminus x}(\text{Right } v_2) &= (mkEps_{r_1}, inj_{r_2 \setminus x} v_2) & (I_{\text{seq}}^R), n(r_1) = \text{true}
\end{aligned} \tag{5.2}$$

The first four rules are direct. A literal was derived to ε , so the only tree it can receive is $()$, and the character goes back. An alternation was derived on both sides, so the incoming tag says which side to descend into. A star was derived to $(r \setminus x) \cdot r^*$, so the incoming tree is a pair whose first component belongs to the iteration in progress.

The three concatenation rules need the derivative rule they invert. Section 3.2 gave two cases. When $n(r_1)$ is false, the derivative is $(r_1 \setminus x) r_2$ and no alternation appears. When $n(r_1)$ is true, the derivative joins that same term with $r_2 \setminus x$ under a *fresh* $+$. That fresh $+$ tags the incoming tree, and the tag says which branch was taken:

Incoming tree	Rule	What the derivative did
(v_1, v_2)	l_{seq}	r_1 was not nullable. No $+$ was introduced, and the derivative went straight into r_1 .
$Left(v_1, v_2)$	l_{seq}^L	r_1 was nullable, and the derivative still continued inside r_1 .
$Right v_2$	l_{seq}^R	r_1 was nullable and the derivative skipped it. Nothing of r_1 was consumed, so its half of the pair has to be synthesized by $mkEps_{r_1}$.

The last row calls *mkEps* a second time, in the middle of the backward pass. Section 5.5 removes that call.

5.2.3 The Parser

The two functions compose into one recursion. It applies *mkEps* once, at the end of the input, and *inj* once per character on the way back:

$$\begin{aligned} \text{parse}(r, \varepsilon) &= \text{mkEps}_r & (\text{if } \varepsilon \in L(r)) \\ \text{parse}(r, xw) &= \text{inj}_{r \setminus x}(\text{parse}(r \setminus x, w)) \end{aligned} \tag{5.3}$$

If the expression reached at the end of the input is not nullable, no parse tree exists and parsing fails.

5.2.4 No Simplification

One rule constrains the forward pass: it must not simplify.

Section 3.2 applied the identities of Equation (3.10) after every derivative step, and for a Boolean matcher that is safe. Simplification replaces r_i by a language-equivalent expression, and language equivalence is all a Boolean answer depends on.

The backward pass depends on more. $\text{inj}_{r \setminus x}$ is indexed by r , and its case split reads r 's shape: whether the node is a *Seq*, and whether that *Seq*'s left factor is nullable. S_2 alone, which rewrites $\varepsilon \cdot r$ to r , deletes exactly such a node. The backward pass would then meet an expression that is not the one its tree came from.

So the direct construction pays for the tree in expression size. The bit-coded construction does not, and Section 5.5.5 explains why it is free to simplify.

5.3 Implementation: deriv_std

5.3.1 What the Parser Needs

The direct parser is four functions over the **Regex** enum of Figure 2.1, plus a driver. Each transcribes one equation, with the case split of the equation becoming the arms of a **match** and the recursion becoming recursion.

Function	Transcribes	Also used by
<code>nullable</code>	Equation (3.3)	every parser and matcher
<code>deriv</code>	Equation (3.4)	<code>match_deriv</code>
<code>mk_eps</code>	Equation (5.1)	<code>pderiv_std</code>
<code>inject</code>	Equation (5.2)	nothing else

The first two are Chapter 3's and are unchanged here. The last two are this chapter's.

`simplify` is absent from the table, and that absence is Section 5.2.4's rule enforced by the module's imports: the direct parser does not import it, so no forward step can simplify by accident. The two matchers of Chapter 3 do import it.

5.3.2 `mk_eps`

Equation (5.1) has four clauses, one per nullable construct. Figure 5.2 is the whole function.

```
pub fn mk_eps(r: &Regex) -> ParseTree {
  match r {
    Regex::Eps => ParseTree::Empty,
    Regex::Star(_) => ParseTree::Star(Vec::new()),
    Regex::Seq(r1, r2) => {
      let v1 = mk_eps(r1);
      let v2 = mk_eps(r2);
      ParseTree::Pair(Box::new(v1), Box::new(v2))
    }
    Regex::Alt(r1, r2) => {
      if nullable(r1) {
        ParseTree::Left(Box::new(mk_eps(r1)))
      } else {
        ParseTree::Right(Box::new(mk_eps(r2)))
      }
    }
    Regex::Lit(c) => panic!("mk_eps called on Lit('{ }')", c),
    Regex::Phi => panic!("mk_eps called on Phi"),
  }
}
```

Figure 5.2: `mk_eps`, transcribing Equation (5.1). The first four arms are E_ε , E_* , E_{seq} and the E_+^L/E_+^R pair, in that order.

Why it is written this way. Three decisions in this function are worth naming.

The expression is borrowed, not consumed. `mk_eps` only reads r , and the same r is read again by `inject` later in the backward pass, so a `&Regex` is the only signature that works. The tree it returns is freshly built and owned.

The `Alt` arm is the entire POSIX decision, and it is one `if`. Nothing compares the two branches. `nullable(r1)` is evaluated, and $r2$ is touched only when it comes back false. That is E_+^L 's guard, $\varepsilon \in L(r_1)$, with no appeal to lengths, because at the empty word there are no lengths to compare.

The last two arms encode a precondition. Equation (5.1) has no clause for ϕ or for a literal, because neither is nullable and the function is never called on one. A Rust `match` on an enum must cover every variant, so the transcription has to write something. It writes `panic!`, which states the precondition rather than working around it. The alternative is to invent a tree and return it, and a wrong parse tree that is well formed is exactly what the agreement tests of Chapter 8 cannot detect, because they compare parsers against each other and two guesses can agree.

5.3.3 inject: The Direct Rules

`inject(r, l, v)` carries three arguments because *inj* is indexed by an expression and a character. The tree alone does not determine the rule.

Section 5.2.2 split Equation (5.2) into four direct rules and three concatenation rules. Figure 5.3 is the first group, with the concatenation arm held back for Figure 5.4.

```

pub fn inject(r: &Regex, l: char, v: ParseTree) -> ParseTree {
  match r {
    Regex::Lit(c) => {
      assert!(*c == l);
      assert!(matches!(v, ParseTree::Empty));
      ParseTree::Char(l)
    }

    Regex::Alt(r1, r2) => match v {
      ParseTree::Left(v1) =>
        ParseTree::Left(Box::new(inject(r1, l, *v1))),
      ParseTree::Right(v2) =>
        ParseTree::Right(Box::new(inject(r2, l, *v2))),
      _ => panic!("inject/Alt: want Left|Right, got {:?}", v),
    },

    Regex::Star(r1) => match v {
      ParseTree::Pair(v1, vs) => {
        let mut iterations = vec![inject(r1, l, *v1)];
        if let ParseTree::Star(rest) = *vs {
          iterations.extend(rest);
        }
        ParseTree::Star(iterations)
      }
      _ => panic!("inject/Star: want Pair, got {:?}", v),
    },

    Regex::Seq(r1, r2) => { /* see the next figure */ }

    Regex::Eps => panic!("inject called on Eps"),
    Regex::Phi => panic!("inject called on Phi"),
  }
}

```

Figure 5.3: `inject`, with the literal, alternation and star arms transcribing l_{lit} , l_+^L , l_+^R and l_* of Equation (5.2).

Why it is written this way. The function matches on `r` at the top level and on `v` inside each arm, and that order is the definition's order. Equation (5.2) indexes every rule by the expression first, and the tree tag then selects among the rules that expression allows. Matching on `v` first would not work, since a `Left` tag means one thing under an alternation and something else under a nullable concatenation.

The literal arm is two assertions and a constructor. The assertions say what l_{lit} leaves implicit in its indexing: the character being reinserted is the literal's own, and the tree handed in is the $()$ that D_{lit} produced. Neither can fail if the tree came from this expression's own derivative sequence.

The star arm is where the parse tree's list representation shows. The derivative turned

r^* into $(r \setminus l) \cdot r^*$, so the incoming tree is a `Pair`: the iteration in progress on the left, and the iterations already recovered on the right. The code injects into the first, then flattens the second onto it with `iterations.extend(rest)`, which is $(inj\ v) : vs$ written for a `Vec`. The `if let` guards the case where `vs` is not a `Star`, which cannot arise from a well-formed derivative and yields an empty tail if it somehow did.

5.3.4 inject: Concatenation

Figure 5.4 is the arm held back above. It is the three-row table of Section 5.2.2.

```
Regex::Seq(r1, r2) => {
  if !nullable(r1) {
    match v {
      ParseTree::Pair(v1, v2) => ParseTree::Pair(
        Box::new(inject(r1, l, *v1)), v2),
      _ => panic!("inject/Seq: want Pair, got {:?}", v),
    }
  } else {
    match v {
      ParseTree::Left(vp) => match *vp {
        ParseTree::Pair(v1, v2) => ParseTree::Pair(
          Box::new(inject(r1, l, *v1)), v2),
        o => panic!("inject/SeqL: want Pair, got {:?}", o),
      },
      ParseTree::Right(v2) => ParseTree::Pair(
        Box::new(mk_eps(r1)),
        Box::new(inject(r2, l, *v2))),
      _ => panic!("inject/Seq: want Left|Right, got {:?}", v),
    }
  }
}
```

Figure 5.4: The concatenation arm of `inject`, transcribing I_{seq} , I_{seq}^L and I_{seq}^R . The three reachable arms appear in the order of Section 5.2.2's table.

Why it is written this way. The outer `if` is the side condition $n(r_1)$ and the inner `match` reads the tag the derivative left behind. The two are separate tests because the rules are indexed on two separate things: the expression decides whether a fresh $+$ was introduced at all, and only then does the tree's tag mean anything.

Calling `nullable(r1)` here is a recomputation. The forward pass already evaluated it at this node, when D_{seq}^n was chosen over D_{seq} , and discarded the answer. Caching it would mean carrying a second structure alongside the expression sequence, which is the kind of bookkeeping Section 5.5 avoids by not having a backward pass at all.

The ownership split follows the data flow. `r` is borrowed and only read; `v` is taken by value, because the tree is taken apart and rebuilt. That is what lets the `Left` arm write `match *vp`, moving the tree out of its `Box` to destructure it. It is also what lets both matching arms hand `v2` straight into the new `Pair` with no copy: in Equation (5.2) the

component v_2 appears unchanged on the right-hand side of I_{seq} and $\mathsf{I}_{\text{seq}}^L$, and in Rust an unchanged component is a move.

The **Right** arm is the only one that builds something new. $\mathsf{I}_{\text{seq}}^R$ has nothing to reuse for r_1 's half of the pair, because the derivative skipped r_1 entirely and consumed none of it, so `mk_eps(r1)` synthesizes that half. This is the second `mk_eps` call of the backward pass, and the one Section 5.5.3's *fuse* moves into the forward pass.

5.3.5 Two Drivers

Equation (5.3) is a recursion. Figure 5.5 transcribes it twice.

```

fn parse_recursive_helper(r: &Regex, input: &str)
  -> Option<ParseTree>
{
  let mut chars = input.chars();
  match chars.next() {
    None => {
      if nullable(r) { Some(mk_eps(r)) } else { None }
    }
    Some(l) => {
      let rest: String = chars.collect();
      let r_deriv = deriv(r, l);
      let subtree = parse_recursive_helper(&r_deriv, &rest)?;
      Some(inject(r, l, subtree))
    }
  }
}

pub fn parse_deriv_std_loop(input: &str, r: &Regex)
  -> Option<ParseTree>
{
  let chars: Vec<char> = input.chars().collect();
  let n = chars.len();

  let mut expressions = Vec::with_capacity(n + 1);
  expressions.push(r.clone());

  for &c in chars.iter() {
    let current = expressions.last().unwrap();
    let next = deriv(current, c);
    expressions.push(next);
  }

  let final_r = expressions.last().unwrap();
  if !nullable(final_r) {
    return None;
  }

  let mut tree = mk_eps(expressions.last().unwrap());
  for i in (0..n).rev() {
    tree = inject(&expressions[i], chars[i], tree);
  }

  Some(tree)
}

```

Figure 5.5: The two entry points. The recursive one is Equation (5.3) read literally. The loop one materializes Figure 5.1’s top row as a `Vec<Regex>` and then walks it backward.

The recursive driver never stores the top row. The call stack is the top row: each frame holds its own `r` and `l`, the descent runs to the end of the input, and `inject` runs as the stack unwinds. The `?` operator propagates a failed match, so a `None` from the recursive call returns immediately and no injection is attempted.

The loop driver stores the top row explicitly, then indexes it backward with `(0..n).rev()`.

The algorithm is the same. The resource behaviour is not. A thread's stack in Rust is a fixed allocation made when the thread starts, while a `Vec` grows on the heap. The recursive driver overflows the stack on input the loop driver handles without difficulty. Section 8.5.1 measures where the difference begins.

5.3.6 Trees and Flattening

Both drivers return an `Option` over the `ParseTree` of Section 4.2. Every parser in this thesis produces that one type, which is what lets the agreement tests compare outputs directly.

`flatten` is kept out of tree construction so that it can serve as an independent check. It discards every tag and every pair boundary and returns the concatenated literals. A successful parse of w must flatten back to w , whatever the disambiguation policy was. Every diagnostic report prints the tree and its flattening side by side for that reason.

5.4 Worked Example: `deriv_std`

This section runs $r_1 = (a + ab)(b + \varepsilon)$ against `ab` through the construction, and checks each step against the tool's own report:

```
cargo run -- parse "(a|ab)(b|)" "ab" --parser deriv_std_rec --diag 3
```

where `"(a|ab)(b|)"` is r_1 in the surface syntax of Section 7.2, with ε spelled as an empty right alternative. The tool's Unicode is transliterated to ASCII below (∂ as `d`, ε as `epsilon`, ϕ as `phi`), and long lines are wrapped; the full report is Appendix C.1.

5.4.1 Forward Pass

Two derivative steps consume the input:

$$\begin{aligned} r_1 &\xrightarrow{a} (\varepsilon + \varepsilon b)(b + \varepsilon) & D_{\text{seq}}, r = a + ab, s = b + \varepsilon, n(r) = \text{false} \\ &\xrightarrow{b} (\phi + (\phi b + \varepsilon))(b + \varepsilon) + (\varepsilon + \phi) & D_{\text{seq}}^n, r = \varepsilon + \varepsilon b, n(r) = \text{true} \end{aligned} \quad (5.4)$$

The two steps take different cases of the same rule. The binding is what changes. Initially the left operand is $a + ab$, which is not nullable. After one character it is $\varepsilon + \varepsilon b$, which is nullable by N_+ and N_ε . That is what adds the summand $(\varepsilon + \phi)$ at the second step, and that summand is what makes the whole expression nullable.

The report confirms both the rule and the full expression:

```

Step 1: d(r0, 'a')
  rule: d(r1.r2, 'a') [nullable(r1)=false]
        = d(r1, 'a').r2
r1 = Seq(Alt(Eps, Seq(Eps, Lit('b'))), Alt(Lit('b'), Eps))
  nullable(r1) = true [OK] position 1 matched

```

```

Step 2: d(r1, 'b')
  rule: d(r1.r2, 'b') [nullable(r1)=true]
        = d(r1, 'b').r2 + d(r2, 'b')
r2 = Alt(Seq(Alt(Phi, Alt(Seq(Phi, Lit('b'))), Eps)), Alt(Lit('b'),
  Eps)), Alt(Eps, Phi))
  nullable(r2) = true [OK] position 2 matched

```

Step 1 also makes Section 5.2.4 concrete. The `Eps` inside `Seq(Eps, Lit('b'))` is exactly the node S_2 would delete. It is printed, so it was kept, and the backward pass will read it.

5.4.2 The Empty-Word Tree

Step 2 introduced a fresh `Alt` at the top of r_2 . Both of its disjuncts are nullable, so this is the case Section 5.2.1 singled out.

Write $X = \phi + (\phi b + \varepsilon)$ and $Y = b + \varepsilon$ for the two factors of the left disjunct:

$$\begin{aligned}
mkEps_{r_2} &= Left\ mkEps_{XY} & E_+^L, r_1 = XY, r_2 = \varepsilon + \phi \\
&= Left\ (mkEps_X, mkEps_Y) & E_{seq}, r_1 = X, r_2 = Y \\
&= Left\ (Right\ mkEps_{\phi b + \varepsilon}, Right\ mkEps_\varepsilon) & E_+^R\ \text{twice}, \varepsilon \notin L(\phi), \varepsilon \notin L(b) \\
&= Left\ (Right\ Right\ (), Right\ ()) & E_+^R, E_\varepsilon
\end{aligned} \tag{5.5}$$

The report prints the first line on its own, because it is the only line with a choice in it:

```

mkEps(r2):
  mkEps(r1 + r2): nullable(r1)=true -> Left(mkEps(r1))
v2 = Left (Right Right (), Right ())

```

One decision was made and three cases followed from it. The outer E_+^L commits to the summand that came from continuing inside $a + ab$. After that, E_{seq} has no choice, and the two E_+^R steps take right branches only because their left operands, ϕ and b , are not nullable.

5.4.3 The Two Injections

The backward pass runs twice. The first injection undoes the `b` step. Write $X' = \varepsilon + \varepsilon b$ for the left factor of r_1 :

$$\begin{aligned}
inj_{r_1 \setminus b}(v_2) &= (inj_{X' \setminus b}(Right\ Right\ ()),\ Right\ ()) & l_{seq}^L, n(X') = \text{true} \\
&= (Right\ inj_{(\varepsilon b) \setminus b}(Right\ ()),\ Right\ ()) & l_+^R, r_2 = \varepsilon b \\
&= (Right\ (mkEps_\varepsilon,\ inj_{b \setminus b}()),\ Right\ ()) & l_{seq}^R, n(\varepsilon) = \text{true} \\
&= (Right\ ((), b),\ Right\ ()) & E_\varepsilon, l_{lit}
\end{aligned} \tag{5.6}$$

Two rows of Section 5.2.2's table appear here. The first line is l_{seq}^L undoing the **Left** that *mkEps* had just produced. The third line is l_{seq}^R : the derivative had skipped the nullable ε and jumped into b , so ε 's half of the pair is synthesized by *mkEps* and not recovered.

The second injection undoes the **a** step:

$$\begin{aligned}
inj_{r_0 \setminus a}(v_1) &= (inj_{(a+ab) \setminus a}(Right\ ((), b),\ Right\ ()),\ Right\ ()) & l_{seq}, n(a+ab) = \text{false} \\
&= (Right\ inj_{(ab) \setminus a}((), b),\ Right\ ()) & l_+^R, r_2 = ab \\
&= (Right\ (inj_{a \setminus a}(),\ b),\ Right\ ()) & l_{seq}, n(a) = \text{false} \\
&= (Right\ (a, b),\ Right\ ()) & l_{lit}
\end{aligned} \tag{5.7}$$

The report gives both results:

```

inject(r1, 'b', v2):
  -> (Right (( ), b), Right ())
v1 = (Right (( ), b), Right ())

inject(r0, 'a', v1):
  -> (Right (a, b), Right ())
v0 = (Right (a, b), Right ())

```

Read together, the two lines show the placeholder moving. After the first injection the tree still holds $()$, sitting where **a** has not yet been put back. After the second, l_{lit} has replaced it with a .

The answer is $(Right(a, b), Right())$, which is v_1 : the POSIX tree identified in Section 5.1.1.

5.4.4 What It Cost

The report's header quantifies the shape in one line:

```
Expressions stored: 3 (r0 -> r1 -> r2)
```

One expression per input character, plus the original. Each stays live until the backward pass has consumed it. That is the requirement Figure 5.1 draws, and the next section removes it.

5.5 Bit-Coded Construction

The direct construction cannot start building the tree until the input is exhausted, because *inj* needs each r_{i-1} to reconstruct against. Sulzmann and Lu's bit-coded construction (their Section 3.2, "Incremental Bit-Coded Forward Parse Tree Construction") [Sulzmann

and Lu, 2014] removes that dependency. The tree is built during the forward pass, so when the last character is consumed the answer is already there.

Figure 5.6 shows the resulting shape on a^* against "aaa".

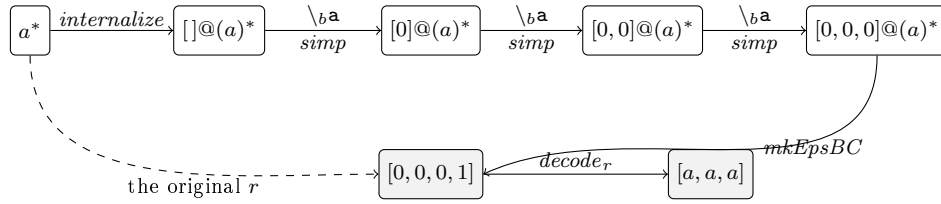


Figure 5.6: The bit-coded construction on a^* against "aaa". Every arrow between expressions points forward, and no ri_i is needed once its successor exists. The expression stays two nodes wide while the bit-prefix grows by one 0 per iteration. The dashed arrow is the one backward dependency left: *decode* needs the original r to know what shape the bits describe.

Compare it with Figure 5.1. There the bottom row moves right to left and needs the whole top row. Here there is no bottom row. The parse tree is carried along in the annotations, and the two nodes at the end of the top row are enough to produce it.

Four things have to be defined to make that work: what a bit-string parse tree is, what an annotated expression is, how the derivative threads bits through one, and how the bits come back out.

5.5.1 Trees as Bit-Strings

A parse tree becomes a flat sequence of bits. At an alternation, 0 marks *Left* and 1 marks *Right*. Inside a Kleene star, 0 marks "one more iteration" and 1 marks "stop". A pair of mutually recursive functions $encode_r$ and $decode_r$ converts between a $\vdash v : r$ parse tree and its bit-string.

The star convention is the one place where a bit does not correspond to an alternation, so it is worth spelling out. Under $r = a^*$ the word **aaa** encodes as $[0, 0, 0, 1]$: three 0s for the three iterations, then a closing 1. The empty word under the same r encodes as $[1]$ alone, a star that stops at once.

Two consequences follow. The bit-string's length grows with the number of iterations and not with the size of r . And the bits alone are not a tree: they record which branch was taken, not what the branches were. Reading them back needs r , which is what $decode_r$ is given.

5.5.2 Annotated Expressions

The idea is to annotate the derivative's *input*, not only its output. Then, by the time a subexpression has become ε , its accumulated annotation already is the relevant slice of the final bit-string.

An annotated expression ri extends r with a bit-list bs at every node except $\emptyset$, which denotes no language and so has no parse tree to annotate:

$$ri ::= \emptyset \mid (bs@_\varepsilon) \mid (bs@x) \mid (bs@ri_1 \oplus ri_2) \mid (bs@ri_1 ri_2) \mid (bs@ri^*) \quad (5.8)$$

Two operations connect the two worlds. *internalize* converts r into ri with every bit-list empty, except at an alternation: the left operand of $r_1 + r_2$ is tagged with a leading 0 and the right with a leading 1, at construction time. That way a subtree always carries its own record of which alternative it came from, with no need to inspect the surrounding structure.

$fuse(bs, ri)$ prepends a bit-list onto whatever annotation ri 's top node already carries. Every other operation here is built from it.

5.5.3 The Annotated Derivative

The derivative is lifted to annotated expressions as $ri \setminus_{bl}$. The case structure is Equation (3.4)'s, with bits threaded through instead of discarded:

$$\begin{aligned} (bs@ri_1 \oplus ri_2) \setminus_{bl} &= (bs@(ri_1 \setminus_{bl}) \oplus (ri_2 \setminus_{bl})) & (B_+) \\ (bs@ri_1 ri_2) \setminus_{bl} &= (bs@(ri_1 \setminus_{bl}) ri_2) & (B_{seq}), \varepsilon \notin L(ri_1) \\ (bs@ri_1 ri_2) \setminus_{bl} &= (bs@(ri_1 \setminus_{bl}) ri_2) \oplus fuse(mkEpsBC_{ri_1}, ri_2 \setminus_{bl}) & (B_{seq}^n), \varepsilon \in L(ri_1) \\ (bs@ri^*) \setminus_{bl} &= (bs@fuse([0], ri \setminus_{bl}) ([]@ri^*)) & (B_*) \end{aligned} \quad (5.9)$$

The concatenation cases mirror the two-way split of D_{seq} and D_{seq}^n , with one addition. On the branch where the derivative abandons ri_1 and jumps into ri_2 , the bits ri_1 would contribute to an empty match are fused onto ri_2 's derivative.

That fusion is the entire optimization. It does immediately, during the forward pass, what l_{seq}^R postponed to a later $mkEps_{r_1}$ call inside inj . The work is the same; the moment is different.

The star case attaches a leading 0, one more iteration, each time the body is derived again. The outer bs stays on the new node the rule produces.

5.5.4 Extracting the Bits

$mkEpsBC$ mirrors $mkEps$ but returns bits:

$$\begin{aligned} mkEpsBC_{(bs@_\varepsilon)} &= bs & (EB_\varepsilon) \\ mkEpsBC_{(bs@ri_1 \oplus ri_2)} &= bs ++ \begin{cases} mkEpsBC_{ri_1} & \varepsilon \in L(ri_1) \\ mkEpsBC_{ri_2} & \text{otherwise} \end{cases} & \begin{matrix} (EB_+^L) \\ (EB_+^R) \end{matrix} \\ mkEpsBC_{(bs@ri_1 ri_2)} &= bs ++ mkEpsBC_{ri_1} ++ mkEpsBC_{ri_2} & (EB_{seq}) \\ mkEpsBC_{(bs@ri^*)} &= bs ++ [1] & (EB_*) \end{aligned} \quad (5.10)$$

The alternation case carries the same left-preference guard as E_+^L , and for the same POSIX reason. This is where the bit-coded construction makes its single decision, and it is the

counterpart of the one `if` in Section 5.3.2. The star case emits the closing 1 that ends a run of iterations.

$decode_r$ then turns the bit-string into an ordinary parse tree. It takes the original, un-annotated r , because the annotated ri no longer describes the shape being decoded once its own annotations have been consumed. That is the dashed arrow in Figure 5.6, and the only thing the construction has to remember from before the forward pass began.

5.5.5 Simplification

Section 5.2.4 forbade simplification in the direct construction. The bit-coded construction is under no such restriction, and it needs one.

The reason it is free to simplify is that the information a backward pass would need is already carried *on* the expression, as bits. Replacing ri_i by a smaller expression with the same bits loses nothing *mkEpsBC* reads later.

The reason it needs one is growth. Sulzmann and Lu [2014] give the concrete case

$$\begin{aligned} a^* &\xrightarrow{a} \varepsilon a^* \xrightarrow{a} \phi a^* + \varepsilon a^* \\ &\xrightarrow{a} (\phi a^* + \varepsilon a^*) + (\phi a^* + \varepsilon a^*) \xrightarrow{a} \dots \end{aligned} \quad (5.11)$$

where every step after the first is language-equivalent to εa^* and syntactically double the size. Unchecked, this defeats the point of a single-pass construction, since a growing ri makes each further $\setminus_b$ step more expensive.

A function *simp*, applied after every derivative step, keeps the annotated expression small. It propagates ϕ through *Seq* and *Alt*, flattens and deduplicates nested alternations keeping the first occurrence of a syntactic duplicate, and eliminates ε from a concatenation by

$$(bs @ (bs' @ \varepsilon) ri) \longrightarrow fuse(bs ++ bs', ri),$$

folding the eliminated node's own bits forward instead of dropping them. All are instances of Equation (3.10), applied to the annotated grammar and combined with an explicit *isPhi* test. That test recognizes an empty-language expression built up from these rules, and not only the literal $\emptyset$ constructor.

Where the bound fails. On a^* , *simp* collapses every step back to a bit-annotated a^* , which is exactly Figure 5.6. That result does not generalize, and it is worth being precise about the failure instead of letting the a^* case stand for the general one. Figure 5.7 measures node counts per step for two patterns, each with and without *simp*.

The two simplified series behave completely differently. For a^* the line is flat. For $(a + (b + ab))^*$ it is straight on a logarithmic axis, which is to say exponential. Every two characters take the size s to $2s + 1$, so $s + 1$ doubles. From 35 at step 4 the series runs 71, 143, 287, 575, 1151, 2303 at step 16, and 36 863 at step 24. The growth is $\Theta(2^{n/2})$ in the input length.

Two checks rule out the obvious explanations. The figures are unchanged if the three-way alternation is grouped the other way, as $((a + b) + ab)^*$, which is what the CLI spelling `"(a|b|ab)*"` parses to (Section 2.1), so the growth depends on the pattern's shape and

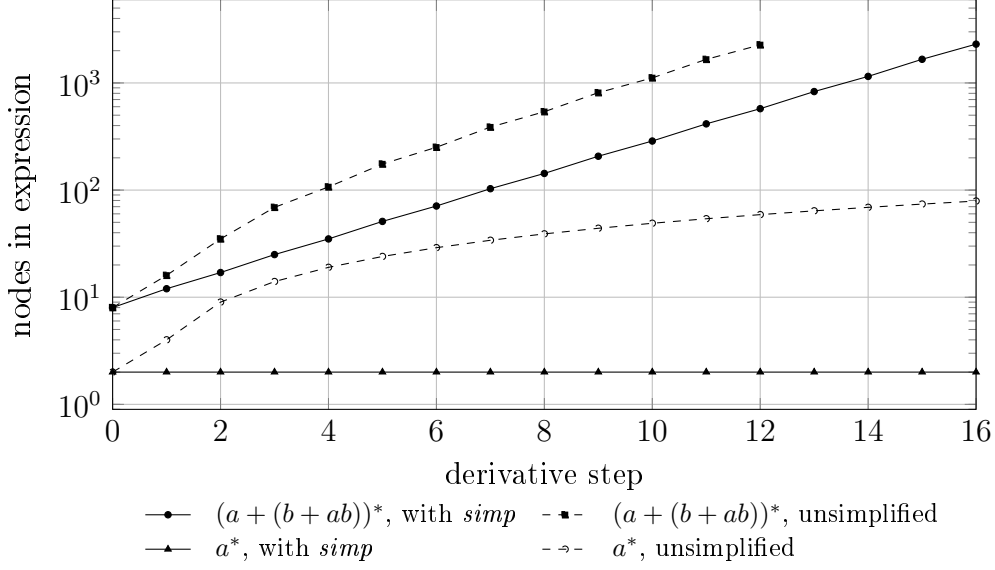


Figure 5.7: Expression size per derivative step, logarithmic vertical scale. For a^* , *simp* holds the expression at two nodes while the unsimplified derivative grows linearly. For $(a + (b + ab))^*$ against $(ab)^n$, *simp* reduces the expression substantially at every step and still leaves it growing exponentially: a straight line on this scale is exponential growth. Regenerate with `cargo run --release --example thesis_figures -- growth; tests/test_thesis_figures.rs` pins these values.

not on that grouping. And applying *simp* repeatedly changes nothing, because a single pass already is the fixpoint here and produces byte-identical expressions at every step, so this is not the implementation stopping too early.

The mechanism is specific. Deduplication discards an alternation branch only when it is *syntactically identical* to an earlier one, and an annotated branch carries its bit-list as part of its syntax. Counting the top-level branches of $\text{simp}(ri_i)$ at each step, and counting them again after erasing every annotation, separates the two:

Step	2	4	6	8	10	12
Branches	2	4	8	16	32	64
Distinct branches	2	4	8	16	32	64
Distinct ignoring bits	1	1	1	1	1	1

At every step all of the branches are the same expression structurally. They differ only in the bits recording which route through the alternation reached them. So the last row stays at 1 while the first two double, and S_4 never fires.

The bits are not redundant. Each branch’s annotation is a different candidate parse tree, and *mkEpsBC*’s left-preference is what selects among them. Dropping duplicates on shape alone would change which tree the parser returns, and not merely shrink the expression. That is why the rule is stated on full syntactic equality, and why this growth is a property of the construction and not a defect in this transcription of it.

Sulzmann and Lu claim, but do not fully prove, that the construction runs in time linear in the input. Their grounds are that each of $\backslash_b$, *simp*, *fuse*, *mkEpsBC* and *isPhi* recurses only over subcalls bounded by the now-simplified expression’s size, and they note that detailed space-complexity estimates are left as future work [Sulzmann and Lu, 2014].

Figure 5.7 is one concrete reason that gap matters: the argument’s premise is that *simp* keeps the expression small, and that premise holds for a^* and fails for $(a + (b + ab))^*$. Chapter 8 measures the wall-clock consequence, and Section 8.5.4 returns to this family.

5.6 Implementation: `deriv_bc`

5.6.1 The Annotated Type

Equation (5.8) becomes a second enum alongside `Regex`, shown in Figure 5.8.

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum ARegex {
    Phi, // no annotation slot
    Eps(Vec<bool>), // bs @ eps
    Lit(Vec<bool>, char), // bs @ 1
    Alt(Vec<bool>, Box<ARegex>, Box<ARegex>),
    Seq(Vec<bool>, Box<ARegex>, Box<ARegex>),
    Star(Vec<bool>, Box<ARegex>),
}
```

Figure 5.8: The annotated-expression type, transcribing Equation (5.8). Comments are transliterated to ASCII. `Phi` is the one variant with no `Vec<bool>`.

The grammar’s omission of an annotation on $\emptyset$ is enforced by the type. There is no field to put one in.

Comparing this with Figure 2.1 shows the one place where a plain `Regex` will not do. The construction threads partial parse-tree information through the expression as it is derived, and that needs somewhere on every node to put it.

The two types also derive different traits, and Rust makes that difference explicit. `Regex` derives `Hash`, because Chapter 6’s deduplication uses it as a `HashSet` key. `ARegex` does not, because `deriv_bc` never hashes an annotated expression. Its own deduplication, inside *simp*, compares branches pairwise through `PartialEq`. A language deriving a fixed bundle of instances at once would leave that difference implicit.

5.6.2 `internalize` and `fuse`

Section 5.5.2 defined two operations on annotated expressions. Figure 5.9 gives both.

```

pub fn fuse(bs: &[bool], ri: ARegex) -> ARegex {
  if bs.is_empty() { return ri; }
  match ri {
    ARegex::Phi => ARegex::Phi,
    ARegex::Eps(p) => ARegex::Eps(combine(bs, p)),
    ARegex::Lit(p, c) => ARegex::Lit(combine(bs, p), c),
    ARegex::Alt(p, r1, r2) =>
      ARegex::Alt(combine(bs, p), r1, r2),
    ARegex::Seq(p, r1, r2) =>
      ARegex::Seq(combine(bs, p), r1, r2),
    ARegex::Star(p, r) => ARegex::Star(combine(bs, p), r),
  }
}

pub fn internalize(r: &Regex) -> ARegex {
  match r {
    Regex::Phi => ARegex::Phi,
    Regex::Eps => ARegex::Eps(vec![]),
    Regex::Lit(c) => ARegex::Lit(vec![], *c),
    Regex::Alt(r1, r2) => {
      let ri1 = fuse(&[false], internalize(r1));
      let ri2 = fuse(&[true], internalize(r2));
      ARegex::Alt(vec![], Box::new(ri1), Box::new(ri2))
    }
    Regex::Seq(r1, r2) => ARegex::Seq(vec![],
      Box::new(internalize(r1)), Box::new(internalize(r2))),
    Regex::Star(r1) =>
      ARegex::Star(vec![], Box::new(internalize(r1))),
  }
}

```

Figure 5.9: `fuse` prepends a bit-prefix to a node’s own annotation. `internalize` converts a `Regex` into an `ARegex`, tagging the two sides of every alternation. `combine` concatenates the two bit-lists.

Why it is written this way. `fuse` exists because the annotation lives *inside* every variant. There is no field common to all of them, so prepending a prefix means rebuilding the node, which is one arm per variant. `Phi` is the exception, and it is the grammar’s exception too: it has no annotation slot, so fusing onto it is a no-op.

The early return is what makes the common case free. `ri` is an owned parameter, so `return ri` is a move and not a copy. The expression is handed straight back and no node is rebuilt. `fuse` runs on every `Seq` and `Star` step of every derivative, and most of those calls carry an empty prefix, so this one line removes most of the work the function would otherwise do.

`internalize` is where every bit in the whole construction originates. Four of its six arms produce an empty `vec![]`. Only the `Alt` arm produces bits, and it produces exactly two, one per side. That is the concrete content of Section 5.7.1’s claim that the derivative

only ever moves bits: no other function in this chapter introduces a 0 or a 1 except B_* 's iteration bit and EB_* 's closing bit, both of which are star bookkeeping.

Tagging at construction time, and not at use time, is what removes the need for a backward pass. A subtree carries its own record of which alternative it came from, so nothing downstream has to look at the surrounding structure to find out.

5.6.3 The Derivative

Figure 5.10 shows the two arms of `deriv_bc` that do something a plain derivative does not.

```

ARegex::Seq(bs, r1, r2) => {
  if nullable_bc(&r1) {
    let eps_bits = mk_eps_bc(&r1);
    let d1 = deriv_bc(*r1, 1);
    let d2 = deriv_bc(*r2.clone(), 1);
    let left_branch = ARegex::Seq(vec![], Box::new(d1), r2);
    let right_branch = fuse(&eps_bits, d2);
    ARegex::Alt(bs, Box::new(left_branch),
                Box::new(right_branch))
  } else {
    let d1 = deriv_bc(*r1, 1);
    ARegex::Seq(bs, Box::new(d1), r2)
  }
}

ARegex::Star(bs, r) => {
  let d = deriv_bc(*r.clone(), 1);
  let fused = fuse(&[false], d);
  let new_star = ARegex::Star(vec![], r);
  ARegex::Seq(bs, Box::new(fused), Box::new(new_star))
}

```

Figure 5.10: The concatenation and star arms of `deriv_bc`, transcribing B_{seq} , B_{seq}^n and B_* of Equation (5.9).

The nullable branch is B_{seq}^n line by line. `eps_bits` is $mkEpsBC_{r_{i1}}$, computed *before* `r1` is consumed by the derivative, and `fuse(&eps_bits, d2)` carries those bits onto the right branch.

Ownership makes the cost of each branch visible. `deriv_bc` takes `ri` by value, so in the non-nullable branch `r2` is moved into the result and never copied, which matches B_{seq} leaving r_{i2} untouched. The nullable branch needs r_{i2} twice, once underived and once derived, and `*r2.clone()` is the deep copy that second use costs. The star arm is the same story: `*r.clone()` pays for the body appearing both derived and underived in B_* .

5.6.4 mk_eps_bc and decode

Figure 5.11 is Equation (5.10). Read it next to Figure 5.2 and the two are the same function with `Vec<bool>` in place of `ParseTree`.

```
pub fn mk_eps_bc(ri: &ARegex) -> Vec<bool> {
  match ri {
    ARegex::Eps(bs) => bs.clone(),
    ARegex::Alt(bs, r1, r2) => {
      let mut result = bs.clone();
      if nullable_bc(r1) {
        result.extend(mk_eps_bc(r1));
      } else {
        result.extend(mk_eps_bc(r2));
      }
      result
    }
    ARegex::Seq(bs, r1, r2) => {
      let mut result = bs.clone();
      result.extend(mk_eps_bc(r1));
      result.extend(mk_eps_bc(r2));
      result
    }
    ARegex::Star(bs, _) => {
      let mut result = bs.clone();
      result.push(true);
      result
    }
    ARegex::Phi => panic!("mk_eps_bc called on Phi"),
    ARegex::Lit(_, c) => panic!("mk_eps_bc on Lit('{ }')", c),
  }
}
```

Figure 5.11: `mk_eps_bc`, transcribing Equation (5.10). Each arm starts from the node's own `bs` and appends what its children contribute.

Why it is written this way. Every arm opens with `bs.clone()` and appends to it. That is `bs ++ ...` read left to right, and it is why the bits come out in the order *decode* expects: a node's own prefix first, then its children's, in tree order.

The `Alt` arm is the same if as Figure 5.2's, on `nullable_bc` instead of `nullable`. The two constructions make their one POSIX decision in the same shape, on the same guard, in functions that share no code.

The `Star` arm ignores its child entirely, written `ARegex::Star(bs, _)`. A star matching the empty word took zero iterations, so the body contributes nothing and the only bit to emit is the closing `true` of `EB*`. The underscore makes that visible in the pattern.

The expression is borrowed here, unlike in `deriv_bc`, because `mk_eps_bc` is called from inside B_{seq}^n on an `r1` that the derivative is about to consume. Reading it without taking it is what lets that rule compute `eps_bits` first and derive afterwards.

`decode` has no counterpart in the direct construction. Figure 5.12 gives the two arms that consume the bit conventions of Section 5.5.1.

```

Regex::Alt(r1, r2) => match bs {
  [false, rest @ ..] => {
    let (v, remaining) = decode_inner(r1, rest);
    (ParseTree::Left(Box::new(v)), remaining)
  }
  [true, rest @ ..] => {
    let (v, remaining) = decode_inner(r2, rest);
    (ParseTree::Right(Box::new(v)), remaining)
  }
  [] => panic!("decode: unexpected end of bits at Alt"),
},

Regex::Star(r1) => {
  let mut iterations = Vec::new();
  let mut remaining = bs;
  loop {
    match remaining {
      [true, rest @ ..] => {
        remaining = rest;
        break;
      }
      [false, rest @ ..] => {
        let (v, after) = decode_inner(r1, rest);
        iterations.push(v);
        remaining = after;
      }
      [] => panic!("decode: end of bits inside Star"),
    }
  }
  (ParseTree::Star(iterations), remaining)
}

```

Figure 5.12: The alternation and star arms of `decode`. Each arm returns the tree it built together with the bits it did not use.

Slice patterns make the bit conventions readable as code. `[false, rest @ ..]` matches a non-empty slice and binds `rest` to everything after the leading bit it has just tested. So the 0-means-Left convention is one pattern, and the 0-means-another-iteration convention is another.

The helper returns a pair, the tree and the unconsumed remainder, because decoding a `Seq` has to hand whatever r_1 left over on to r_2 . The `[]` arms assert that a bit-string produced by *mkEpsBC* never runs out early, and the public wrapper asserts that none is left over once the expression has been walked. That is the same discipline as `mk_eps` and `inject`, at the opposite end of the construction.

`decode` takes a plain `Regex` and a flat `Vec<bool>`, which is why it is the one function

this chapter shares with Chapter 6’s `pderiv_bc`. That parser reaches a `(Regex, bits)` pair by an entirely different route and needs the identical final step. It does not share `mk_eps_bc`, which it reimplements over its own frontier representation.

5.6.5 The Driver

The driver is four lines of loop and three of finish:

```
pub fn parse_bitcoded_loop(input: &str, r: &Regex)
  -> Option<ParseTree>
{
  let mut ri = internalize(r);
  for l in input.chars() {
    ri = simp(deriv_bc(ri, l));
  }
  if !nullable_bc(&ri) {
    return None;
  }
  let bits = mk_eps_bc(&ri);
  Some(decode(r, &bits))
}
```

Compare this with Figure 5.5. There is no `Vec<Regex>` and no backward loop. The variable `ri` is reassigned in place, so only one annotated expression is ever live, and the `simp` call is Section 5.5.5’s requirement applied after every step.

Note that `r`, the original expression, is still borrowed at the last line. That is the dashed dependency of Figure 5.6, visible in the signature.

5.7 Worked Example: `deriv_bc`

The same r_1 against `ab`, through the other construction, checked against `cargo run - parse "(a|ab)(b|)" "ab" -parser deriv_bc -diag 3` (Appendix C.2).

5.7.1 Before the Input

`internalize` runs first, and the report prints its result:

```
ri0 = []@([]@([false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([false]@'b' (+) [true]@epsilon))
```

Read the four tags off. `[false]` on `a` and `[true]` on `ab` record the two alternatives of $a + ab$. The same pair appears again on `b` and ε . Every other bit-list is empty.

Those four tags are the only bits this expression will ever produce. Everything that follows moves them; nothing invents new ones.

5.7.2 Two Steps

The left operand is not nullable, so the first character takes B_{seq} :

Step 1 (char: 'a', position 1)

```
rule: (bs@ri1.ri2) \ 'a' [nullable_bc(ri1)=false]
      = bs@(ri1\'a'.ri2)
simp -> []@([]@([false]@epsilon (+) [true]@'b')) . []@([false]@'b'
      (+) [true]@epsilon))
```

Compare against ri_0 . The literal 'a' has become `epsilon`, and `[]@'a'` . `[]@'b'` has become 'b'. Both alternation tags are exactly where they were. One character was consumed and no bit moved.

The left operand is now nullable, through `[false]@epsilon`, so the second character takes B_{seq}^n :

Step 2 (char: 'b', position 2)

```
rule: (bs@ri1.ri2) \ 'b' [nullable_bc(ri1)=true]
      = bs@((ri1\'b'.ri2) (+) (fuse (mkEpsBC ri1) (ri2\'b')))
simp -> []@([true]@([false]@'b' (+) [true]@epsilon) (+) [false,
      false]@epsilon)
```

Two bits moved in that step, and both can be traced to their origin. The `[false, false]` prefix on the right disjunct is the rule's `fuse (mkEpsBC ri1)`: the first `false` is ri_1 's own empty-match bit, recording that $a + ab$ matched through its left alternative, and the second is $(b + \varepsilon)$'s left-alternative tag that it was fused in front of.

The `[true]` on the left disjunct is new in a different sense. It tags "the derivative continued into the left operand" as the *Right* alternative of the node B_{seq}^n has just introduced. Where *inj* discovered that tagging on the way back, $\backslash_b$ has to commit to it now.

5.7.3 Bits to Tree

Both disjuncts of ri_2 are nullable, so *mkEpsBC* faces the choice *mkEps* faced in Equation (5.5):

$$\begin{aligned}
mkEpsBC_{ri_2} &= [] \mathbin{++} mkEpsBC_{[1]@([0]@b \oplus [1]@e)} \quad EB_+^L, bs = [], \varepsilon \in L(ri_1) \\
&= [] \mathbin{++} [1] \mathbin{++} mkEpsBC_{[1]@e} \quad EB_+^R, bs = [1], \varepsilon \notin L([0]@b) \quad (5.12) \\
&= [] \mathbin{++} [1] \mathbin{++} [1] = [1, 1] \quad EB_e, bs = [1]
\end{aligned}$$

The outer step takes EB_+^L and the inner one EB_+^R , for the same reason in both cases: each rule tests only its own ri_1 . At the outer node that is the `[1]@(...)` disjunct, which is nullable, so the left branch is taken and contributes its $bs = [1]$. At the inner node it is `[0]@b`, a bare literal, which is not, so the right branch is taken and `[1]@e` contributes the second 1.

The report confirms the bits and the decode:

```
mkEpsBC(ri2) = [11]
```

```
decode((a|ab)(b|), [11]):
  = (Right (a, b), Right ())
```

The two bits select *Right* at both alternation points of r_1 . That is v_1 again, the tree Equation (5.7) reached through an explicit backward pass, read here off the forward pass alone.

5.7.4 The Star Case

r_1 has no star, so it does not exercise B_* or the 0-means-another-iteration convention. Running a^* against "aaa" does, and it is the case Figure 5.6 draws:

```
Step 1 ... simp -> [false]@([]@'a')*
Step 2 ... simp -> [false, false]@([]@'a')*
Step 3 ... simp -> [false, false, false]@([]@'a')*
```

```
mkEpsBC(ri3) = [0001]
decode(a*, [0001]):
  = [a, a, a]
```

The expression never changes. It is the two-node $([]@a)^*$ at every step, which is *simp* collapsing B_* 's output back to a star. What grows is the prefix, by one **false** per character, each one contributed by the *fuse* $([0], \dots)$ in B_* . *mkEpsBC* appends the closing **true** by EB_* , and *decode* reads the four bits back as three iterations followed by a stop.

This is the clearest case for what the construction is for. All of the parse-tree information is in the prefix and none of it is in the expression.

5.8 Comparison

Both constructions have now run the same expression against the same input. Setting the two reports against each other shows the structural difference in one view. The full reports are Appendix C.1 and Appendix C.2.

	deriv_std	deriv_bc
Report sections	forward, nullability, backward	internalize, forward, nullability, decode
Named intermediates	r0 -> r1 -> r2	ri0 -> ri1 -> ri2
Retained at the end	all three	the last one only
Tree-building calls	1 mkEps + 2 inject	1 mkEpsBC + 1 decode
Simplification	none, by necessity	simp after every step
Final answer	(Right (a, b), Right ())	

The tree-building row is the one that does not scale with the input. **deriv_std** needs one **inject** per character. **deriv_bc** needs exactly one **mkEpsBC** and one **decode**, however long the input is. That is the whole content of the optimization, and it shows in the reports as the difference between a **BACKWARD PASS** section whose length depends on the input and a fixed-size **MKEEPSBC + DECODE** section.

The final bits [11] and the final tree **(Right (a, b), Right ())** encode the same answer. The correspondence is exact: each **Right** in the tree is a 1 in the bit-string, in the order **decode** walks the expression.

On a failure. Run the same r_1 against **b**. That word is not in $L(r_1)$, since every derivation of r_1 must begin by matching **a**. Both parsers stop after one derivative step and neither reaches a tree-building phase. What each holds at that point differs:

```
deriv_std:  r1 = Seq(Alt(Phi, Seq(Phi, Lit('b'))), Alt(Lit('b'), Eps))
             nullable(r1) = false  [FAIL] position 1
```

```
deriv_bc:   simp -> phi
             ri1 = phi
             nullable_bc(ri1) = false  [FAIL]
```

`deriv_std`'s expression denotes the empty language but is not syntactically `Phi`. It is a `Seq` whose left operand has both alternatives dead. `deriv_bc`'s `simp` collapses that same expression to `phi`. This is the clearest illustration of what Section 5.5.5's *isPhi* test adds over plain `simplify`.

Both then report the identical failure, character for character:

```
Failure at position:  1 (1-indexed)
Found:                'b'
Expected:             'a'
  b
  ^
```

The diagnostics layer reads its expected set off the first-literal set of the expression at the point of failure. It therefore does not depend on which construction produced that expression, or on how much simplification was applied to it. This is the pattern across all four parsers: they differ in what they build on a match, and agree on what they report when there is nothing to build. The two full failure reports are Appendix C.5 and Appendix C.6.

Agreement. The two parsers of this chapter, plus `deriv_std`'s loop driver, are three routes to the same tree. Running all three over 300 randomly generated expressions against 39 inputs each, 11 700 cases in total, they agree on every one: on whether the input matched, and on the exact `ParseTree` returned when it did. Section 8.4.2 describes the generator and the test setup. What matters here is that the bit-coded route and the inject route share no code beyond `Regex` itself and `nullable`, and are not observed to diverge.

Where this leaves the next chapter. Both constructions compute the POSIX tree, and both do it by the same move. Consume the input with $r \backslash x$, then resolve the one remaining choice, at an alternation whose two sides both match the empty word, by preferring the left. The direct construction makes that choice in *mkEps* on the way back. The bit-coded one makes it in *mkEpsBC* at the end of the way out. Everything else follows from where the choice sits.

Chapter 6 keeps the shape and replaces $r \backslash x$ with Antimirov's $pd(r, x)$, which returns a *set*. With a set there is no single expression to be nullable, and no single alternation to prefer the left of. That turns out to change not only the machinery but the answer.

6 Partial-Derivative-Based Parser

Chapter 5 built two parsers around Brzozowski’s derivative. This chapter repeats the exercise with Antimirov’s partial derivative from Section 3.3 in place of $r \backslash x$ throughout.

The substitution looks local and is not. Both operators consume one character and return what remains to be matched, and either can drive a parser. But $r \backslash x$ returns one expression and $pd(r, x)$ returns a *set*, and a set has no single top-level alternation whose left branch a rule can prefer. That is where Chapter 5’s two constructions made their POSIX choice, in *mkEps* and in *mkEpsBC*, and with the set representation there is nothing there to make it on. The choice moves somewhere else, and it comes out differently. Section 6.1 locates it, and Section 6.1.2 shows the consequence on an expression small enough to check by inspection.

Neither *flops14-extended* [Sulzmann and Lu, 2014] nor Sulzmann’s teaching-page Haskell reference [Sulzmann, 2026] defines a POSIX partial-derivative parser at all. The paper’s bit-coded construction is Brzozowski-only, and the teaching page’s own bit-coded partial-derivative parser, `pDerivBC`, is built around a deduplication rule that settles the matter by itself, without comparing candidate trees. This chapter builds two parsers on that basis: `pderiv_bc` (Sections 6.2 and 6.3), following `pDerivBC` directly, and `pderiv_std` (Section 6.4), this project’s own construction, built to give `pderiv_bc` the same kind of matched non-bit-coded counterpart Chapter 5’s `deriv_bc` has in `deriv_std`. Both are verified to agree with each other exactly, and with the two POSIX parsers on membership alone, in Chapter 8 and again in Section 6.6.

Trace excerpts below use the same running example, r_1 against `ab` and against `b`.

6.1 From One Expression to a Frontier

Section 3.3 defined $pd(r, x)$ as a set of residual expressions, not one combined expression, and noted the NFA reading: each element is one state of an implicit NFA that r denotes, alive after reading x . Extending this to parsing means pairing every surviving element with enough information to rebuild a parse tree, should that element turn out to be the winner at the end of the input. That extra payload is either an accumulated bit-string per element (Section 6.2) or, in this project’s own extension, an accumulated tree-injection closure per element (Section 6.4).

Either way the algorithm has the same shape. Maintain a *frontier* of `(residual, extra)` pairs, one per live *strand*; at each character replace it with the union of pd applied to every current strand, each contributing its own possibly-empty set of successors; and at the end

of the input select the first strand in the frontier whose residual is nullable. Figure 6.1 draws this for the running example.

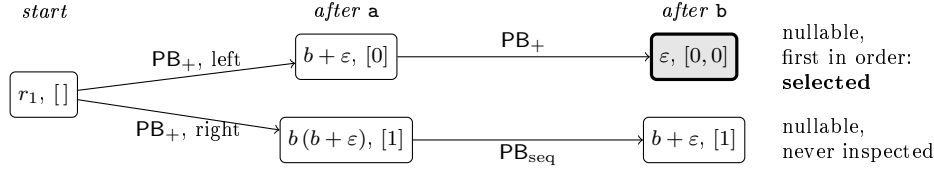


Figure 6.1: The frontier for $r_1 = (a + ab)(b + \varepsilon)$ against "ab", with `pderiv_bc`'s bits as the per-strand extra and the rule producing each strand on its arrow. One character advances every strand independently. At the end *both* strands are nullable, and selection takes the first in list order without comparing them, which is where the construction settles on one tree and not the other.

6.1.1 Where the Choice Is Actually Made

That last step, "the *first* nullable strand in the frontier", is this construction's disambiguation policy. It is decided implicitly, as a side effect of keeping the frontier's growth finite, and no rule anywhere in the construction states a preference between candidate trees.

The reason no such rule appears is worth following, because it is not an oversight. Antimirov's finiteness theorem (Section 3.3) bounds the set of *distinct* partial derivatives, but nothing about the extended, tree-carrying construction keeps strands distinct as they accumulate payload: two different derivation paths can reach the identical residual *shape* by different routes, each carrying its own bits or injection. Kept without pruning, the frontier would multiply copies of an already-explored residual instead of staying inside the bounded syntactic set *pd* guarantees. So pruning is needed. But pruning by comparing the accumulated bits or trees, keeping whichever strand has consumed more in the sense of Section 5.1.1, would mean comparing incomplete, still-growing state: a strand's tree is finished only once the input is exhausted, and at the moment the pruning must happen there is nothing complete to compare.

The Haskell reference's choice is `nub2`: deduplicate purely on residual *shape*, keeping whichever strand reached that shape *first*. This sidesteps the problem by discarding the comparison altogether. Once a shape has been claimed, every later strand reaching the identical shape is dropped, whatever either one's payload would eventually have produced. "First" means first in the list-ordered traversal *pd*'s own case structure produces, which explores the left operand of $+$ before the right at every step. The construction is therefore left-biased and commits as soon as something succeeds, which is not the longest-wins-with-left-as-tiebreak rule of Section 5.1.1.

It is worth being careful about what that establishes. It says which strand this construction selects. It does not say that the construction implements any particular named disambiguation policy in general. The trace states the consequence in one line, at the point of decision:

```
SELECTION (first nullable residual, priority/list order)
any nullable residual in final frontier = true -> proceed to
    mkEpsBC + decode
```

There is no comparison here, because there is nothing to compare against: the moment a nullable residual is found, scanning stops. That is the shaded node in Figure 6.1, and the strand below it is never examined although it is nullable too.

This reading is this thesis’s own account of where the construction settles the question. Neither primary source states it in these terms, and neither claims the result coincides with a named policy. What can be checked is narrower, and is checked in Chapter 8: the two parsers of this chapter agree with each other on every case tried, they agree with the derivative parsers on membership always, and they disagree with them on the tree wherever the expression is ambiguous, including the `paper_r1` and `paper_r2` examples of Section 4.3.

6.1.2 A Family of Witnesses

The two paper examples are not the only place the divergence shows, and it helps to have a case where the pattern is simple enough that both answers can be read off by inspection. Take $r = (b + bb)^*$ against a run of `bs`. The derivative parsers prefer the longer alternative at each iteration; the partial-derivative parsers take the left alternative as soon as it works. The two therefore partition the same input differently:

Input	Derivative	Partial derivative
"bb"	[Right (b, b)]	[Left b, Left b]
"bbb"	[Right (b, b), Left b]	[Left b, Left b, Left b]
"bbbbbb"	[Right (b, b) $\times 3$]	[Left b] $\times 6$

The derivative parsers use as few iterations as the longest-alternative rule allows, padding with a single `Left b` only when the length is odd. The partial-derivative parsers use one iteration per character throughout, because b is the left alternative and it always matches. Both flatten back to the same input and both are valid parse trees for it: the expression is ambiguous, and the two constructions resolve the ambiguity differently. These rows are the output of `cargo run - parse "(b|bb)*" "bbb" -parser all`, which runs all five entry points on one input and prints the trees side by side.

6.2 The Bit-Coded Construction

Sulzmann’s teaching-page Haskell transcript gives a bit-coded partial-derivative construction, `pDerivBC`, that this thesis’s `pderiv_bc` follows directly [Sulzmann, 2026]. It extends `pd` with per-strand bits the same way Section 5.5’s $\backslash_b$ extends the Brzozowski derivative, with one visible difference: the bits stay *external* to the expression tree, as a plain list carried alongside a plain, un-annotated r , instead of being folded into an `ARegex`-shaped tree.

Writing D_r for `pDerivBC`(x, r) throughout, to keep the set comprehensions readable, the base and non-sequence cases are

$$\begin{aligned}
D_\varepsilon &= \{\} & D_\emptyset &= \{\} & (\text{PB}_\varepsilon), (\text{PB}_\emptyset) \\
D_y &= \begin{cases} \{(\varepsilon, [])\} & x = y \\ \{\} & x \neq y \end{cases} & & (\text{PB}_{\text{lit}}) \\
D_{r+s} &= \text{nub2}(\{(r', 0:bs) \mid (r', bs) \in D_r\} \\
&\quad \cup \{(s', 1:bs) \mid (s', bs) \in D_s\}) & (\text{PB}_+) \\
D_{r^*} &= \text{nub2}(\{(smartC(r', r^*), 0:bs) \mid (r', bs) \in D_r\}) & (\text{PB}_*)
\end{aligned} \tag{6.1}$$

and the sequence case splits on the nullability of the left factor exactly as Equation (3.4) does:

$$\begin{aligned}
D_{rs} &= \{(smartC(r', s), bs) \mid (r', bs) \in D_r\} & (\text{PB}_{\text{seq}}), n(r) = \text{false} \\
D_{rs} &= \text{nub2}(\{(r's, bs) \mid (r', bs) \in D_r\} \\
&\quad \cup \{(s', mkEpsBC_r ++ bs) \mid (s', bs) \in D_s\}) & (\text{PB}_{\text{seq}}^n), n(r) = \text{true}
\end{aligned} \tag{6.2}$$

The names extend Section 3.3's P scheme, one PB rule per P rule, and the derivation below cites them with the binding of r and s given at each application. Here $mkEpsBC_r$ is the plain- r analogue of Section 5.6.4's $mkEpsBC$, identical in structure but walking an unannotated tree; $smartC$ is the smart concatenation constructor of Section 3.3, collapsing $\varepsilon \cdot s$ to s ; and nub2 deduplicates a list of (residual, bits) pairs by comparing the residual alone, keeping the first occurrence.

Two details of Equation (6.2) are worth holding onto. PB_{seq}^n 's second set is where a strand inherits $mkEpsBC_r$, the bits r would have contributed by matching empty, which is the same move B_{seq}^n made with fuse in Chapter 5 and the same one I_{seq}^R made with $mkEps$. And the two sequence cases build their residuals differently: PB_{seq} uses $smartC$ while PB_{seq}^n builds its first strand with a plain $r's$. That asymmetry is present in the reference itself and is preserved here and not "corrected", since correctness of this construction is judged by literal agreement with that reference (Chapter 8) and not against an independently derived ideal form. Section 6.4 returns to the same question from the other side, where the choice between the two constructors is not free at all.

6.2.1 The Running Example, Strand by Strand

Run $r_1 = (a + ab)(b + \varepsilon)$ against **ab** through this construction, checked against **cargo run - parse "(a|ab)(b|)" "ab" -parser pderiv_bc -diag 3** (Appendix C.4), so that it can be seen disagreeing with Chapter 5 on the same input, and not merely be asserted to do so. The frontier starts as one strand, the whole expression with no bits, which is the leftmost node of Figure 6.1.

The first character binds $r = a + ab$ and $s = b + \varepsilon$ at the outer node. That r is not nullable, by N_+ over a literal and a concatenation, so PB_{seq} applies and s contributes no strand at all. The strands that do appear come from PB_+ one level down:

$$\begin{aligned}
D_{a+ab} &= \{(\varepsilon, [0])\} \cup \{(b, [1])\} && \text{PB}_+, r = a, s = ab \\
D_{r_1} &= \{(\text{smartC}(\varepsilon, b + \varepsilon), [0]), (\text{smartC}(b, b + \varepsilon), [1])\} && \text{PB}_{\text{seq}}, n(a + ab) = \text{false} \\
&= \{(b + \varepsilon, [0]), (b(b + \varepsilon), [1])\} && \text{SC twice}
\end{aligned} \tag{6.3}$$

The two applications of SC take different branches, which is why the two strands look so different. Against the first, $r' = \varepsilon$, so SC absorbs it and leaves $b + \varepsilon$ alone; against the second, $r' = b \neq \varepsilon$, so it keeps the concatenation. The trace agrees:

```

Step 1 (char: 'a', position 1)
  rule: pDerivBC('a', r1.r2) [nullable(r1)=false]
        = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
frontier after (2 residual(s)):
[0] Alt(Lit('b'), Eps)    bits = [0]
[1] Seq(Lit('b'), Alt(Lit('b'), Eps))    bits = [1]

```

Both bits were emitted by PB_+ one level down, and PB_{seq} carried them outward unchanged, which is why neither strand's bits grew at this step. Where the plain bit-coded construction of Chapter 5 kept the two alternatives inside one expression, joined by $\oplus$, they are now two separate objects that will be advanced independently.

The second character steps *both* strands, and each is matched against the rule set on its own, so two different rules fire on the same character:

$$\begin{aligned}
D_{b+\varepsilon} &= \{(\varepsilon, [0])\} \cup \{\} && \text{PB}_+, r = b, s = \varepsilon; \text{PB}_\varepsilon \\
D_{b(b+\varepsilon)} &= \{(\text{smartC}(\varepsilon, b + \varepsilon), [])\} = \{(b + \varepsilon, [])\} && \text{PB}_{\text{seq}}, n(b) = \text{false}
\end{aligned} \tag{6.4}$$

Strand [0] is an alternation whose left disjunct contributes ε tagged 0 and whose right contributes nothing, since PB_ε returns the empty set. Strand [1] is a concatenation whose left factor is a bare literal, so PB_{seq} applies and only that factor is derived. Each strand's accumulated bits are its own prefix followed by whatever the step produced, so [0] gains a 0 and reaches [0,0] while [1] gains nothing and stays at [1]:

```

Step 2 (char: 'b', position 2)
  from [0]:
    rule: pDerivBC('b', r1+r2) = tag(0, pDerivBC('b',r1)) ++
          tag(1, pDerivBC('b',r2)), nub2
    -> (Eps, [0])
  from [1]:
    rule: pDerivBC('b', r1.r2) [nullable(r1)=false]
          = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
    -> (Alt(Lit('b'), Eps), [])
frontier after (2 residual(s)):
[0] Eps    bits = [00]
[1] Alt(Lit('b'), Eps)    bits = [1]

```

Selection now scans this final frontier in list order and takes the first nullable strand. Strand [0], $(\varepsilon, [0,0])$, is checked and accepted before strand [1] is inspected at all, although [1] is nullable too:

```
selected residual's accumulated bits ++ mkEpsBC(residual) = [00]
```

```
decode((a|ab)(b|), [00]):  
  = (Left a, Left b)
```

This is the leftmost-commits rule made mechanical. Strand [0] is the "matched *a*, then *b* separately" reading, the first alternative *pDerivBC* generated back at step one, and it wins purely because it comes first in the frontier, not because any length comparison favours it. The decoded tree is the *shorter* reading, and it is the opposite answer from Chapter 5's (*Right(a, b)*, *Right()*) on the identical expression and input. Nothing about r_1 or "ab" changed between the two traces. Only the policy governing which nullable candidate wins did.

Where nub2 is and is not applied. nub2 appears inside *pDerivBC*, once per case that can produce more than one strand, and nowhere else. In particular the frontier-driving loop concatenates the per-strand results into the next frontier without deduplicating across them, so the guarantee nub2 provides is local to a single strand's expansion and does not extend to the frontier as a whole. Two strands that were distinct at step i can converge on the same residual at step $i + 1$ and both survive. Section 6.5 measures what that costs.

6.3 Rust Implementation: pderiv_bc

The module `pderiv_bc/pderiv.rs` transcribes *pDerivBC* case for case. Each D_r becomes a vector of (`Regex`, `Vec<bool>`) pairs and not a set, because order is what encodes priority here, and a `HashSet` would discard exactly the information selection depends on. `mk_eps_bits` implements *mkEpsBC_r*. nub2 deduplicates on the residual alone, through a `HashSet` used only to test membership, keeping the first-seen pair for each distinct residual; this is the one place in the codebase where `Regex`'s derived `Hash` implementation (Section 2.2) is load-bearing. The `Seq` case matches the reference literally, down to the same asymmetric use of plain `Regex::seq` in place of `smart_seq` for its first strand.

`step_frontier` and `select_bits` (`src/parsers/pderiv_bc/parse.rs`) implement the loop of Section 6.1. At each character, `step_frontier` replaces every (`residual`, `bits`) pair with the union of `pderiv_bc` applied to it, concatenating each result's bits onto that strand's accumulated prefix. At the end of the input, `select_bits` scans in list order and returns the first nullable strand's `bits ++ mk_eps_bits(residual)`. `parse_pderiv_bc` drives the loop and passes the winning bit-string, together with the *original* `Regex` and never the derived residual, to `decode` (Section 5.6.4). That is the same shared decoder `deriv_bc` uses, since both constructions arrive at the identical (`Regex`, `bits`) shape by the time decoding is needed, whatever route each took to get there.

6.4 Rust Implementation: pderiv_std

`pderiv_std` (`src/parsers/pderiv_std/`) is this project's own construction, with no counterpart in either reference. Its purpose is to stand to `pderiv_bc` as `deriv_std` stands to `deriv_bc`: the same traversal, the same disambiguation policy and the same answers, but building a `ParseTree` directly instead of a bit-string that has to be decoded

afterwards. Having both means this chapter’s answer is computed twice by routes sharing almost no code, which is what makes the agreement check of Section 6.6 worth anything.

6.4.1 What Each Strand Carries

Where `pderiv_bc` pairs each residual with a `Vec<bool>`, `pderiv_std` pairs it with an *injection closure*, defined in this module’s `inject.rs`:

```
pub(crate) type Inj = Rc<dyn Fn(ParseTree) -> ParseTree>;
```

A strand’s `Inj` is a function that, given the parse tree of that strand’s *residual*, returns the parse tree of the *original* expression. The frontier starts with one strand carrying `identity_inj()`, and each step composes the step’s own injection inside the strand’s accumulated one. Figure 6.2 contrasts the two representations after two steps.

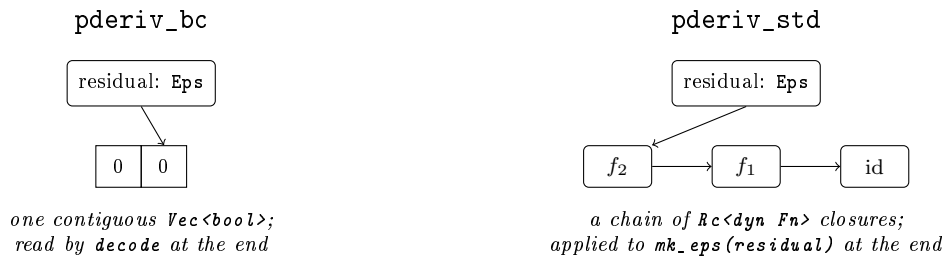


Figure 6.2: What one strand holds after two derivative steps, for the winning strand of Figure 6.1. The bit-coded strand accumulates into a flat buffer; the tree-building strand accumulates a chain of heap-allocated closures, one link per step, each holding the next. Both are consumed exactly once, on the winning strand, at the end of the input.

This is the same idea as Chapter 5’s *inj*, with one difference that matters. `deriv_std` applies `inject` once per character during an explicit backward pass, walking back along the expression sequence as it unwinds; `pderiv_std` never runs a backward pass at all. It builds the composition forward, one layer per step, and applies the finished composition exactly once at the end. In that sense `pderiv_std` is as single-pass as `pderiv_bc`: the closure chain plays the role the bit-string plays there.

The `Rc` is required and not incidental. A single step can expand one strand into several, and every successor needs the same parent injection, so the parent has to be shared, not cloned or moved. `Rc<dyn Fn(...)>` gives shared ownership of a heap-allocated closure at the cost of a reference-count bump per successor. Section 8.5.2 measures what that costs against `pderiv_bc`’s flat `Vec<bool>`; the structural point here is the one Figure 6.2 draws, that a strand’s accumulated state is a chain whose length grows with the input where `pderiv_bc`’s is a contiguous buffer.

6.4.2 One Table, Two Representations

The module `pderiv_std/pderiv_tree.rs` is the `(Regex, Inj)` analogue of `pderiv_bc`, case for case, with each case’s bit-tagging replaced by the tree constructor that bit would have decoded to:

Case	pderiv_bc attaches	pderiv_tree attaches
$\varepsilon, \emptyset$	no strands	no strands
x (matching)	<code>[]</code>	<code> _ Char(x)</code>
$r + s$, left	<code>0:bs</code>	<code> v Left(inj1(v))</code>
$r + s$, right	<code>1:bs</code>	<code> v Right(inj2(v))</code>
rs , continue r	<code>bs</code>	<code> Pair(v1,v2) Pair(inj1(v1), v2)</code>
rs , jump to s	<code>mkEpsBC_r ++ bs</code>	<code> v2 Pair(mk_eps(r), inj2(v2))</code>
r^*	<code>0:bs</code>	<code> Pair(v1,vs) Star(inj1(v1) : vs)</code>

Every row is the same decision, expressed once as a bit and once as a constructor. The correspondence is exact enough that the "jump to s " row calls the shared `mk_eps` (Section 5.3.2) at precisely the point `pderiv_bc` calls `mk_eps_bits`, and for the reason $\mathsf{I}_{\text{seq}}^R$ gave in Chapter 5: the left factor matched empty, so its contribution to the pair has to be synthesized and not derived. `nub_tree` is the `Inj`-carrying `nub2`, deduplicating on the residual alone and keeping the first occurrence, so the priority order is preserved identically.

6.4.3 Why the Smart Constructor Cannot Be Used Here

`pderiv_bc` is free to apply *smartC* in its non-nullable `Seq` case and in its `Star` case, collapsing $\varepsilon \cdot s$ to s . `pderiv_tree` is not, and uses plain `Regex::seq` everywhere its sibling uses the smart one. The reason follows directly from what an `Inj` is applied to.

Selection ends by calling `inj(mk_eps(r))` on the winning strand, so each accumulated closure is applied to a tree whose *shape is determined by its own residual*. The `Seq` strands' closures pattern-match on `ParseTree::Pair`, which is what `mk_eps` produces for a `Seq` residual. Collapsing `Seq(Eps, s)` to s would make `mk_eps` produce s 's own tree shape instead, and the closure expecting a `Pair` would receive something else. The residual and the injection are two halves of one invariant, and a smart constructor rewrites one half without the other.

This is not a speculative concern. Substituting `smart_seq` for `Regex::seq` in the two `Seq` strands of `pderiv_tree` makes the test suite fail immediately, on the simplest possible sequence pattern, with the panic that case's own guard raises:

```
---- parsers::pderiv_std::parse::tests::seq_matches stdout ----
thread '...::seq_matches' panicked at
  src/parsers/pderiv_std/pderiv_tree.rs:71:38:
pderiv_tree: Seq(non-nullable) strand expected Pair, got Char('b')
```

The panics in `pderiv_tree` therefore serve the purpose those in `inject` (Section 5.3.3) and `decode` (Section 5.6.4) serve: they encode a precondition that holds by construction and convert a violation into an immediate, located failure instead of a silently wrong parse tree.

6.4.4 The Same Example, and What the Trace Cannot Show

The driver in `src/parsers/pderiv_std/parse.rs` mirrors its `pderiv_bc` counterpart exactly. `step_frontier` expands every strand and composes injections; `select` scans in list order and returns `inj(mk_eps(r))` for the first nullable residual; `parse_pderiv_std` returns `None` as soon as the frontier becomes empty, since a frontier with no strands can

never regain one. As with `pderiv_bc`, no deduplication is applied across the flattened frontier, only within each strand’s own expansion.

Running r_1 against `ab` (`-parser pderiv_std`, Appendix C.3) produces the same two-strand fork at step one and the same winner at step two. Two differences from `pderiv_bc`’s trace are visible, and both were predicted by the smart-constructor argument above.

The first is the residuals themselves. Where `pderiv_bc` lists `Alt(Lit('b'), Eps)`, this parser lists `Seq(Eps, Alt(Lit('b'), Eps))`; where that one lists `Seq(Lit('b'), ...)`, this one lists `Seq(Seq(Eps, Lit('b')), ...)`. The ε its sibling discards is exactly the one whose `Pair` shape the closure needs.

The second follows from the first, and is sharper: because the residuals differ, the two parsers do not even take the same rules. At step two, `pderiv_bc`’s strand `[0]` is an `Alt` and fires `PB+`, while `pderiv_std`’s is a `Seq` whose left factor is the retained `Eps`, so it fires the *nullable* sequence rule instead:

```
rule: pderiv_tree('b', r1.r2) [nullable(r1)=true]
    = nub_tree $ [(r1'.r2, inj: continue r1) | ...]
      ++ [(r2', inj: mk_eps(r1) paired with r2's
              result) | ...]
```

The two parsers are therefore not running over the same residuals, and not applying the same rules to them, and they still select the same tree. Because `nub_tree` deduplicates on residual syntax, they can in principle prune at different points as well, which makes their agreement on every tested case a stronger check than it would be if they shared a residual representation.

One thing the trace cannot show is what each strand would build if it won:

```
-> Seq(Eps, Alt(Lit('b'), Eps)) (+ injection closure, not
    printable)
```

A `Vec<bool>` can be printed; an `Rc<dyn Fn>` cannot. The trace reports which residuals are alive and which rule produced each, but not the tree each carries, which is precisely the information `pderiv_bc`’s `bits = [00]` annotations do carry. That is a direct cost of the representation choice in Figure 6.2: a trace can only report state the construction actually keeps, and this one does not keep the bits.

6.5 Frontier Growth

Both parsers inherit Antimirov’s finiteness result for the set of distinct partial derivatives (Section 3.3), and neither inherits a bound on the size of the frontier, for the reason Section 6.1 identified: the frontier holds `(residual, extra)` pairs, and only the residual component is subject to *pd*’s bound. This section measures the consequence.

Take $(a + (b + ab))^*$, the `paper_r2` expression, against $(ab)^n$, and record both the number of strands in the frontier after each character and the number of *distinct* residuals among them.

Every strand beyond the second is a duplicate of one already present, differing only in the bits or injection it carries. `nub2` does not remove them because it never sees them

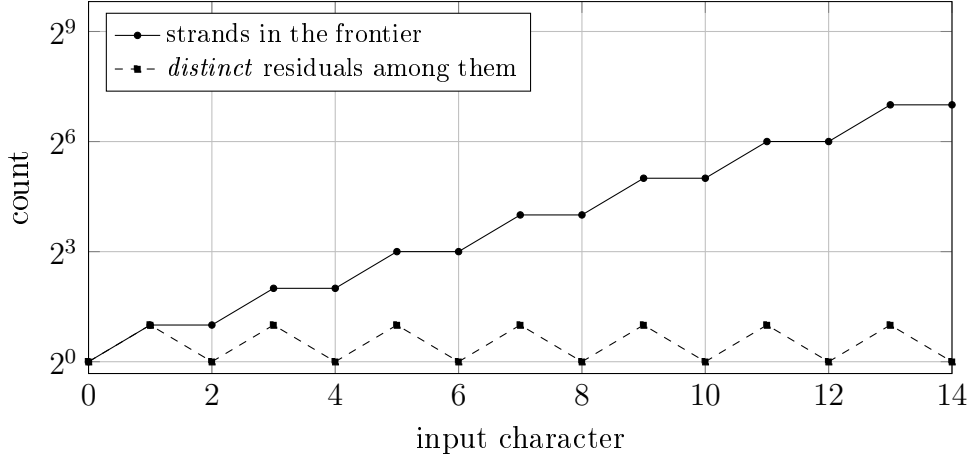


Figure 6.3: Frontier size against distinct residual count for $(a + (b + ab))^*$ on $(ab)^n$, logarithmic vertical scale. The frontier doubles every two characters while the number of distinct residuals never exceeds two: every strand beyond the second is a duplicate carrying different accumulated state. Regenerate with `cargo run --release --example thesis_figures -- frontier; tests/test_thesis_figures.rs` pins these values.

together: it runs inside *pDerivBC*, on the successors of one strand, and these duplicates arise *between* strands, at the point `step_frontier` concatenates the per-strand results.

This is the partial-derivative counterpart of Section 5.5.5’s finding for `deriv_bc`, and the two have one root cause seen from different sides. There, alternation branches that were structurally identical survived deduplication because their bit annotations made them syntactically distinct. Here, residuals that are syntactically identical survive because deduplication is never applied at the level where they meet. In both cases the bounded object is the set of residual *shapes*, and in both cases what actually grows is the set of shapes paired with accumulated parse-tree state.

Deduplicating across the frontier. The obvious question is whether the duplicates can simply be dropped. Moving `nub2` out of *pDerivBC* and applying it across the whole flattened frontier, keeping the first occurrence, is a change of one deduplication site, and it preserves the priority order for the same reason `nub2` does inside a single expansion: the frontier is built in priority order, so the first occurrence of a residual is the highest-priority strand that reaches it.

Whether that argument is enough to preserve the selected *tree* is a different question, and this thesis does not prove it. It was tested instead, by running both variants over the 300 generated expressions and 39 inputs of Section 8.4.2, 11 700 cases in total. The two agree on every case, on membership and on the exact tree returned, and the peak frontier over the whole run falls from 170 strands to 4.

The shipped `pderiv_bc` nonetheless keeps `nub2` where the reference puts it, for the reason given in Section 6.2: this parser’s correctness criterion is literal agreement with *pDerivBC*, and a change verified on 11 700 cases but not proved is a change to the thing being verified against. Section 8.7 records it as an available optimization and not an applied one, with the measurement above as the evidence for its value.

6.6 The Two Policies Compared

Section 4.3 constructed by hand the *second* valid derivation of $r_1 = (a + ab)(b + \varepsilon)$ against **ab** (`paper_r1`) that Chapter 5 set aside: $v' = (\text{Left } a, \text{Left } b)$, matching the *first* alternative of $a + ab$ against **a** alone instead of the second against **ab** in one step. That is this chapter's result, and both of its parsers have now been shown computing it. Figure 6.4 sets the two answers side by side.

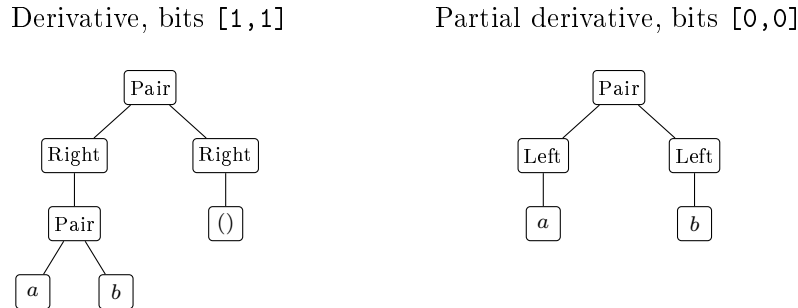


Figure 6.4: The two parse trees for $(a + ab)(b + \varepsilon)$ against "ab". Both flatten to "ab" and both are valid. The derivative parsers take the second alternative of $a + ab$, consuming both characters at once and leaving $b + \varepsilon$ to match empty; the partial-derivative parsers take the first, leaving $b + \varepsilon$ to consume the **b**.

The bit-strings underneath make the comparison exact. Four alternation choices exist in r_1 , two of which are reached on any given parse, and the two parsers disagree on both: $[1, 1]$ against $[0, 0]$. Nothing else differs. The same expression, the same input, the same membership answer, and the entire distinction between the two chapters compressed into two bits.

Failure behaviour is shared. Against **b** instead of **ab**, the first partial derivative of r_1 produces no residual at all, since neither alternative of $a + ab$ begins with **b**:

```
rule: pDerivBC('b', r1.r2) [nullable(r1)=false]
    = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
-> []
frontier after (0 residual(s)):
no nullable residual in this frontier [FAIL]
```

An empty frontier is the partial-derivative counterpart of `deriv_bc` simplifying to `phi` (Section 5.8), and it is detected the same way but earlier: both parsers return as soon as the frontier empties, without consuming the rest of the input, because a frontier with no strands has no way to acquire one. For a long input whose first character already fails, that is the difference between one step and n .

Both then emit the same error summary as Chapter 5's two, character for character:

```
Failure at position: 1 (1-indexed)
Found:               'b'
Expected:            'a'
```

The two chapters' constructions disagree, deliberately, on which tree a successful match produces, and agree exactly on the Boolean question of membership and on the diagnostics

reported for a non-match. Neither the disambiguation policy nor the choice between bits and closures is ever reached until the underlying walk has already found at least one live residual. The full reports are Appendix C.7 and Appendix C.8.

All five entry points at once. The `-parser all` mode runs every parser on one input and checks the three invariants this and the previous chapter claim, with no diagnostics level selected:

```
$ cargo run -- parse "(a|ab)(b|)" "ab" --parser all
Regex: (a|ab)(b|)
Input: "ab"
```

Parser	Policy	Result
DERIV_REC	POSIX	(Right (a, b), Right ()) -> "ab"
DERIV_LOOP	POSIX	(Right (a, b), Right ()) -> "ab"
DERIV_BC	POSIX	(Right (a, b), Right ()) -> "ab"
PDERIV_STD	NON-POSIX	(Left a, Left b) -> "ab"
PDERIV_BC	NON-POSIX	(Left a, Left b) -> "ab"

```
POSIX parsers agree
PDERIV parsers agree
NON-POSIX agrees with POSIX on membership
```

The three closing lines are the invariants themselves, checked on this input: the three derivative entry points must return identical trees, the two partial-derivative ones must return identical trees, and all five must agree on whether the input matched. Nothing requires the two *policies* to agree, and on this input they do not, which is the entire subject of both chapters. Section 8.4 runs the same check at scale.

What the substitution cost. Replacing $r \setminus x$ with $pd(r, x)$ changed three things, and only the first was intended. The representation changed, from one expression to a frontier of strands. The bound changed: Antimirov's theorem bounds the residual shapes, and Section 6.5 showed the frontier growing anyway, because what the algorithm carries is shapes paired with parse-tree state. And the answer changed, because the left-preference that *mkEps* applied to a single alternation had nowhere to live in a set, and *nub2*'s first-occurrence-wins rule took its place. The last of these is not a consequence of partial derivatives as such. It is a consequence of how this particular construction keeps its frontier finite, and Section 6.1 set out why a construction of this shape has little room to do otherwise.

7 External versus Internal Regex

Every example so far in this thesis has been written directly in `Regex` terms, the six-constructor core algebra of Section 2.2 that `deriv` and `pderiv` operate on. No real regular-expression user writes that way. A Snort rule, a SpamAssassin filter, or an ordinary call to `cargo run - match` arrives as an ordinary PCRE-style pattern *string*, `"[a-c]+\d{2}"` rather than `Seq(Star(...), ...)`. `src/frontend/` is the module that closes this gap: it parses that string into its own surface-syntax AST, then lowers that AST into the `Regex` every parser in Chapters 5 and 6 already understands.

It is modeled throughout on the `ExtPattern`, `Parse` and `Translate` modules of `Text.Regex.PDeriv`, from `luzhuomi/regex-pderiv`, Kenny Lu's own reference implementation of Sulzmann and Lu [2012]'s partial-derivative submatching. That is a second, independent reference alongside Sulzmann and Lu [2014]'s own teaching-page material, used here specifically because the `flops14`-extended paper itself has nothing to say about surface syntax: POSIX parsing is defined directly over the same six-constructor `Regex` algebra this thesis's core inherits, with no PCRE-facing frontend of its own.

Section 8.3's real-world corpus depends on everything described here, surface-syntax parsing, desugaring, and the substring-search padding scheme, to turn raw Suricata, SpamAssassin and `RegexLib` rule text into `Regex` values in the first place. The chapter closes (Section 7.4) with the price that translation charges: how large a `Regex` an ordinary-looking pattern actually becomes, which is what the four parsers then have to walk once per input character.

7.1 Two ASTs, One Lowering Step

`src/frontend/` is built around a deliberate separation into two abstract syntax trees connected by a single function, rather than parsing pattern text directly into `Regex`. Figure 7.1 gives the whole module in one view: a common path that every entry point follows, and two optional stages that only some of them switch on.

`ExtPat` (`ext_pattern.rs`) is the *external*, surface-syntax AST: marking and non-marking groups, `{m,n}` bounds, `\d/\w/\s` shorthands, `^/$` anchors, and a `greedy`: `bool` flag on every quantifier (`a*` against `a*?`). It holds everything a PCRE-style string can express, kept exactly as written, with nothing desugared yet. `Regex` (Section 2.2) is the *internal*, six-constructor core every parser derives or partial-derives directly: `Phi` | `Eps` | `Lit` | `Seq` | `Alt` | `Star`, with no groups, no bounds, no anchors and no laziness flag. Anything `ExtPat` can say that `Regex` cannot represent is desugared away by `translate` (Section 7.3) or, for `^` and `$` specifically, erased into `Eps` and handled one level up instead (Section 8.3.1).

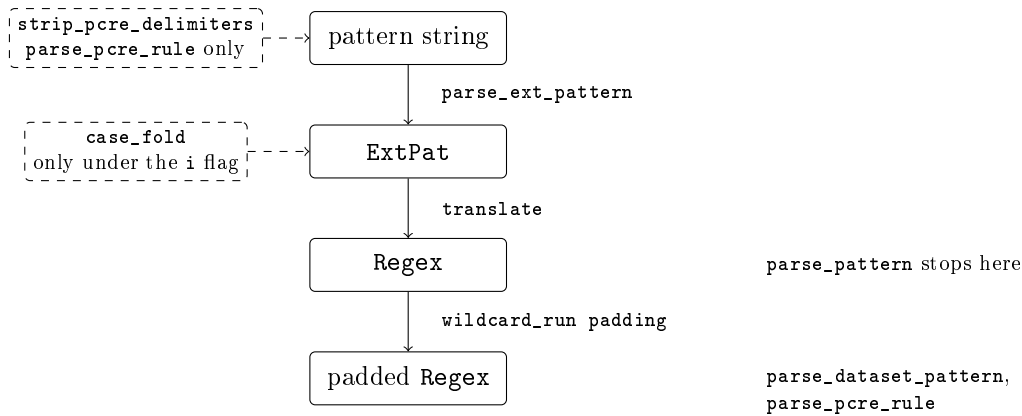


Figure 7.1: The frontend as a pipeline. The solid path is common to all three entry points; the dashed stages apply only where marked. `parse_pattern` stops at `Regex` and means full-string equality, while `parse_dataset_pattern` and `parse_pcre_rule` continue into the padding of Section 8.3.1.

Keeping `ExtPat` as an honest AST in its own right, rather than routing straight from the parser into `Regex` in a single pass, is what makes `translate` independently testable. The unit tests in `translate.rs` construct `ExtPat` values by hand and check their `Regex` output directly, with no dependency on the recursive-descent logic in `parse.rs` at all. It is the same separation-of-concerns argument that Section 5.3.6 makes for keeping `ParseTree` construction separate from `flatten` and `decode`, applied one layer further out, at the point where a PCRE-style string first becomes structured data at all.

The separation earns its keep a second time at the entry points. `mod.rs` exposes three of them, all sharing one parser and one `translate`, and differing only in what they do to the `ExtPat` in between:

Entry point	Behaviour
<code>parse_pattern</code>	<code>translate</code> alone: full-string match
<code>parse_dataset_pattern</code>	plus anchor-aware padding (Section 8.3.1)
<code>parse_pcre_rule</code>	plus <code>/.../i</code> delimiter stripping and case folding

Every example in Chapters 5 and 6, and every invocation of `cargo run - parse`, goes through `parse_pattern`, which is why `"(a|ab)(b|)"` against `"ab"` means the whole input and not a substring of it. The corpus of Section 8.3 goes through the other two, which is where real rule text is handled.

7.2 Parsing Surface Syntax

`frontend/parse.rs` is an ordinary recursive-descent parser over a `Chars` iterator, structured directly after the grammar of `Text.Regex.PDeriv.Parse`. Its productions correspond one for one, each handling exactly the surface-syntax layer its reference counterpart does:

Reference	This frontend	Layer handled
p_ere	parse_or	alternation
p_branch	parse_concat	concatenation
p_exp	parse_postfixed	postfix quantifiers
p_atom	parse_atom	a single atom
p_group	parse_group	(...) forms
p_charclass	parse_charclass	[...]
p_enum	parse_class_enum	class members
p_bound	parse_bound	{m,n}

The table is a list of names; Figure 7.2 is what those names mean operationally. Each production consumes exactly one syntactic layer and then calls the next one down, so the order of the calls *is* the precedence order, and the single back-edge from `parse_group` is what makes the grammar recursive at all.

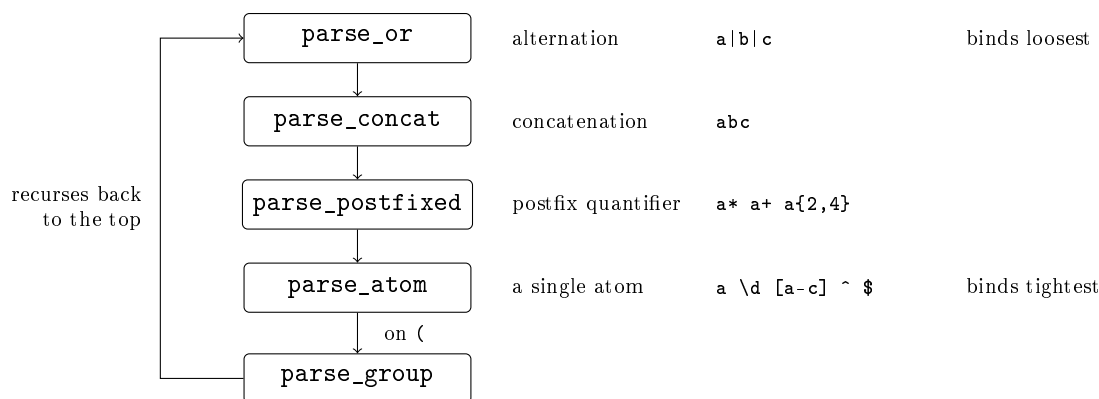


Figure 7.2: The recursive-descent ladder of `frontend/parse.rs`. Each production strips one layer and delegates the rest downward, so precedence falls out of the call order. `parse_group` is the only production that climbs back to the top, which is why parentheses override precedence and why nesting is unbounded.

Four deliberate departures from the reference:

- **Class shorthands are PCRE character classes, not literal escapes.** The reference’s `Translate.hs` reads a bare `EEscape c` as the literal character `c` always; this frontend special-cases `d/D/w/W/s/S` into character classes (Section 7.3’s `alphabet.rs`) instead, since real datasets use them that way throughout.
- **Backreferences and lookahead are rejected outright.** The backreferences `\1` to `\9`, and the four lookahead openers `(?=`, `(?!`, `(?<=` and `(?<!`, all produce a descriptive parse error rather than the reference’s literal-digit or three-digit octal-ASCII escape reading. Both describe languages that are not regular in the formal sense this thesis’s entire construction depends on, so silently mis-parsing them into *some* `Regex` would be a worse outcome than refusing them, and Section 8.3’s extraction pipeline depends on this refusal being loud and categorized rather than silent.
- **Named groups are accepted.** `(?<name>...)` and `(?P<name>...)` parse successfully, and the name is dropped during `translate`. A name carries no meaning once lowered to `Regex`, since this project has no capture-group semantics at all, but

rejecting a construct this common in real Suricata and RegexLib rules purely because the reference parser predates it would reject working patterns for no benefit. `(?P=name)`, a named *backreference*, is rejected along with the numeric ones.

- **\xHH hex escapes are accepted**, another addition absent from the reference, for the same reason.

Two small extensions exist for this thesis’s own sake rather than for the datasets. A literal ε character in the pattern text parses to `ExtPat::Eps`, and a literal `∅` to `ExtPat::Never`, so that Section 2.1’s own notation for ε and `∅` can be written directly on the command line. PCRE syntax has no way to spell either, and the alternative used throughout Chapters 5 and 6 is an empty alternative: `(b|)` is $b + \varepsilon$, because `parse_concat` returns `ExtPat::Empty` when a branch is empty rather than failing.

Bounds are parsed speculatively. `parse_bound` is the one place where the parser backtracks. A `{` in a pattern is only sometimes the start of a repetition bound, so `parse_bound` clones the iterator, tries to read a complete `{m}`, `{m,}` or `{m,n}` specification from the clone, and commits the clone back only on success. On failure the `{` is left unconsumed, and since `{` is in the parser’s `SPECIALS` set, the next `parse_atom` call reports it as unexpected.

This has a consequence worth recording, because it makes one check in `translate` unreachable. An inverted bound such as `a{4,2}` is rejected by `try_parse_bound_spec`, which returns `None` when the upper bound is below the lower one, so the error the user sees comes from the *atom* parser:

```
$ cargo run -- parse "a{4,2}" "aa"
Regex parse error: unexpected special character '{' here
```

rather than from `desugar_bound`’s own "upper bound is less than lower bound" message, which no input reaching it through `parse_pattern` can trigger. The message is less specific than the one that exists for the condition, which is worth knowing when reading extraction-failure categories in Section 8.3.

`parse_bound` additionally accepts a lazy-quantifier marker (`a*?`, `a+?`, `a{2,4}?`) and records it as `ExtPat`’s `greedy`: `bool` flag: correctly read, but, as Section 7.3 explains, not honored by `translate`. Grouping, alternation and concatenation otherwise follow the reference’s precedence exactly: alternation binds loosest, then concatenation, then postfix quantifiers, then a single atom, which may be a literal, an escape, a character class, a group or an anchor.

7.3 Translating ExtPat into Regex

`translate` (`frontend/translate.rs`) is a single structural recursion over `ExtPat`, one match arm per constructor, each either a direct one-to-one mapping or an explicit desugaring:

ExtPat	Regex
Empty, Eps	Eps
Never	Phi
Char(c)	Lit(c)
Group/GroupNonMarking(r)	translate(r) (grouping is precedence-only)
Or([r1, ..., rn])	left-folded Alt chain
Concat([r1, ..., rn])	left-folded Seq chain
Opt(r)	Alt(translate(r), Eps)
Plus(r)	Seq(translate(r), Star(translate(r)))
Star(r)	Star(translate(r))
Carat, Dollar	Eps (see Section 8.3.1)
Dot, Any(cs), NoneOf(cs)	sorted, deduplicated Alt of Lit
Escape(c)	class shorthand, or Lit(c)
Bound(r, lo, hi)	unrolled exact/optional repetition, below

The left-folding of `Or` and `Concat` is what makes the associativity convention of Section 2.1 concrete: `abc` becomes `Seq(Seq(a, b), c)`, and it is that tree, not the string, that every derivative in this thesis is taken of.

Character classes are ordinary alternation, not a primitive. `Any`, `NoneOf`, `Dot` and the class-shorthand escapes all route through `alt_of_chars`, which sorts and deduplicates the character set and left-folds it into a plain `Alt` chain of `Lit` nodes. `alphabet.rs` supplies the underlying sets: `alphabet()`, which is 98 characters (printable ASCII plus tab, newline and carriage return), `digit_chars`/`word_chars`/`space_chars` for `\d`/`\w`/`\s`, and `complement` (the alphabet minus a given set) for their negated and uppercase counterparts and for `[^...]`.

Every one of the four parsers only ever needs to understand `Regex`'s original six constructors, at the cost of a `\d` inside a real pattern becoming a ten-way `Alt` that the parser walks one branch at a time on every character. Section 7.4 measures how large that cost actually is.

Two consequences of drawing the alphabet this way are worth stating, since both are departures from PCRE that a rule author would not expect. The alphabet is ASCII only, so a negated class `[^a]` denotes 97 specific characters rather than "every character except `a`", and a non-ASCII input character matches no class at all. And `alphabet()` *includes* newline, so `.` here matches a newline where PCRE's `.` by default does not.

Bounds unroll literally, capped against blowup. `desugar_bound` builds `r` repeated exactly `lo` times, via `repeat_exact`, a left-folded `Seq` chain seeded with `Eps`, then appends either `Star(r)` for an unbounded upper end (`r{2,}`) or `hi - lo` further nested optional copies for a bounded one (`r{2,4}`), each layer of `optional_extra` being `Alt(Eps, Seq(r, ...))`. An exact bound `r{2}` takes a third path, returning `repeat_exact` alone with nothing appended. The `Eps` seed stays in the result rather than being smart-constructed away, so `a{2}` lowers to `Seq(Seq(Eps, Lit('a')), Lit('a'))` rather than `Seq(Lit('a'), Lit('a'))`. That costs one node and one nullable-test per derivative step, and it keeps `repeat_exact` a single unconditional fold with `Eps` as the natural identity for `lo == 0`.

Unlike every other desugaring in this table, this one can make a pattern arbitrarily large from a short piece of source text. `r{1586,}`, which is Section 8.3's own example of a

rejected real-world bound, would otherwise unroll into 1586 nested `Seq` nodes before the `Star` even starts. `MAX_BOUND`, at 1000 repetitions, is this project's own safety limit rather than something carried over from the reference, and `translate` refuses any bound past it with a descriptive error rather than silently building an oversized `Regex`:

```
$ cargo run -- parse "a{1586,}" "aa"
Regex parse error: bound {1586,} exceeds this frontend's cap of
1000 repetitions (literal unrolling would blow up the regex size)
```

Greedy and lazy quantifier syntax is parsed but not honored. Every `Opt`, `Plus`, `Star` and `Bound` match arm above ignores the `greedy` flag that `parse_bound` (Section 7.2) recorded, because `Regex` has no per-operator laziness concept at all. POSIX-versus-Greedy disambiguation in this thesis is a whole-parser policy, `deriv_std/deriv_bc` against `pderiv_std/pderiv_bc` (Section 4.3), rather than a per-quantifier one. `a*` and `a*?` therefore produce identical `Regex` values.

A pattern mixing lazy and greedy quantifiers, expecting the per-operator control PCRE engines provide, will parse successfully but not behave as its author intended. This is a genuine expressiveness gap between the surface syntax this frontend accepts and what `Regex` can distinguish, noted here rather than left to be discovered. The flag is nonetheless carried on `ExtPat` rather than discarded at parse time, so that the information survives to the one layer that could act on it if per-quantifier policy were ever added.

Case folding. `parse_pcre_rule` handles a `/PATTERN/FLAGS` wrapper, the form Suricata and SpamAssassin rules are written in. `strip_pcre_delimiters` splits the body from the flags, passing a pattern through unchanged if it is not `/`-delimited, and the `i` flag triggers `case_fold` on the `ExtPat` before `translate` runs. `case_fold` rewrites each literal letter into a two-way `Any`, so `a` becomes `[aA]`, and leaves digits, punctuation and class shorthands untouched, since those are already case-agnostic. Folding at the `ExtPat` layer rather than after lowering is what keeps this to a single recursion: at the `Regex` layer the same change would have to distinguish literals that came from a class from literals that came from source text, a distinction `translate` has by then already erased.

7.4 The Cost of Translation

Every desugaring in this chapter is exact, and several of them are expensive. Since the object that reaches Chapters 5 and 6 is the lowered `Regex` rather than the pattern string, the size of that object, and not the length of the pattern, is what the parsers' per-character work scales with. Counting nodes in the lowered result for a few patterns makes the translation's real price visible:

The purely structural desugarings stay small, and their shapes are worth seeing written out, since each is exactly what Section 7.3's table prescribes:

Pattern	Lowers to	Nodes
<code>a*</code>	<code>Star(Lit)</code>	2
<code>a?</code>	<code>Alt(Lit, Eps)</code>	3
<code>a+</code>	<code>Seq(Lit, Star(Lit))</code>	4
<code>a{2}</code>	<code>Seq(Seq(Eps, Lit), Lit)</code>	5
<code>a{2,}</code>	exact copies then <code>Star</code>	8
<code>a{2,4}</code>	exact copies then nested options	15

Character classes and search padding are what change the order of magnitude. A `\d` is a 10-way `Alt` chain at 19 nodes, `\w` a 63-way one at 125, `[^a]` a 97-way one at 193, and `.` a 98-way one at 195. An unanchored search for the three-character literal `abc` reaches 399.

That last figure is the one that matters most for Section 8.3. Searching for a three-character literal builds a 399-node expression, because each `wildcard_run()` is a 195-node alternation under a `Star` and two of them are needed. Every derivative step then walks that structure.

Figure 7.3 puts these on one logarithmic axis, together with two patterns closer to what the corpus actually contains. The span is the point: the same six constructors carry expressions differing by more than two orders of magnitude, and nothing about the pattern’s source length predicts where in that range it lands.

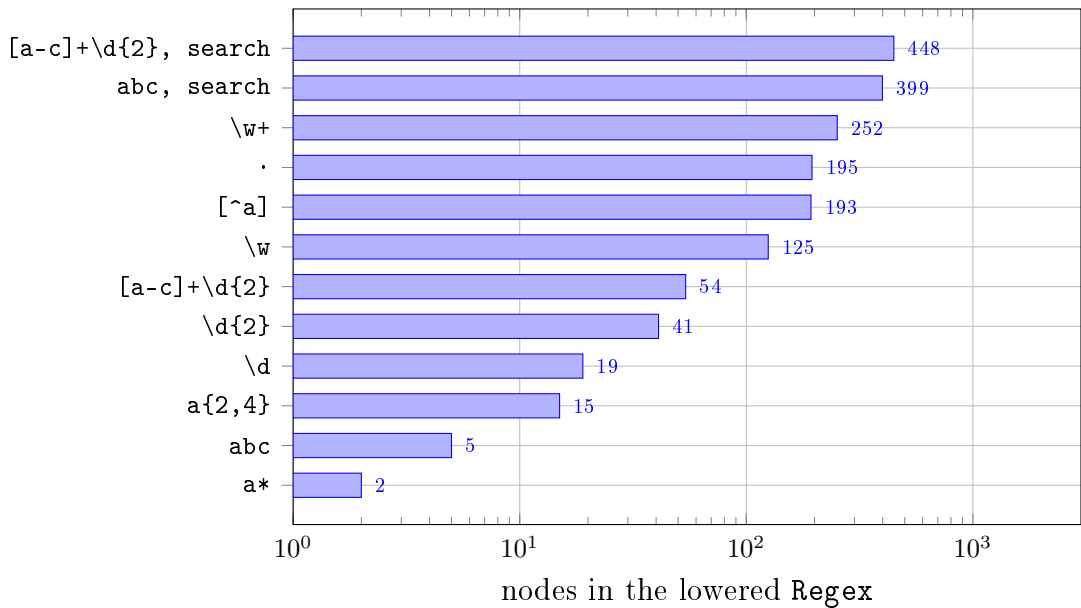


Figure 7.3: Lowered node counts, logarithmic horizontal axis. The three smallest are plain structural desugarings, with no character class involved. Every entry from `\d` upward expands one, and the two `search` entries additionally carry the padding of Figure 8.1. A realistic corpus pattern therefore starts around 450 nodes before any input is read. Regenerate with `cargo run --release --example thesis_figures -- lowered`.

This is the single largest constant factor separating this implementation from an engine with a native character-class representation, and it applies to the corpus patterns uniformly, since essentially all of them are unanchored searches containing at least one class shorthand.

The expansion is also visible in the diagnostics, in a way that is unhelpful rather than

merely large. Because the expected-set reported on a failure is read off the first-literal set of the expression at the point of failure, and a lowered class *is* that set, a single mismatched character against `.` reports all 98 alternatives individually:

```
$ cargo run -- parse "." "ä" --diag 1
Regex:  .
Input:  "ä"
Match:  false
Error:  position 1: found 'ä', expected '\t', '\n', '\r', ' ',
        '!', '"', '#', '$', '%', '&', ... , '}', or '~'
```

(abbreviated here; the tool prints the full list). The message is accurate, and it is exactly what the pattern denotes after lowering, but it no longer resembles the pattern the user wrote. A native class representation would let the diagnostics layer report **expected: any character** instead. It is also an instance of a general limit: once an abstraction has been compiled away, no amount of tracing can recover it, because the information is no longer in the object being traced.

The effect compounds rather than merely adding, which is why the chart climbs faster than the patterns lengthen. `\d{2}` lowers to two copies of the 19-node digit chain, reaching 41 nodes, and `\w+` to a 63-way alternation appearing twice, once for the required copy and once inside the **Star**, reaching 252. Stacking both effects, `[a-c]+\d{2}` is 54 nodes as a full-string pattern and 448 once padded for search, from twelve characters of source text.

Case folding is the one remaining multiplier, and it is comparatively cheap: `case_fold` turns each literal *letter* into a two-way **Alt** and leaves everything else alone, so `/abc/i` lowers to 405 nodes against `/abc/`'s 399, six more for three folded letters. Unlike the class expansion, this one scales with the number of letters in the pattern rather than with the size of the alphabet.

None of this changes which language is denoted, and none of it changes which parse tree any of the four parsers selects. It changes only how much structure each of them traverses per input character, which is exactly the quantity Section 8.5 measures.

8 Evaluation

Chapters 5 and 6 developed four parser combinations – `deriv_std`, `deriv_bc`, `pderiv_std`, `pderiv_bc` – each claimed to compute either the POSIX or the leftmost-first parse tree, by four structurally different constructions. This chapter verifies those claims empirically: Section 8.2 describes how the test suite checks the four parsers, and the three Boolean matchers of Chapter 3, against each other and against the paper’s own worked examples; Section 8.3 describes how a corpus of real-world regular expressions was assembled to test and benchmark against, alongside the synthetic patterns used elsewhere in this thesis; Section 8.4 reports the results of that testing; and Section 8.5 measures where the four parsers’ costs actually diverge from each other, on both the synthetic and real-world workloads.

8.1 Compared Engines

The primary object of comparison in this chapter is the four parser combinations against *each other*: two built on the derivative and two on the partial derivative, one tree-building and one bit-coded within each pair (Chapters 5-6) – since that internal comparison is what this thesis’s own theoretical claims (POSIX-correctness of the derivative parsers, agreement of all four on membership, the tree-versus-bit-coded performance trade-off) are actually about. A second, external comparison against established regular expression engines is motivated in Chapter 1.1 and positions these results relative to the wider field. It is implemented as an optional, feature-gated addition to this codebase (Rust’s `regex` crate as an ordinary optional dependency; Google’s RE2 via a small C++ shim over its own API, since RE2 offers no C API of its own) rather than a default one, specifically so that the default build stays free of any external regular-expression engine dependency – `cargo build/cargo test/cargo bench` never touch either engine unless the `external-engines` feature is explicitly enabled. Section 8.6 reports and discusses the resulting numbers; what follows briefly positions both engines conceptually, since that section refers back to this framing.

8.1.1 Rust’s `regex` Crate

Rust’s `regex` crate compiles a pattern to a Thompson-construction NFA and simulates it without backtracking, guaranteeing linear-time matching in the input length independent of the pattern – the same automaton-based guarantee Chapter 1.1 attributes to finite-automata approaches generally. Its disambiguation policy is leftmost-first, matching Section ??’s Greedy order rather than POSIX leftmost-longest; it offers no POSIX mode. That makes it the natural external comparison for this thesis’s partial-derivative

parsers (`pderiv_std/pderiv_bc`), which are also leftmost-first, on performance. Whether the two agree on the parse tree as well is a question this thesis does not settle: the partial-derivative construction is left-biased and commits on first success (Section 6.1.2), which is not by itself a claim that it implements the same policy the crate does.

8.1.2 RE2

Google’s RE2 is likewise automaton-based (Thompson NFA compiled lazily to a DFA on demand), explicitly designed to avoid the exponential backtracking blowup of backreference-supporting engines [Google LLC, 2024, Cox, 2007]. Unlike Rust’s `regex` crate, RE2 supports an explicit POSIX leftmost-longest mode alongside its default Perl-style leftmost-first mode, making it a comparison point for *both* disambiguation policies studied in this thesis, in one engine, depending on which mode is selected.

8.2 Testing Strategy

The test suite is split, following ordinary Rust convention, into unit tests (`#[cfg(test)] mod tests` blocks inside the `src/` files they exercise, testing private helper functions alongside public ones) and integration tests (`tests/`, exercising only the public API each parser or matcher module exposes). `tests/common/mod.rs` factors out what is shared across the integration suites: the two worked examples from the paper, `paper_r1` and `paper_r2` (Section 4.2), and two assertion helpers, `assert_round_trip` (checking `flatten(parse(r,w)) == w`, i.e. that a produced parse tree is actually evidence for the word it was built from) and `assert_parsers_agree` (comparing two `Option<ParseTree>` results with a labeled failure message identifying which two implementations disagreed).

8.3 Dataset Preparation

The synthetic patterns used throughout this thesis – `paper_r1`, `paper_r2`, the pathological (a^*) -style expressions of Section ?? – are deliberately small and hand-constructed, chosen to isolate one specific algorithmic property at a time. They say little about how any of the four parsers behaves on the regular expressions people actually write. Mamouras et al.’s disambiguation-robustness study [Mamouras et al., 2024] motivates closing that gap: it studies exactly the POSIX-versus-Greedy question this thesis studies, via real Snort/-Suricata intrusion-detection signatures, and finds concrete real-world patterns where the two policies visibly diverge. That paper provides no public dataset artifact of its own, so this thesis’s corpus was independently assembled from three families of real-world regular expression sources chosen to cover different "shapes" of real regex use, not three samples of the same thing:

- **Snort/Suricata** network intrusion-detection signatures (`pcre:"..."` rule options, sourced from the Emerging Threats Open ruleset) – literal-heavy, moderate complexity, the closest of the three to "everyday" regex use.
- **SpamAssassin** email spam-filtering rules (`body/header` Perl-regex rules) – content matching over unstructured text, a different shape than network signatures.
- **RegexLib** crowd-sourced general-purpose patterns (validators, extractors) – the

opposite end, where chained lookahead assertions are idiomatic, stress-testing the frontend’s rejection path hardest of the three.

Extraction. Each source embeds its patterns in a different surrounding syntax, so each needs its own extractor (`examples/extract_dataset.rs`): Suricata’s rule lines carry a `pcre:"/PATTERN/FLAGS"`; option; SpamAssassin’s carry a bare `body/header NAME = /PATTERN/FLAGS` without the Suricata’s trailing-quote convention; RegexLib’s sample file lists plain patterns directly, one (possibly multi-line) entry per blank-line-delimited block. All three converge on the same `/PATTERN/FLAGS` PCRE delimiter convention once extracted, parsed by `frontend::parse_pcre_rule` – including the case-insensitive `i` flag, applied by case-folding every literal character in the translated `ExtPat` to an `Any` of both cases (Chapter 7’s frontend) rather than as a runtime matching option, since `Regex` itself has no case-insensitivity concept.

Filtering. Real rules routinely use constructs no derivative-based engine over the six-constructor `Regex` of Section 2.2 can express at all – backreferences and lookahead assertions describe languages that are not regular in the formal sense this thesis’s entire construction depends on. `parse_pcre_rule` rejects both outright with a descriptive error rather than silently mis-parsing them, and `extract_dataset` tallies every rejection by cause (`backreference`, `lookaround`, or `other` – chiefly bound expressions such as `{1586,}` that would unroll past this frontend’s repetition cap, and a handful of unsupported `(?...)` group syntaxes). Re-running this pipeline against the tracked raw sources gives the current triage:

Format	Extracted	Accepted	Backreference	Lookaround	Other
Suricata	231	209 (90.5%)	5	11	6
SpamAssassin	82	69 (84.1%)	1	11	1
RegexLib	499	407 (81.6%)	8	55	29
Total	812	685 (84.4%)	14	77	36

Lookaround dominates the rejections and is concentrated almost entirely in RegexLib, matching that family’s idiomatic use of chained assertions; Suricata and SpamAssassin reject mostly on the same handful of negative-lookahead idioms (guarding against one specific following word). The accepted 685 patterns, deduplicated across all three formats into `data/corpus_patterns.txt` (665 lines after deduplication), are what Section 8.5.5’s dataset benchmark actually runs against.

Unanchored matching. Real rules are *written* as search patterns: a Suricata rule fires on seeing a match anywhere in a packet payload, not on the payload being exactly that pattern. They are therefore parsed through the frontend’s two search entry points, `parse_dataset_pattern` and `parse_pcre_rule`, instead of the full-string-equality `parse_pattern` used everywhere else in this thesis’s own examples. That difference is not cosmetic: real dataset patterns, run unanchored the way they have to be, are measurably bigger than the pattern text suggests and expose scaling behaviour the small synthetic patterns of Section 8.5.4 do not. The mechanism is worth setting out before the measurements that depend on it.

8.3.1 Substring Search: How the Corpus Is Run

No parser in Chapters 5 and 6 answers any question but "does this entire input derive to nullable". There is no "found somewhere inside a longer string" mode anywhere below the frontend, and neither [Sulzmann and Lu \[2014\]](#) nor Sulzmann's teaching-page material defines one: POSIX and Antimirov derivatives are inherently full-match operations. Real rules, though, are *written* as search patterns. A Suricata rule fires on seeing its pattern anywhere in a packet payload, not on the payload being exactly that pattern.

`frontend/mod.rs` therefore implements substring search entirely at this layer, before a `Regex` ever reaches a parser, by padding the translated core with `wildcard_run()`, a `Star` over a 98-way `Alt` of the full alphabet meaning "any run of characters at all", on whichever side lacks an explicit anchor. Figure 8.1 gives the four cases, with the size of each lowered result alongside.

<code>^abc\$</code>	<table><tr><td>abc</td></tr></table>	abc	9 nodes, no padding		
abc					
<code>^abc</code>	<table><tr><td>abc</td><td>Σ^*</td></tr></table>	abc	Σ^*	204 nodes, end padded	
abc	Σ^*				
<code>abc\$</code>	<table><tr><td>Σ^*</td><td>abc</td></tr></table>	Σ^*	abc	204 nodes, start padded	
Σ^*	abc				
<code>abc</code>	<table><tr><td>Σ^*</td><td>abc</td><td>Σ^*</td></tr></table>	Σ^*	abc	Σ^*	399 nodes, both sides padded
Σ^*	abc	Σ^*			

Figure 8.1: Anchor-aware padding under the search entry points. Each Σ^* is one `wildcard_run()`, 196 nodes on its own, so the padding dominates the lowered expression whenever the pattern is short. The fully anchored case still costs 9 rather than 5 nodes, because the erased `^` and `$` leave an `Eps` apiece behind in the concatenation.

`starts_with_carat` and `ends_with_dollar` (`ext_pattern.rs`) decide which sides need padding by walking into `Concat` and `Group` to find a leading `^` or trailing `$` even when nested, before `Carat` and `Dollar` are themselves translated away to `Eps`. Under the search-oriented entry points `parse_dataset_pattern` and `parse_pcre_rule`, as against `parse_pattern`'s plain full-string equality used everywhere else in this thesis's own examples, deciding which side of this padding is skipped is the anchors' only real effect.

Because the anchors are recognized structurally rather than positionally, a `^` that is not at the start, as in `a^b`, is not an anchor at all. It still parses to `Carat` and still translates to `Eps`, so it contributes a node to the lowered expression but nothing to the language:

```
$ cargo run -- parse "a^b" "ab" --diag 1
Regex:  a^b
Input:  "ab"
Match:  true
Tree:   ((a, ()), b)
```

The `()` in the middle of that tree is the erased anchor. PCRE would treat the same pattern as unmatchable outside multiline mode. This is a silent difference rather than an error, and it is the one place in the frontend where a mis-written pattern is neither rejected nor faithfully represented.

Edge case. An unanchored pattern already nullable on empty input, with `"a*"` the simplest example, gets padded on both sides regardless of its own content. The padding's

own empty match, combined with either `Star`, is then enough by itself to match *any* string at all, not just runs of `a`. This is correct substring-search semantics, since "some substring matches" is satisfied by the empty substring that every string contains, and `mod.rs`'s own test `unanchored_nullable_pattern_matches_anything` asserts exactly this behaviour rather than treating it as a defect. It is worth flagging nonetheless, because such a pattern carries no information in a corpus: it matches every input and so contributes nothing to Section 8.3's measurements beyond its own parse cost.

anchors are not a language feature here. Recognising `^` and `$` structurally, and letting them do nothing but suppress one side of the padding, is a decision about the corpus pipeline and not an account of what anchors mean. This thesis's core algebra has six constructors and no notion of a position within an input, so there is nowhere in it for an anchor to live: a derivative consumes a character and knows nothing about where that character sits. Extended matchers that do support anchors get them from a richer algebra. [Varatalu et al. \[2023\]](#) handle intersection, complement and lookarounds on top of derivatives, and treat anchors as expressible through lookarounds rather than as primitives of their own. Adopting that route would mean extending the core algebra and every derivative rule defined over it, which is well beyond the scope of this implementation, but it is the direction in which the gap described above closes, and it is the reason the gap is a scope boundary rather than an oversight.

What this dataset does not (yet) give. The corpus was triaged for "does it parse at all," not for "does POSIX visibly diverge from the partial-derivative answer on this pattern the way `paper_r1`/`paper_r2` do", and unlike Mamouras et al.'s own disambiguation-robustness analysis [[Mamouras et al., 2024](#)], this thesis does not (yet) identify which, if any, of the 685 accepted real-world patterns are themselves disambiguation-non-robust in their sense. Combining this dataset with their static-analysis technique to find such patterns automatically, or constructing inputs that provoke visible divergence between the two constructions on the existing corpus directly, is a natural extension of this evaluation left to future work (Section 8.7).

8.4 Correctness Verification

8.4.1 Paper Examples Reproduced

Both worked examples from Sulzmann and Lu's paper are reproduced as executable tests rather than only as the manual derivations of Sections 4.3 and 5.1.1: `parse_recursive_paper_r1_on_ab` and `parse_recursive_paper_r2_on_ab` (`tests/test_deriv_std.rs`) assert the exact POSIX tree shape derived by hand in Section 5.1.1; `paper_r1_computes_greedy_not_posix` and `paper_r2_computes_greedy_not_posix` (`src/parsers/pderiv_bc/parse.rs`) assert both the expected tree shape of Section ?? *and* that it disagrees with the POSIX answer on the identical input – encoding the divergence itself, not merely each answer separately, as the property under test. Every one of these four assertions currently passes, confirming this implementation reproduces both documented policies exactly on the paper's own examples.

8.4.2 Cross-Implementation Equivalence

Following the agreement-testing strategy of Section 8.2, the full test suite checks, and at the time of writing passes, agreement at every layer this thesis introduces more than one implementation of the same specification:

- `match_naive = match_deriv = match_pderiv` on membership, across the hand-picked cases of `tests/test_matchers.rs`.
- `parse_recursive` and `parse_loop` produce identical parse trees on every shared test case (Section 5.3.5), including `recursive_traced_and_loop_traced_agree_on_all_paper_c` comparing full derivation traces, not just final answers.
- `deriv_bc` agrees with `deriv_std` (via `parse_recursive`) on every shared test case (`bitcoded_agrees_with_recursive`, `tests/test_deriv_bc.rs`), the empirical counterpart of Section 5’s claim that both compute the identical POSIX tree by different means.
- `pderiv_std` and `pderiv_bc` agree exactly, not just on membership, on every hand-picked and fuzzed case (Section 8.2).
- All four parser combinations agree on membership wherever the two constructions are not known to diverge, and each pair agrees exactly on tree shape within itself: `deriv_std` with `deriv_bc`, and `pderiv_std` with `pderiv_bc`. This is exercised directly by the CLI’s `--parser all` comparison mode (`src/cli/parser.rs`), which reports all three checks side by side for a given regex/input pair on demand.

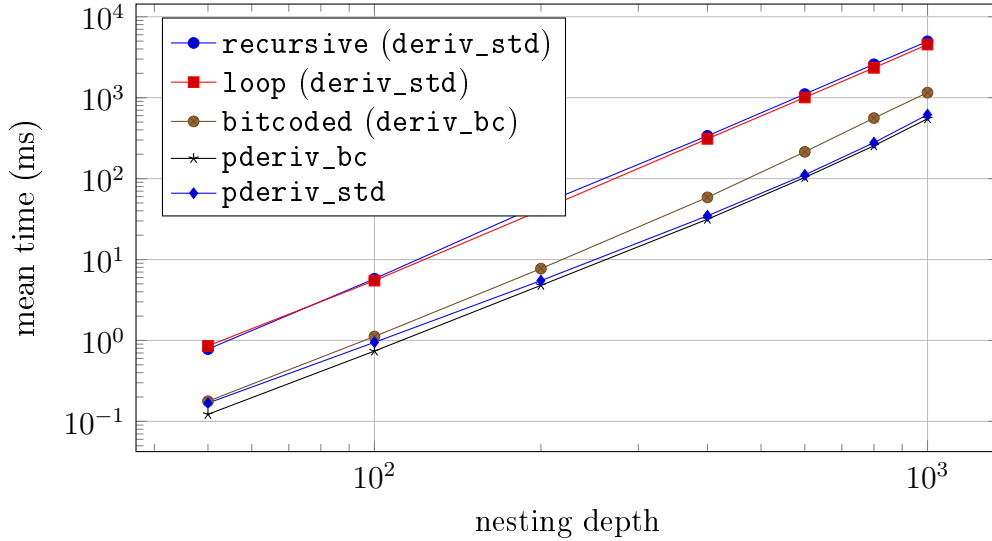
No disagreement has been observed in any of these checks against the current implementation; a regression in any one parser would show up first as a failure here, in the specific pairing that parser participates in, before it could show up as an incorrect answer with no diagnosis of which implementation is at fault.

8.5 Performance Analysis

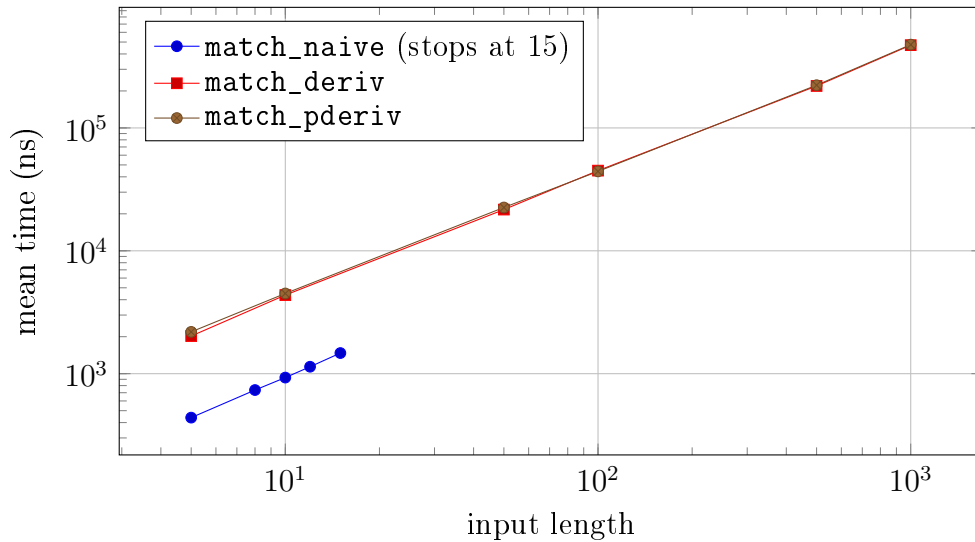
8.5.1 Stack Depth: Recursive vs. Iterative Construction

8.5.2 Heap Allocation: deriv_std vs. deriv_bc

8.5.3 Derivative-Based vs. Partial-Derivative-Based



8.5.4 Pathological Inputs



8.5.5 Benchmark Results

Parser	$n=5$	$n=10$	$n=15$	$n=18$
recursive (deriv_std)	8.0 μ s	110.7 μ s	1.427 ms	6.052 ms
bitcoded (deriv_bc)	11.4 μ s	183.8 μ s	4.175 ms	49.491 ms
pderiv_bc	13.4 μ s	210.6 μ s	2.207 ms	9.236 ms
pderiv_std	22.4 μ s	323.9 μ s	3.374 ms	14.263 ms

Parser	Mean time / sweep
<code>deriv_std_rec</code>	1.311 s
<code>deriv_std_loop</code>	1.091 s
<code>deriv_bc</code>	643.5 ms
<code>pderiv_bc</code>	159.2 ms
<code>pderiv_std</code>	158.6 ms

Parser	Mean time
<code>deriv_std_rec</code>	1.000 s
<code>deriv_bc</code>	81.1 ms

8.6 Comparison with Existing Engines

8.6.1 Performance

Engine	Mean time / sweep	vs. <code>regex</code>
<code>deriv_std_rec</code> (POSIX, recursive)	1.503 s	×1,415,254
<code>deriv_std_loop</code> (POSIX, loop)	1.218 s	×1,146,893
<code>deriv_bc</code> (POSIX, bit-coded)	690.74 ms	×650,417
<code>pderiv_bc</code> (partial derivative, bit-coded)	171.85 ms	×161,815
<code>pderiv_std</code> (partial derivative, tree)	173.28 ms	×163,167
<code>regex</code> (Rust crate)	1.06 μs	×1
<code>re2_perl</code> (leftmost-first)	6.61 μs	×6.2
<code>re2_posix</code> (leftmost-longest)	2.75 μs	×2.6

Engine	Mean time / sweep	vs. <code>regex</code>
<code>match_deriv</code>	273.07 ms	×259,815
<code>match_pderiv</code>	134.95 ms	×128,397
<code>regex</code> (Rust crate)	1.05 μs	×1
<code>re2_perl</code> (leftmost-first)	6.63 μs	×6.3
<code>re2_posix</code> (leftmost-longest)	2.72 μs	×2.6

8.6.2 Standardized Benchmarking via rebar

8.7 Optimization Challenges and Insights

Two optimizations were identified while implementing the four parsers. Neither is applied in the shipped code, and in both cases the reason is the same: the change is worth more than it costs only if correctness is measured against something other than the reference construction. They are recorded here with the measurements that establish their value, since those measurements are the product of this evaluation rather than of the implementation.

Character classes in place of literal alternation. The core algebra of Chapter 2 has six constructors and no character class among them, so the frontend lowers every class to an explicit alternation (Section 7.3). The cost of that decision is visible in the lowered node counts of Section 7.4: a `\d` becomes ten nodes, a `\w` becomes 125, and a wildcard becomes 195. The padding a search pattern receives is one `wildcard_run()` of 196 nodes

per unanchored side, so an unanchored three-letter literal reaches 399 nodes against the 5 its own content needs.

Every one of those nodes is structure the parser walks once per input character. A dedicated class constructor carrying a set of characters would collapse each of them to a single node whose derivative is one membership test, which changes the per-character work from linear in the size of the class to constant. Section 8.5’s dataset measurements are dominated by exactly this effect, since essentially every real-world pattern in the corpus is an unanchored search containing at least one class shorthand.

The optimization is not applied because it is not local. A seventh constructor has to be understood by `nullable`, `deriv`, `simp`, `mkEps`, `inj`, the annotated `ARegex` and its refined derivative, `pDerivBC`, and `decode`, which is every definition in Chapters 5 and 6. The four parsers would then no longer be transcriptions of the constructions they are transcriptions of, which is the property the correctness argument of Section 8.4 rests on. There is a second benefit worth recording alongside the performance one: because the expected set reported on a failure is read off the first-literal set of the expression, a mismatch against a lowered wildcard currently reports all 98 alternatives individually, where a native class could report `expected: any character`.

Frontier-wide deduplication. The partial-derivative parsers apply `nub2` inside a single expansion and never across the flattened frontier, which is what lets the frontier double on the expressions of Section 6.5 while the number of distinct residuals stays bounded. Moving that deduplication to the frontier as a whole is a change of one call site. Measured over the same 11700 cases as Section 8.4.2, it changes no answer, on membership or on the tree returned, and takes the peak frontier from 170 strands to 4.

It is not applied for the reason given in Section 6.2: this parser’s correctness criterion is literal agreement with the reference `pDerivBC`, and a change verified on finitely many cases without a proof is a change to the thing being verified against. The measurement establishes that the optimization is available and what it is worth; adopting it would require replacing agreement with an argument.

8.8 Discussion of Results

9 Related Work and Conclusion

9.1 Related Work

9.2 Conclusion

Bibliography

- Valentin Antimirov. Partial derivatives of regular expressions and finite automaton constructions. *Theoretical Computer Science*, 155(2):291–319, 1996. doi: 10.1016/0304-3975(95)00182-4.
- Janusz A. Brzozowski. Derivatives of regular expressions. *Journal of the ACM*, 11(4): 481–494, 1964. doi: 10.1145/321239.321249.
- Russ Cox. Regular expression matching can be simple and fast (but is slow in Java, Perl, PHP, Python, Ruby, ...). <https://swtch.com/~rsc/regexp/regexp1.html>, 2007. Accessed: 2026-08-05.
- Alain Frisch and Luca Cardelli. Greedy regular expression matching. In *Automata, Languages and Programming (ICALP 2004)*, volume 3142, pages 618–629. Springer, 2004. doi: 10.1007/978-3-540-27836-8_53.
- Google LLC. RE2: A fast, safe, POSIX-compatible regular expression library. <https://github.com/google/re2>, 2024. Accessed: 2026-08-05.
- IEEE and The Open Group. Posix.1-2017: Base definitions, chapter 9 (regular expressions), 2017. IEEE Std 1003.1-2017.
- Konstantinos Mamouras, Alexis Le Glaunec, Wu Angela Li, and Agnishom Chattopadhyay. Static analysis for checking the disambiguation robustness of regular expressions. *Proceedings of the ACM on Programming Languages*, 8(PLDI):231, 2024. doi: 10.1145/3656461.
- Martin Sulzmann. Programming paradigms: Regular expressions. <https://sulzmann.github.io/ProgrammingParadigms/pp-regular-expressions.html>, 2026. Accessed: 2026-08-08.
- Martin Sulzmann and Kenny Zhuo Ming Lu. Regular expression sub-matching using partial derivatives. In *PPDP*, pages 79–90, 2012. doi: 10.1145/2370776.2370788.
- Martin Sulzmann and Kenny Zhuo Ming Lu. Posix regular expression parsing with derivatives. In *Functional and Logic Programming (FLOPS 2014)*, volume 8475. Springer, 2014.
- The Rust Project Developers. Using `Box<T>` to point to data on the heap. The Rust Programming Language, 2026a. URL <https://doc.rust-lang.org/book/ch15-01-box.html>. Accessed: 2026-09-11.

- The Rust Project Developers. Enums and pattern matching. The Rust Programming Language, 2026b. URL <https://doc.rust-lang.org/book/ch06-00-enums.html>. Accessed: 2026-09-11.
- The Rust Project Developers. The `match` control flow construct. The Rust Programming Language, 2026c. URL <https://doc.rust-lang.org/stable/book/ch06-02-match.html>. Accessed: 2026-09-11.
- The Rust Project Developers. `std::hash`. Rust Standard Library, 2026d. URL <https://doc.rust-lang.org/std/hash/index.html>. Accessed: 2026-09-11.
- The Rust Project Developers. `HashSet` in `std::collections`. Rust Standard Library, 2026e. URL <https://doc.rust-lang.org/std/collections/struct.HashSet.html>. Accessed: 2026-09-11.
- The Rust Project Developers. Enumerated types. The Rust Reference, 2026f. URL <https://doc.rust-lang.org/reference/types/enum.html>. Accessed: 2026-09-11.
- The Rust Project Developers. Match expressions. The Rust Reference, 2026g. URL <https://doc.rust-lang.org/reference/expressions/match-expr.html>. Accessed: 2026-09-11.
- The Rust Project Developers. Types. The Rust Reference, 2026h. URL <https://doc.rust-lang.org/reference/types.html>. Accessed: 2026-09-11.
- Ian Erik Varatalu, Margus Veanes, and Juhan-Peep Ernits. Derivative based extended regular expression matching supporting intersection, complement and lookarounds. <https://arxiv.org/abs/2309.14401>, 2023. Accessed: 2026-09-18.

A Code

A.1 Core Types

Regex

src/types/regex.rs

```
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub enum Regex {
    /// Empty language: L(Phi) = {}
    Phi,

    /// Empty word: L(Eps) = {epsilon}
    Eps,

    /// Single character: L(Lit('a')) = {"a"}
    Lit(char),

    /// Sequence: L(Seq(r,s)) = { v++w | v in L(r), w in L(s) }
    Seq(Box<Regex>, Box<Regex>),

    /// Alternative: L(Alt(r,s)) = L(r) union L(s)
    Alt(Box<Regex>, Box<Regex>),

    /// Kleene star: L(Star(r)) = {epsilon} union L(r).L(Star(r))
    Star(Box<Regex>),
}
```

ARegex

src/types/aregex.rs

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum ARegex {
    /// Empty language
    Phi,
    /// bs @ eps
    Eps(Vec<bool>),
    /// bs @ 1
    Lit(Vec<bool>, char),
    /// bs @ (ri1 (+) ri2)
    Alt(Vec<bool>, Box<ARegex>, Box<ARegex>),
    /// bs @ (ri1 ri2)
    Seq(Vec<bool>, Box<ARegex>, Box<ARegex>),
    /// bs @ ri*
    Star(Vec<bool>, Box<ARegex>),
}
```

ParseTree

src/types/tree.rs

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum ParseTree {
    /// Empty parse tree (for epsilon)
    Empty,

    /// Literal character
    Char(char),
}
```

```

    /// Pair for concatenation (v1, v2)
    Pair(Box<ParseTree>, Box<ParseTree>),

    /// Left injection for alternative (Left v)
    Left(Box<ParseTree>),

    /// Right injection for alternative (Right v)
    Right(Box<ParseTree>),

    /// List for Kleene star iterations [v1, v2, ..., vn]
    Star(Vec<ParseTree>),
}

```

flatten
src/types/tree.rs

```

/// Flattens parse tree to underlying word (string)
pub fn flatten(v: &ParseTree) -> String {
    match v {
        ParseTree::Empty => String::new(),
        ParseTree::Char(c) => c.to_string(),
        ParseTree::Pair(v1, v2) => format!("{}", flatten(v1), flatten(v2)),
        ParseTree::Left(v) => flatten(v),
        ParseTree::Right(v) => flatten(v),
        ParseTree::Star(vs) => vs.iter().map(flatten).collect(),
    }
}

```

A.2 Shared Algorithms

nullable
src/regex/nullable.rs

```

/// Decides whether epsilon is in L(r)
pub fn nullable(r: &Regex) -> bool {
    match r {
        Regex::Phi => false,
        Regex::Eps => true,
        Regex::Lit(_) => false,
        Regex::Alt(r, s) => nullable(r) || nullable(s),
        Regex::Seq(r, s) => nullable(r) && nullable(s),
        Regex::Star(_) => true,
    }
}

```

deriv
src/regex/deriv.rs

```

/// Computes derivative of r based on character x
pub fn deriv(r: &Regex, x: char) -> Regex {
    match r {
        Regex::Phi => Regex::Phi,
        Regex::Eps => Regex::Phi,
        Regex::Lit(c) => {
            if *c == x { Regex::Eps } else { Regex::Phi }
        }
        Regex::Alt(r1, r2) => {
            Regex::alt(deriv(r1, x), deriv(r2, x))
        }
        Regex::Seq(r1, r2) => {
            if nullable(r1) {
                let dr1 = Regex::seq(deriv(r1, x), *r2.clone());
                Regex::alt(dr1, deriv(r2, x))
            } else {
                Regex::seq(deriv(r1, x), *r2.clone())
            }
        }
        Regex::Star(r1) => {
            Regex::seq(deriv(r1, x), Regex::star(*r1.clone()))
        }
    }
}

```

```
}
```

mk_eps
src/regex/mk_eps.rs

```
pub fn mk_eps(r: &Regex) -> ParseTree {
    match r {
        Regex::Eps => ParseTree::Empty,

        Regex::Star(_) => ParseTree::Star(Vec::new()),

        Regex::Seq(r1, r2) => {
            let v1 = mk_eps(r1);
            let v2 = mk_eps(r2);
            ParseTree::Pair(Box::new(v1), Box::new(v2))
        }

        Regex::Alt(r1, r2) => {
            if nullable(r1) {
                ParseTree::Left(Box::new(mk_eps(r1)))
            } else {
                ParseTree::Right(Box::new(mk_eps(r2)))
            }
        }

        Regex::Lit(c) => panic!("mk_eps called on Lit('{ }')", c),
        Regex::Phi => panic!("mk_eps called on Phi"),
    }
}
```

decode
src/regex/decode.rs

```
pub fn decode(r: &Regex, bs: &[bool]) -> ParseTree {
    let (v, rest) = decode_inner(r, bs);
    assert!(
        rest.is_empty(),
        "decode: {} leftover bit(s) after decoding {:?}",
        rest.len(),
        r
    );
    v
}
```

simplify
src/regex/simplify.rs

/// Simplify regular expression using algebraic laws

```
pub fn simplify(r: Regex) -> Regex {
    match r {
        Regex::Seq(r, s) => {
            let r = simplify(*r);
            let s = simplify(*s);
            match (&r, &s) {
                (Regex::Phi, _) => Regex::Phi,
                (_, Regex::Phi) => Regex::Phi,
                (Regex::Eps, _) => s,
                (_, Regex::Eps) => r,
                _ => Regex::seq(r, s),
            }
        }
        Regex::Alt(r, s) => {
            let r = simplify(*r);
            let s = simplify(*s);
            match (&r, &s) {
                (Regex::Phi, _) => s,
                (_, Regex::Phi) => r,
                _ if r == s => r,
                _ => Regex::alt(r, s),
            }
        }
        Regex::Star(r) => {
            let r = simplify(*r);

```

```

        match r {
            Regex::Eps => Regex::Eps,
            Regex::Phi => Regex::Eps,
            r => Regex::star(r),
        }
    }
    other => other,
}
}

```

A.3 Matchers

match_naive
src/matchers/match_naive.rs

```

pub fn match_naive(input: &str, r: &Regex) -> bool {
    let chars: Vec<char> = input.chars().collect();
    let len = chars.len();
    match_naive_range(&chars, 0, len, r)
}

```

match_deriv
src/matchers/match_deriv.rs

```

pub fn match_deriv(input: &str, r: &Regex) -> bool {
    let mut current = r.clone();
    for c in input.chars() {
        current = simplify(deriv(&current, c));
    }
    nullable(&current)
}

```

match_pderiv
src/matchers/match_pderiv.rs

```

pub fn match_pderiv(input: &str, r: &Regex) -> bool {
    let mut states: HashSet<Regex> = HashSet::new();
    states.insert(r.clone());

    for c in input.chars() {
        let mut next_states: HashSet<Regex> = HashSet::with_capacity(states.len());
        for state in &states {
            next_states.extend(pderiv(state, c));
        }
        if next_states.is_empty() {
            return false;
        }
        states = next_states;
    }

    states.iter().any(nullable)
}

```

A.4 The deriv_std Parser

inject
src/parsers/deriv_std/inject.rs

```

pub fn inject(r: &Regex, l: char, v: ParseTree) -> ParseTree {
    match r {
        Regex::Lit(c) => {
            assert!(*c == l);
            assert!(matches!(v, ParseTree::Empty));
            ParseTree::Char(l)
        }
        Regex::Star(r1) => {
            match v {

```

```

    ParseTree::Pair(v1, vs) => {
        let v1_inj = inject(r1, 1, *v1);
        let mut iterations = vec![v1_inj];
        if let ParseTree::Star(rest) = *vs {
            iterations.extend(rest);
        }
        ParseTree::Star(iterations)
    }
    _ => panic!("inject on Star: expected Pair, got {:?}", v),
}

Regex::Seq(r1, r2) => {
    let r1_nullable = nullable(r1);

    if !r1_nullable {
        match v {
            ParseTree::Pair(v1, v2) => {
                let v1_inj = inject(r1, 1, *v1);
                ParseTree::Pair(Box::new(v1_inj), v2)
            }
            _ => panic!("inject on Seq (non-nullable r1): expected Pair, got {:?}", v),
        }
    } else {
        match v {
            ParseTree::Left(v_pair) => {
                match *v_pair {
                    ParseTree::Pair(v1, v2) => {
                        let v1_inj = inject(r1, 1, *v1);
                        ParseTree::Pair(Box::new(v1_inj), v2)
                    }
                    _ => panic!(
                        "inject on Seq (nullable r1, Left): expected Pair, got {:?}",
                        v_pair
                    ),
                }
            }
            ParseTree::Right(v2) => {
                let v2_inj = inject(r2, 1, *v2);
                let v1_eps = mk_eps(r1);
                ParseTree::Pair(Box::new(v1_eps), Box::new(v2_inj))
            }
            _ => panic!("inject on Seq (nullable r1): expected Left or Right, got {:?}", v),
        }
    }
}

Regex::Alt(r1, r2) => {
    match v {
        ParseTree::Left(v1) => {
            ParseTree::Left(Box::new(inject(r1, 1, *v1)))
        }
        ParseTree::Right(v2) => {
            ParseTree::Right(Box::new(inject(r2, 1, *v2)))
        }
        _ => panic!("inject on Alt: expected Left or Right, got {:?}", v),
    }
}

Regex::Eps => panic!("inject called on Eps"),
Regex::Phi => panic!("inject called on Phi"),
}
}

```

A.5 The deriv_bc Parser

`src/parsers/deriv_bc/internalize.rs`

```

pub fn fuse(bs: &[bool], ri: ARegex) -> ARegex {
    if bs.is_empty() { return ri; }
}

```

```

match ri {
  ARegex::Phi => ARegex::Phi,
  ARegex::Eps(p) => ARegex::Eps(combine(bs, p)),
  ARegex::Lit(p, c) => ARegex::Lit(combine(bs, p), c),
  ARegex::Alt(p, r1, r2) => ARegex::Alt(combine(bs, p), r1, r2),
  ARegex::Seq(p, r1, r2) => ARegex::Seq(combine(bs, p), r1, r2),
  ARegex::Star(p, r) => ARegex::Star(combine(bs, p), r),
}
}

```

deriv_bc
src/parsers/deriv_bc/deriv.rs

```

/// Bit-coded derivative (paper Figure 5)
pub fn deriv_bc(ri: ARegex, l: char) -> ARegex {
  match ri {
    ARegex::Phi => ARegex::Phi,
    ARegex::Eps(_) => ARegex::Phi,
    ARegex::Lit(bs, c) => {
      if c == 1 {
        ARegex::Eps(bs)
      } else {
        ARegex::Phi
      }
    }
    ARegex::Alt(bs, r1, r2) => {
      let d1 = deriv_bc(*r1, l);
      let d2 = deriv_bc(*r2, l);
      ARegex::Alt(bs, Box::new(d1), Box::new(d2))
    }
    ARegex::Seq(bs, r1, r2) => {
      if nullable_bc(&r1) {
        let eps_bits = mk_eps_bc(&r1);
        let d1 = deriv_bc(*r1, l);
        let d2 = deriv_bc(*r2.clone(), l);
        let left_branch = ARegex::Seq(vec![], Box::new(d1), r2);
        let right_branch = fuse(&eps_bits, d2);
        ARegex::Alt(bs, Box::new(left_branch), Box::new(right_branch))
      } else {
        let d1 = deriv_bc(*r1, l);
        ARegex::Seq(bs, Box::new(d1), r2)
      }
    }
    // Empty star prefix: bs = []; Inner exp: r = Box<Lit([], 'a')>
    ARegex::Star(bs, r) => {
      let d = deriv_bc(*r.clone(), l); // ri\'a\' = Lit([], 'a')\'a\' = Eps([])
      let fused = fuse(&[false], d); // fuse [0] Eps([]) = Eps([false])
      let new_star = ARegex::Star(vec![], r); // []@ri* = []@([]@\'a\')*
      ARegex::Seq(bs, Box::new(fused), Box::new(new_star))
      // = []@Eps([false]) . []@([]@\'a\')*
    }
  }
}

```

simp
src/parsers/deriv_bc/simplify.rs

```

/// Simplify an annotated expression to a fixpoint.
pub fn simp(ri: ARegex) -> ARegex {
  simp_once(ri)
}

```

A.6 The pderiv_bc Parser

pderiv_bc
src/parsers/pderiv_bc/pderiv.rs

```

/// Bit-coded partial derivative (pDerivBC): one '(residual, bits)' pair per surviving strand.
pub fn pderiv_bc(r: &Regex, x: char) -> Vec<(Regex, Vec<bool>)> {
  match r {

```

```

Regex::Eps => Vec::new(),
Regex::Phi => Vec::new(),

Regex::Lit(y) => {
  if *y == x {
    vec![(Regex::Eps, Vec::new())]
  } else {
    Vec::new()
  }
}

Regex::Alt(r1, r2) => {
  let mut out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
    .into_iter()
    .map(|(r1_, mut bs)| {
      let mut b = vec![false];
      b.append(&mut bs);
      (r1_, b)
    })
    .collect();
  out.extend(
    pderiv_bc(r2, x)
      .into_iter()
      .map(|(r2_, mut bs)| {
        let mut b = vec![true];
        b.append(&mut bs);
        (r2_, b)
      })
  );
  nub2(out)
}

Regex::Seq(r1, r2) => {
  if nullable(r1) {
    // Strand 1: continue r1, r2 untouched (not smart-constructed,
    // matching the reference literally)
    let mut out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
      .into_iter()
      .map(|(r1_, bs)| (Regex::seq(r1_, *r2.clone()), bs))
      .collect();

    // Strand 2: r1 matched empty, jump straight into r2.
    let eps_bits = mk_eps_bits(r1);
    out.extend(pderiv_bc(r2, x).into_iter().map(|(r2_, bs)| {
      let mut b = eps_bits.clone();
      b.extend(bs);
      (r2_, b)
    }));

    nub2(out)
  } else {
    pderiv_bc(r1, x)
      .into_iter()
      .map(|(r1_, bs)| (smart_seq(r1_, r2), bs))
      .collect()
  }
}

Regex::Star(r1) => {
  let out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
    .into_iter()
    .map(|(r1_, mut bs)| {
      let mut b = vec![false];
      b.append(&mut bs);
      (smart_seq(r1_, &Regex::star(*r1.clone())), b)
    })
    .collect();
  nub2(out)
}
}

```

nub2

src/parsers/pderiv_bc/deriv.rs

```
/// Stable dedup on the residual, keeping the first (highest-priority)
fn nub2(pairs: Vec<Regex, Vec<bool>>>) -> Vec<Regex, Vec<bool>> {
    let mut seen: HashSet<Regex> = HashSet::with_capacity(pairs.len());
    let mut out = Vec::with_capacity(pairs.len());
    for (r, bs) in pairs {
        if seen.insert(r.clone()) {
            out.push((r, bs));
        }
    }
    out
}
```

step_frontier.
src/parsers/pderiv_bc/parse.rs

```
/// One step of 'parsePDerivBC2': replace the frontier with union of 'pderiv_bc(x, .)'
pub(super) fn step_frontier(frontier: &[(Regex, Vec<bool>)], x: char) -> Vec<Regex, Vec<bool>> {
    let mut out = Vec::new();
    for (r, bits) in frontier {
        for (r_next, extra) in pderiv_bc(r, x) {
            let mut combined = bits.clone();
            combined.extend(extra);
            out.push((r_next, combined));
        }
    }
    out
}
```

select_bits
src/parsers/pderiv_bc/parse.rs

```
/// 'filter ((r,_) -> nullable r) rs', take head, and append 'mkEpsBC'.
pub(super) fn select_bits(frontier: &[(Regex, Vec<bool>)]) -> Option<Vec<bool>> {
    for (r, bits) in frontier {
        if nullable(r) {
            let mut result = bits.clone();
            result.extend(mk_eps_bits(r));
            return Some(result);
        }
    }
    None
}
```

parse_pderiv_bc
src/parsers/pderiv_bc/parse.rs

```
pub fn parse_pderiv_bc(input: &str, r: &Regex) -> Option<ParseTree> {
    let mut frontier = vec![(r.clone(), Vec::new())];
    for c in input.chars() {
        frontier = step_frontier(&frontier, c);
        if frontier.is_empty() {
            return None;
        }
    }
    let bits = select_bits(&frontier)?;
    Some(decode(r, &bits))
}
```

A.7 The pderiv_std Parser

Inj
src/parsers/pderiv_std/inject.rs

```
/// Reconstructs a strand's 'ParseTree'; 'Rc' since a fan-out step (e.g. 'Alt') shares one parent injection
pub(crate) type Inj = Rc<dyn Fn(ParseTree) -> ParseTree>;
```

```
pub(crate) fn identity_inj() -> Inj {
    Rc::new(|v| v)
}
```

nub_tree
src/parsers/pderiv_std/inject.rs

```
/// Dedup on the residual alone, keeping the first (highest-priority, GREEDY-ordered)
pub(crate) fn nub_tree(strands: Vec<(Regex, Inj)> -> Vec<(Regex, Inj)> {
    let mut seen: HashSet<Regex> = HashSet::with_capacity(strands.len());
    let mut out = Vec::with_capacity(strands.len());
    for (r, inj) in strands {
        if seen.insert(r.clone()) {
            out.push((r, inj));
        }
    }
    out
}
```

pderiv_tree
src/parsers/pderiv_std/pderiv_tree.rs

```
pub(crate) fn pderiv_tree(r: &Regex, x: char) -> Vec<(Regex, Inj)> {
    match r {
        Regex::Eps | Regex::Phi => vec![],

        Regex::Lit(c) => {
            if *c == x {
                vec![(Regex::Eps, Rc::new(move |_| ParseTree::Char(x)) as Inj)]
            } else {
                vec![]
            }
        }

        Regex::Alt(r1, r2) => {
            let mut out: Vec<(Regex, Inj)> = pderiv_tree(r1, x)
                .into_iter()
                .map(|(r1_, inj1)| {
                    let f: Inj = Rc::new(move |v| ParseTree::Left(Box::new(inj1(v))));
                    (r1_, f)
                })
                .collect();
            out.extend(pderiv_tree(r2, x).into_iter().map(|(r2_, inj2)| {
                let f: Inj = Rc::new(move |v| ParseTree::Right(Box::new(inj2(v))));
                (r2_, f)
            }));
            nub_tree(out)
        }

        Regex::Seq(r1, r2) => {
            if nullable(r1) {
                let mut out: Vec<(Regex, Inj)> = pderiv_tree(r1, x)
                    .into_iter()
                    .map(|(r1_, inj1)| {
                        let r2c = (**r2).clone();
                        let f: Inj = Rc::new(move |v: ParseTree| match v {
                            ParseTree::Pair(v1, v2) => ParseTree::Pair(Box::new(inj1(*v1)), v2),
                            other => panic!(
                                "pderiv_tree: Seq-continue strand expected Pair, got {:?}",
                                other
                            ),
                        },
                    );
                        (Regex::seq(r1_, r2c), f)
                    })
                    .collect();

                let v1_eps = mk_eps(r1);
                out.extend(pderiv_tree(r2, x).into_iter().map(|(r2_, inj2)| {
                    let v1c = v1_eps.clone();
                    let f: Inj = Rc::new(move |v2: ParseTree| {
                        ParseTree::Pair(Box::new(v1c.clone()), Box::new(inj2(v2)))
                    });
                    (r2_, f)
                }));
                nub_tree(out)
            } else {
                pderiv_tree(r1, x)
            }
        }
    }
}
```

```

        .into_iter()
        .map(|(r1_, inj1)| {
            let r2c = (**r2).clone();
            let f: Inj = Rc::new(move |v: ParseTree| match v {
                ParseTree::Pair(v1, v2) => ParseTree::Pair(Box::new(inj1(*v1)), v2),
                other => panic!(
                    "pderiv_tree: Seq(non-nullable) strand expected Pair, got {:?}",
                    other
                ),
            });
            (Regex::seq(r1_, r2c), f)
        })
        .collect()
    }
}

Regex::Star(r1) => {
    let out: Vec<(Regex, Inj)> = pderiv_tree(r1, x)
        .into_iter()
        .map(|(r1_, inj1)| {
            let r1c = (**r1).clone();
            let f: Inj = Rc::new(move |v: ParseTree| match v {
                ParseTree::Pair(v1, vs) => match *vs {
                    ParseTree::Star(mut items) => {
                        items.insert(0, inj1(*v1));
                        ParseTree::Star(items)
                    }
                    other => panic!(
                        "pderiv_tree: Star strand expected Star tail, got {:?}",
                        other
                    ),
                },
                other => panic!(
                    "pderiv_tree: Star strand expected Pair, got {:?}",
                    other
                ),
            });
            (Regex::seq(r1_, Regex::star(r1c)), f)
        })
        .collect();
    nub_tree(out)
}
}
}
}

```

parse_pderiv_std
src/parsers/pderiv_std/parse.rs

```

pub fn parse_pderiv_std(input: &str, r: &Regex) -> Option<ParseTree> {
    let mut frontier: Vec<(Regex, Inj)> = vec![(r.clone(), identity_inj())];
    for c in input.chars() {
        frontier = step_frontier(frontier, c);
        if frontier.is_empty() {
            return None;
        }
    }
    select(&frontier)
}

```

A.8 Frontend

ExtPat
src/frontend/ext_pattern.rs

```

#[derive(Debug, Clone, PartialEq)]
pub enum ExtPat {
    /// the empty string, 'translate's to 'Regex::Eps'.
    Empty,
    Eps,
    /// Literal 'phi' token 'translate's to 'Regex::Phi'.
    Never,
}

```

```

    /// A marking group '( ... )'.
    Group(Box<ExtPat>),
    /// A non-marking group '(?: ... )'.
    GroupNonMarking(Box<ExtPat>),
    /// Alternation 'a|b|c'.
    Or(Vec<ExtPat>),
    /// Concatenation 'ab c'.
    Concat(Vec<ExtPat>),
    /// 'r?' / 'r??' (bool: greedy).
    Opt(Box<ExtPat>, bool),
    /// 'r+' / 'r+?' (bool: greedy).
    Plus(Box<ExtPat>, bool),
    /// 'r*' / 'r*?' (bool: greedy).
    Star(Box<ExtPat>, bool),
    /// 'r{lo,hi}' / 'r{lo,}' / 'r{lo}' (bool: greedy).
    Bound(Box<ExtPat>, u32, Option<u32>, bool),
    /// '^'.
    Carat,
    /// '$'.
    Dollar,
    /// '.'.
    Dot,
    /// '[ ... ]' listed characters (ranges and '\d'/'\w'/'\s'
    /// shorthands already expanded by the parser).
    Any(Vec<char>),
    /// '[^ ... ]'.
    NoneOf(Vec<char>),
    /// '\c' for a 'c' that carries regex meaning to 'translate'
    /// ('d'/'D'/'w'/'W'/'s'/'S') or, for any other 'c', an escaped literal
    /// ('\.', '\(', '\\', ...).
    Escape(char),
    /// An ordinary, unescaped literal character.
    Char(char),
}

```

translate

src/frontend/translate.rs

```

/// Lower an 'ExtPat' into this crate's core 'Regex'.
pub fn translate(ep: &ExtPat) -> Result<Regex, String> {
    match ep {
        ExtPat::Empty => Ok(Regex::Eps),
        ExtPat::Eps => Ok(Regex::Eps),
        ExtPat::Never => Ok(Regex::Phi),
        ExtPat::Group(inner) | ExtPat::GroupNonMarking(inner) => translate(inner),
        ExtPat::Or(alts) => {
            let mut iter = alts.iter();
            let mut acc = translate(iter.next().ok_or("empty alternation")??);
            for a in iter {
                acc = Regex::alt(acc, translate(a)?);
            }
            Ok(acc)
        }
        ExtPat::Concat(parts) => {
            let mut iter = parts.iter();
            let mut acc = match iter.next() {
                Some(p) => translate(p)?,
                None => return Ok(Regex::Eps),
            };
            for p in iter {
                acc = Regex::seq(acc, translate(p)?);
            }
            Ok(acc)
        }
        ExtPat::Opt(inner, _greedy) => Ok(Regex::alt(translate(inner)?, Regex::Eps)),
        ExtPat::Plus(inner, _greedy) => {
            let r = translate(inner)?;
            Ok(Regex::seq(r.clone(), Regex::star(r)))
        }
        ExtPat::Star(inner, _greedy) => Ok(Regex::star(translate(inner)?)),
        ExtPat::Bound(inner, lo, hi, _greedy) => desugar_bound(translate(inner)?, *lo, *hi),
        ExtPat::Carat | ExtPat::Dollar => Ok(Regex::Eps),
        ExtPat::Dot => alt_of_chars(alphabet::alphabet()),
    }
}

```

```
ExtPat::Any(cs) => alt_of_chars(cs.clone()),
ExtPat::NoneOf(cs) => alt_of_chars(alphabet::complement(cs)),
ExtPat::Escape(c) => translate_escape(*c),
ExtPat::Char(c) => Ok(Regex::lit(*c)),
    }
}
```

B Tests

B.1 Shared Fixtures

paper_r1
tests/common/mod.rs

```
pub fn paper_r1() -> Regex {
    Regex::seq(
        Regex::alt(Regex::lit('a'), Regex::seq(Regex::lit('a'), Regex::lit('b'))),
        Regex::alt(Regex::lit('b'), Regex::Eps),
    )
}
```

paper_r2
tests/common/mod.rs

```
/// (a + (b + ab))* - POSIX star-preference example from flops14 p.1
/// Structure: Star(Alt(a, Alt(b, ab))) - nested Alt, ab is Right(Right(...))
pub fn paper_r2() -> Regex {
    Regex::star(Regex::alt(
        Regex::lit('a'),
        Regex::alt(
            Regex::lit('b'),
            Regex::seq(Regex::lit('a'), Regex::lit('b')),
        ),
    ))
}
```

assert_round_trip
tests/common/mod.rs

```
#[allow(dead_code)]
pub fn assert_round_trip(tree: &ParseTree, expected_word: &str) {
    assert_eq!(
        flatten(tree), expected_word,
        "flatten(tree) should equal the matched word"
    );
}
```

assert_parsers_agree
tests/common/mod.rs

```
#[allow(dead_code)]
pub fn assert_parsers_agree(
    label_a: &str,
    tree_a: &Option<ParseTree>,
    label_b: &str,
    tree_b: &Option<ParseTree>,
) {
    assert_eq!(
        tree_a, tree_b,
        "{} and {} disagree:\n {}: {:?}\n {}: {:?}",
        label_a, label_b, label_a, tree_a, label_b, tree_b
    );
}
```

B.2 Paper Examples Reproduced

`parse_recursive_paper_r1_on_ab`
tests/test_deriv_std.rs

```
/// (a + ab)(b + eps) on "ab": A1 picks the longer "ab" branch on the left factor.
#[test]
fn parse_recursive_paper_r1_on_ab() {
    let r = paper_r1();
    let tree = parse_deriv_std_rec("ab", &r).expect("should match");
    assert_round_trip(&tree, "ab");

    assert!(
        matches!(&tree, ParseTree::Pair(_, _)),
        "outer should be Pair (regex is Seq), got {:?}", tree
    );

    if let ParseTree::Pair(left, right) = &tree {
        assert!(
            matches!(left.as_ref(), ParseTree::Right(_)),
            "left factor should be Right (ab branch chosen by A1), got {:?}", left
        );
        assert!(
            matches!(right.as_ref(), ParseTree::Right(_)),
            "right factor should be Right (eps branch, only valid choice), got {:?}", right
        );
    }
}
```

`parse_recursive_paper_r2_on_ab`
tests/test_deriv_std.rs

```
/// (a + b + ab)* on "ab": A1 picks one "ab" iteration over Greedy's two-iteration answer.
#[test]
fn parse_recursive_paper_r2_on_ab() {
    let r = paper_r2();
    let tree = parse_deriv_std_rec("ab", &r).expect("should match");
    assert_round_trip(&tree, "ab");

    if let ParseTree::Star(ref iters) = tree {
        assert_eq!(iters.len(), 1,
            "POSIX should pick 'ab' as one star iteration (A1), got {:?}", iters);

        assert!(
            matches!(&iters[0], ParseTree::Right(inner)
                if matches!(inner.as_ref(), ParseTree::Right(innermost)
                    if matches!(innermost.as_ref(), ParseTree::Pair(_, _)))),
            "iteration should be Right(Right(Pair(a,b))), got {:?}", iters[0]
        );
    } else {
        panic!("expected Star, got {:?}", tree);
    }
}
```

`paper_r1_computes_greedy_not_posix`
src/parsers/pderiv_bc/parse.rs

```
#[test]
fn paper_r1_computes_greedy_not_posix() {
    // (a + ab)(b + eps) on "ab"
    let r = Regex::seq(
        Regex::alt(Regex::lit('a'), Regex::seq(Regex::lit('a'), Regex::lit('b'))),
        Regex::alt(Regex::lit('b'), Regex::Eps),
    );

    let posix = parse_deriv_std_rec("ab", &r);
    let greedy = parse_pderiv_bc("ab", &r);

    assert!(posix.is_some() && greedy.is_some(), "both should match");
    assert_eq!(flatten(posix.as_ref().unwrap()), "ab");
    assert_eq!(flatten(greedy.as_ref().unwrap()), "ab");
}
```

```

// POSIX: the longer "ab" alternative wins (rule A1)
assert_eq!(
  posix,
  Some(ParseTree::Pair(
    Box::new(ParseTree::Right(Box::new(ParseTree::Pair(
      Box::new(ParseTree::Char('a')),
      Box::new(ParseTree::Char('b')),
    )))),
    Box::new(ParseTree::Right(Box::new(ParseTree::Empty))),
  ))
);
// Greedy: the leftmost alternative 'a' wins, committed to immediately
assert_eq!(
  greedy,
  Some(ParseTree::Pair(
    Box::new(ParseTree::Left(Box::new(ParseTree::Char('a')))),
    Box::new(ParseTree::Left(Box::new(ParseTree::Char('b')))),
  ))
);
assert_ne!(posix, greedy, "this is exactly the documented divergence");
}

```

paper_r2_computes_greedy_not_posix *src/parsers/pderiv_bc/parse.rs*

```

#[test]
fn paper_r2_computes_greedy_not_posix() {
  // (a + (b + ab))* on "ab"
  let r = Regex::star(Regex::alt(
    Regex::lit('a'),
    Regex::alt(Regex::lit('b'), Regex::seq(Regex::lit('a'), Regex::lit('b'))),
  ));

  let posix = parse_deriv_std_rec("ab", &r);
  let greedy = parse_pderiv_bc("ab", &r);

  assert!(posix.is_some() && greedy.is_some(), "both should match");
  assert_eq!(flatten(posix.as_ref().unwrap()), "ab");
  assert_eq!(flatten(greedy.as_ref().unwrap()), "ab");

  // POSIX: one iteration, the longer "ab" alternative wins (rule A1)
  assert_eq!(
    posix,
    Some(ParseTree::Star(vec! [ParseTree::Right(Box::new(ParseTree::Right(
      Box::new(ParseTree::Pair(
        Box::new(ParseTree::Char('a')),
        Box::new(ParseTree::Char('b')),
      ))
    ))]))
  );
  // Greedy: two iterations, leftmost alternative wins at each step
  assert_eq!(
    greedy,
    Some(ParseTree::Star(vec! [
      ParseTree::Left(Box::new(ParseTree::Char('a'))),
      ParseTree::Right(Box::new(ParseTree::Left(Box::new(ParseTree::Char('b'))))),
    ]))
  );
  assert_ne!(posix, greedy, "this is exactly the documented divergence");
}

```

B.3 Cross-Implementation Equivalence

recursive_traced_and_loop_traced_agree_on_all_paper_examples *tests/test_deriv_std.rs*

```

#[test]
fn recursive_traced_and_loop_traced_agree_on_all_paper_examples() {
  use regex_engine::parsers::{parse_recursive_traced, parse_loop_traced};

```

```

let cases: Vec<(&str, Regex)> = vec![
    ("aaa", Regex::star(Regex::lit('a'))),
    ("aab", Regex::star(Regex::lit('a'))),
    ("ab", paper_r1()),
    ("a", paper_r1()),
    ("ab", paper_r2()),
    ("", Regex::star(Regex::lit('a'))),
];

for (input, r) in &cases {
    let (rec_tree, _) = parse_recursive_traced(input, r);
    let (loop_tree, _) = parse_loop_traced(input, r);
    assert_eq!(
        rec_tree, loop_tree,
        "recursive_traced and loop_traced disagree on {:?}" , input
    );
}

}

bitcoded_agrees_with_recursive
tests/test_deriv_bc.rs

fn bitcoded_agrees_with_recursive(input: &str, r: &Regex) {
    let rec = parse_deriv_std_rec(input, r);
    let bc = parse_deriv_bc(input, r);
    assert_parsers_agree("recursive", &rec, "bitcoded", &bc);
}

```

B.4 Frontend

unanchored_nullable_pattern_matches_anything
src/frontend/mod.rs

```

#[test]
fn unanchored_nullable_pattern_matches_anything() {
    let r = parse_dataset_pattern("a*").expect("should parse");
    assert!(parse_deriv_std_rec("", &r).is_some());
    assert!(parse_deriv_std_rec("xyz", &r).is_some());
    assert!(parse_deriv_std_rec("zzz999", &r).is_some());
}

```

C Diagnostics

```
cargo run -- parse "(a|ab)(b|)" <INPUT> --parser <PARSER> --diag 3
```

Field	Value
Expression	"(a ab)(b)", that is $(a + ab)(b + \varepsilon)$
Inputs	"ab" (match), "b" (no match)
Parsers	deriv_std_rec, deriv_bc, pderiv_std, pderiv_bc
Output file	reports/report.txt
Omitted	the Timestamp line
Wrapped	long lines
∂	d
.	.
$\oplus$	(+)
ε	epsilon
ϕ	phi
✓	[OK]
Policy	the implementation's internal label, not a claim about disambiguation policy

C.1 deriv_std_rec, match

```
$ cargo run -- parse "(a|ab)(b|)" "ab" \
  --parser deriv_std_rec --diag 3
```

```
=====
REGEX ENGINE DEBUG REPORT
=====
Mode:                POSIX Standard (recursive parser)
Regex:               (a|ab)(b|)
Input:               "ab"
Result:              MATCH
Policy:              POSIX
=====
```

```
TIMING
=====
Parse time:          0.012ms
Expressions stored:  3 (r0 -> r1 -> r2)
=====
```

```
FORWARD PASS (Derivatives)
=====
```

```
r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))
```

```
Step 1: d(r0, 'a')
```

```
rule: d(r1.r2, 'a') [nullable(r1)=false]
```

```
= d(r1, 'a').r2
```

```
r1 = Seq(Alt(Eps, Seq(Eps, Lit('b'))), Alt(Lit('b'), Eps))
```

```
nullable(r1) = true [OK] position 1 matched
```

```
Step 2: d(r1, 'b')
```

```
rule: d(r1.r2, 'b') [nullable(r1)=true]
```

```
= d(r1, 'b').r2 + d(r2, 'b')
```

```
r2 = Alt(Seq(Alt(Phi, Alt(Seq(Phi, Lit('b'))), Eps)), Alt(Lit('b'), Eps)), Alt(Eps, Phi))
```

```
nullable(r2) = true [OK] position 2 matched
```

```
-----=====
NULLABILITY CHECK
```

```
-----=====
nullable(r2) = true -> proceed to backward pass
```

```
-----=====
BACKWARD PASS (mkEps + inject)
```

```
-----=====
mkEps(r2):
```

```
mkEps(r1 + r2): nullable(r1)=true -> Left(mkEps(r1))
```

```
v2 = Left (Right Right (), Right ())
```

```
inject(r1, 'b', v2):
```

```
inject(Seq(Alt(Eps, Seq(Eps, Lit('b'))), Alt(Lit('b'), Eps)), 'b',
Left (Right Right (), Right ()))
```

```
-> (Right ((), b), Right ())
```

```
v1 = (Right ((), b), Right ())
```

```
inject(r0, 'a', v1):
```

```
inject(Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps)), 'a', (Right ((), b), Right ()))
```

```
-> (Right (a, b), Right ())
```

```
v0 = (Right (a, b), Right ())
```

```
-----=====
RESULT
```

```
-----=====
Parse tree: (Right (a, b), Right ())
```

```
Flattened: "ab"
```

```
-----=====
END OF REPORT
```

C.2 deriv_bc, match

```
$ cargo run -- parse "(a|ab)(b|)" "ab" \
  --parser deriv_bc --diag 3
-----=====
REGEX ENGINE DEBUG REPORT
-----=====
Mode:                POSIX Bitcoded
Regex:               (a|ab)(b|)
Input:               "ab"
Result:              MATCH
Policy:              POSIX
-----=====
TIMING
-----=====
Parse time:          0.015ms
Derivative steps:    2
-----=====
INTERNALIZE
-----=====
internalize((a|ab)(b|))
  rule: internalize(r1 . r2) = []@(internalize r1)(internalize r2)
ri0 = []@([]@([]@[false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([]@[false]@'b' (+) [true]@epsilon))
-----=====
FORWARD PASS (deriv_bc + simp)
-----=====
Step 1 (char: 'a', position 1)
ri0 = []@([]@([]@[false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([]@[false]@'b' (+) [true]@epsilon))
deriv_bc(ri0, 'a'):
  rule: (bs@ri1.ri2) \ 'a' [nullable_bc(ri1)=false]
    = bs@(ri1\'a'.ri2)
simp -> []@([]@([]@[false]@epsilon (+) [true]@'b') . []@([]@[false]@'b' (+)
  [true]@epsilon))
ri1 = []@([]@([]@[false]@epsilon (+) [true]@'b') . []@([]@[false]@'b' (+)
  [true]@epsilon))
  nullable_bc(ri1) = true [OK] bits accumulating

Step 2 (char: 'b', position 2)
ri1 = []@([]@([]@[false]@epsilon (+) [true]@'b') . []@([]@[false]@'b' (+)
  [true]@epsilon))
deriv_bc(ri1, 'b'):
  rule: (bs@ri1.ri2) \ 'b' [nullable_bc(ri1)=true]
    = bs@((ri1\'b'.ri2) (+) (fuse (mkEpsBC ri1) (ri2\'b')))
simp -> []@([true]@([]@[false]@'b' (+) [true]@epsilon) (+) [false,
  false]@epsilon)
ri2 = []@([true]@([]@[false]@'b' (+) [true]@epsilon) (+) [false,
```

```

    false]@epsilon)
nullable_bc(ri2) = true   [OK]   bits accumulating

-----
NULLABILITY CHECK
-----
nullable_bc(ri2) = true -> proceed to mkEpsBC + decode
-----
MKEEPSBC + DECODE
-----
mkEpsBC(ri2) = [11]

decode((a|ab)(b|), [11]):
  = (Right (a, b), Right ())
-----
RESULT
-----
Bits:                [11]
Parse tree:          (Right (a, b), Right ())
Flattened:           "ab"
-----
END OF REPORT
-----

```

C.3 pderiv_std, match

```

$ cargo run -- parse "(a|ab)(b|)" "ab" \
  --parser pderiv_std --diag 3
-----
REGEX ENGINE DEBUG REPORT
-----
Mode:                Standard Partial-Derivative (pderiv_std)
Regex:               (a|ab)(b|)
Input:               "ab"
Result:              MATCH
Policy:              GREEDY
-----
TIMING
-----
Parse time:          0.017ms
Steps:               2
-----
INITIAL FRONTIER
-----
[r0]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
  Alt(Lit('b'), Eps))
-----
FORWARD PASS (pderiv_tree per residual, frontier flattened)

```

```

-----=====
Step 1 (char: 'a', position 1)
  frontier before (1 residual(s)):
[0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))
  pderiv_tree('a', .) applied to each:
    from [0]:
      rule: pderiv_tree('a', r1.r2) [nullable(r1)=false]
= [(r1'.r2, inj: continue r1) | (r1',_) <- pderiv_tree(c,r1)]
-> Seq(Eps, Alt(Lit('b'), Eps)) (+ injection closure, not printable)
-> Seq(Seq(Eps, Lit('b')), Alt(Lit('b'), Eps)) (+ injection closure,
      not printable)
  frontier after (2 residual(s)):
    [0] Seq(Eps, Alt(Lit('b'), Eps))
    [1] Seq(Seq(Eps, Lit('b')), Alt(Lit('b'), Eps))
  a residual in this frontier is nullable [OK] parse tree available

Step 2 (char: 'b', position 2)
  frontier before (2 residual(s)):
    [0] Seq(Eps, Alt(Lit('b'), Eps))
    [1] Seq(Seq(Eps, Lit('b')), Alt(Lit('b'), Eps))
  pderiv_tree('b', .) applied to each:
    from [0]:
      rule: pderiv_tree('b', r1.r2) [nullable(r1)=true]
= nub_tree $ [(r1'.r2, inj: continue r1) | (r1',_) <-
      pderiv_tree(c,r1)]
++ [(r2', inj: mk_eps(r1) paired with r2's result) | (r2',_) <-
      pderiv_tree(c,r2)]
    -> Eps (+ injection closure, not printable)
    from [1]:
      rule: pderiv_tree('b', r1.r2) [nullable(r1)=false]
= [(r1'.r2, inj: continue r1) | (r1',_) <- pderiv_tree(c,r1)]
-> Seq(Eps, Alt(Lit('b'), Eps)) (+ injection closure, not printable)
  frontier after (2 residual(s)):
    [0] Eps
    [1] Seq(Eps, Alt(Lit('b'), Eps))
  a residual in this frontier is nullable [OK] parse tree available

-----=====
SELECTION (first nullable residual, priority/list order)
-----=====
any nullable residual in final frontier = true -> proceed to injection
  (already applied by select)
-----=====
RESULT
-----=====
Parse tree:          (Left a, Left b)
Flattened:           "ab"
-----=====

```

END OF REPORT

C.4 pderiv_bc, match

```
$ cargo run -- parse "(a|ab)(b|)" "ab" \
  --parser pderiv_bc --diag 3
```

REGEX ENGINE DEBUG REPORT

```
Mode:                Bitcoded Partial-Derivative (pDerivBC)
Regex:               (a|ab)(b|)
Input:               "ab"
Result:              MATCH
Policy:              GREEDY
```

TIMING

```
Parse time:          0.012ms
Steps:               2
```

INITIAL FRONTIER

```
[(r0, [])]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
  Alt(Lit('b'), Eps))
```

FORWARD PASS (pDerivBC per residual, frontier flattened)

Step 1 (char: 'a', position 1)

frontier before (1 residual(s)):

```
[0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))
  bits-so-far = []
```

pDerivBC('a', .) applied to each:

from [0]:

```
rule: pDerivBC('a', r1.r2) [nullable(r1)=false]
      = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
-> (Alt(Lit('b'), Eps), [0])
-> (Seq(Lit('b'), Alt(Lit('b'), Eps)), [1])
```

frontier after (2 residual(s)):

```
[0] Alt(Lit('b'), Eps)  bits = [0]
[1] Seq(Lit('b'), Alt(Lit('b'), Eps))  bits = [1]
```

a residual in this frontier is nullable [OK] bits accumulating

Step 2 (char: 'b', position 2)

frontier before (2 residual(s)):

```
[0] Alt(Lit('b'), Eps)  bits-so-far = [0]
[1] Seq(Lit('b'), Alt(Lit('b'), Eps))  bits-so-far = [1]
```

pDerivBC('b', .) applied to each:

```

    from [0]:
rule: pDerivBC('b', r1+r2) = tag(0, pDerivBC('b',r1)) ++ tag(1,
    pDerivBC('b',r2)), nub2
    -> (Eps, [0])
    from [1]:
    rule: pDerivBC('b', r1.r2) [nullable(r1)=false]
        = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
    -> (Alt(Lit('b'), Eps), [])
frontier after (2 residual(s)):
[0] Eps    bits = [00]
[1] Alt(Lit('b'), Eps)    bits = [1]
a residual in this frontier is nullable [OK] bits accumulating

=====
SELECTION (first nullable residual, priority/list order)
=====
any nullable residual in final frontier = true -> proceed to mkEpsBC +
    decode
=====
MKEEPSBC + DECODE
=====
selected residual's accumulated bits ++ mkEpsBC(residual) = [00]

decode((a|ab)(b|), [00]):
    = (Left a, Left b)
=====
RESULT
=====
Bits:                [00]
Parse tree:          (Left a, Left b)
Flattened:           "ab"
=====
END OF REPORT
=====

```

C.5 deriv_std_rec, no match

```

$ cargo run -- parse "(a|ab)(b|)" "b" \
    --parser deriv_std_rec --diag 3
=====
REGEX ENGINE DEBUG REPORT
=====
Mode:                POSIX Standard (recursive parser)
Regex:               (a|ab)(b|)
Input:               "b"
Result:              NO MATCH
Policy:              POSIX
=====

```

```

TIMING
=====
Parse time:          0.005ms
Expressions stored:  2 (r0 -> r1)
=====
FORWARD PASS (Derivatives)
=====
r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))

Step 1: d(r0, 'b')
  rule: d(r1.r2, 'b') [nullable(r1)=false]
        = d(r1, 'b').r2
r1 = Seq(Alt(Phi, Seq(Phi, Lit('b'))), Alt(Lit('b'), Eps))
  nullable(r1) = false [FAIL] position 1

=====
NULLABILITY CHECK
=====
nullable(r1) = false -> no parse tree exists for full input
=====
PARTIAL RECOVERY
=====
No prefix matched.
=====
ERROR SUMMARY
=====
Failure at position:  1 (1-indexed)
Found:                'b'
Expected:              'a'
  b
  ^
=====
END OF REPORT
=====

```

C.6 deriv_bc, no match

```

$ cargo run -- parse "(a|ab)(b|)" "b" \
  --parser deriv_bc --diag 3

```

```

=====
REGEX ENGINE DEBUG REPORT
=====
Mode:                POSIX Bitcoded
Regex:               (a|ab)(b|)
Input:               "b"
Result:              NO MATCH
Policy:              POSIX
=====

```

```

TIMING
=====
Parse time:          0.008ms
Derivative steps:    1
=====
INTERNALIZE
=====
internalize((a|ab)(b|))
  rule: internalize(r1 . r2) = []@(internalize r1)(internalize r2)
ri0 = []@([]@([]@([false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([false]@'b' (+) [true]@epsilon))
=====
FORWARD PASS (deriv_bc + simp)
=====
Step 1 (char: 'b', position 1)
ri0 = []@([]@([]@([false]@'a' (+) [true]@([]@'a' . []@'b')) .
  []@([false]@'b' (+) [true]@epsilon))
deriv_bc(ri0, 'b'):
  rule: (bs@ri1.ri2) \ 'b' [nullable_bc(ri1)=false]
    = bs@(ri1\'b'.ri2)
simp -> phi
ri1 = phi
nullable_bc(ri1) = false [FAIL]

=====
NULLABILITY CHECK
=====
nullable_bc(ri1) = false -> no full parse tree exists
=====
PARTIAL RECOVERY
=====
No prefix matched.
=====
ERROR SUMMARY
=====
Failure at position:  1 (1-indexed)
Found:                'b'
Expected:              'a'
  b
  ^
=====
END OF REPORT
=====

```

C.7 pderiv_std, no match

```

$ cargo run -- parse "(a|ab)(b|)" "b" \
  --parser pderiv_std --diag 3

```

```

-----=====
REGEX ENGINE DEBUG REPORT
-----=====
Mode:                Standard Partial-Derivative (pderiv_std)
Regex:               (a|ab)(b|)
Input:               "b"
Result:              NO MATCH
Policy:              GREEDY
-----=====

TIMING
-----=====
Parse time:          0.007ms
Steps:               1
-----=====

INITIAL FRONTIER
-----=====
[r0]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
    Alt(Lit('b'), Eps))
-----=====

FORWARD PASS (pderiv_tree per residual, frontier flattened)
-----=====
Step 1 (char: 'b', position 1)
  frontier before (1 residual(s)):
[0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))
  pderiv_tree('b', .) applied to each:
    from [0]:
      rule: pderiv_tree('b', r1.r2) [nullable(r1)=false]
= [(r1'.r2, inj: continue r1) | (r1',_) <- pderiv_tree(c,r1)]
  -> []
  frontier after (0 residual(s)):
  no nullable residual in this frontier [FAIL]

-----=====

SELECTION (first nullable residual, priority/list order)
-----=====
any nullable residual in final frontier = false -> no full parse tree
exists
-----=====

PARTIAL RECOVERY
-----=====
No prefix matched.
-----=====

ERROR SUMMARY
-----=====
Failure at position: 1 (1-indexed)
Found:               'b'
Expected:             'a'
  b

```

```

~
-----=====
END OF REPORT
-----=====

```

C.8 pderiv_bc, no match

```

$ cargo run -- parse "(a|ab)(b|)" "b" \
  --parser pderiv_bc --diag 3

```

```

-----=====
REGEX ENGINE DEBUG REPORT

```

```

-----=====
Mode:                Bitcoded Partial-Derivative (pDerivBC)
Regex:               (a|ab)(b|)
Input:               "b"
Result:              NO MATCH
Policy:              GREEDY

```

```

-----=====
TIMING

```

```

-----=====
Parse time:          0.006ms
Steps:               1

```

```

-----=====
INITIAL FRONTIER

```

```

-----=====
[(r0, [])]  where r0 = Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))),
  Alt(Lit('b'), Eps))

```

```

-----=====
FORWARD PASS (pDerivBC per residual, frontier flattened)

```

```

-----=====
Step 1 (char: 'b', position 1)
  frontier before (1 residual(s)):
[0] Seq(Alt(Lit('a'), Seq(Lit('a'), Lit('b'))), Alt(Lit('b'), Eps))
    bits-so-far = []
  pDerivBC('b', .) applied to each:
    from [0]:
      rule: pDerivBC('b', r1.r2) [nullable(r1)=false]
           = [(smartC(r1',r2), bs) | (r1',bs) <- pDerivBC(c,r1)]
      -> []
  frontier after (0 residual(s)):
  no nullable residual in this frontier [FAIL]

```

```

-----=====
SELECTION (first nullable residual, priority/list order)

```

```

-----=====
any nullable residual in final frontier = false -> no full parse tree
exists

```

PARTIAL RECOVERY

No prefix matched.

ERROR SUMMARY

Failure at position: 1 (1-indexed)
Found: 'b'
Expected: 'a'
b
^

END OF REPORT
